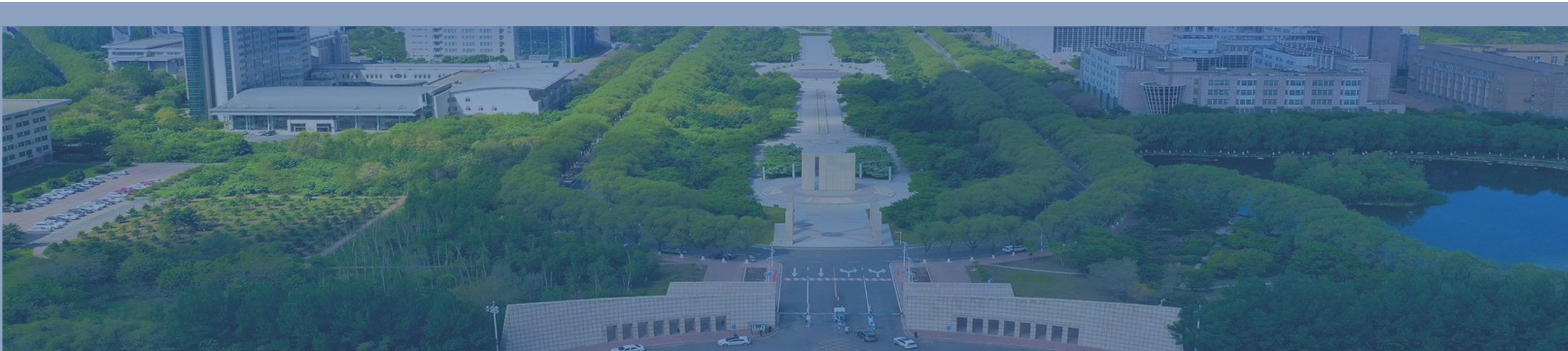


吉林大学



第四章 分治法



分治法的求解思想

- 思想：将整个问题分成若干个小问题后分而治之。
 - 分(Divide)：子问题与原问题具有相同的特征，但规模更小。
 - 治(Conquer)：反复使用分治策略，直到可以直接求解子问题为止。
 - 合(Combine)：将子问题的解组合成原问题的解。

分治法天然适
合递归实现



二分查找

```
BINSRCH(A, n, x, j)
  low ← 1; high ← n
  while low ≤ high do
    mid ←  $\lfloor (low + high) / 2 \rfloor$  //取中间值
    case
      :x < A(mid): high ← mid - 1 //前半
      :x > A(mid): low ← mid + 1 //后半
      :else: j ← mid; return; //查找成功结束
    end
  end
  j ← 0 //查找失败
end BINSRCH
```

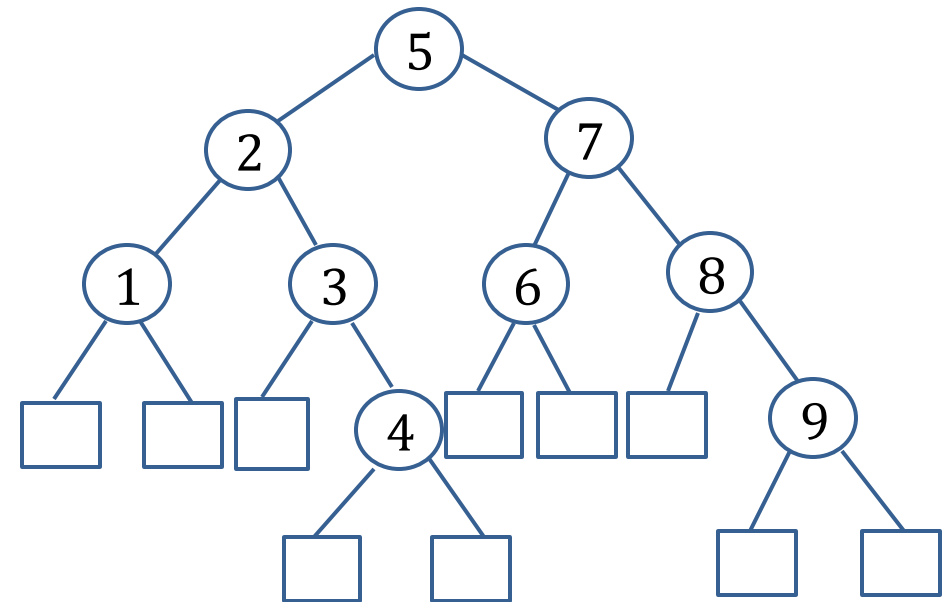


二元比较树

- 描述二分查找算法执行过程的二元比较树由内结点和外结点组成
 - 内结点记录一个mid值，表示一次元素的比较
 - 外结点表示不成功查找的一种情况
 - 一条路径表示一个元素的比较序列

$n=9$

A[1]	A[2]	A[3]	A[4]	A[5]	A[6]	A[7]	A[8]	A[9]
-15	-6	0	7	9	23	54	82	101



算法BINSRCH的时间复杂度和空间复杂度

• 时间

计算时间	最好情况	最坏情况	平均情况
成功查找	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
失败查找	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$

• 空间

- 用 n 个位置存放数组 A ，还有 $low, high, mid, x, j$ 五个变量需要存储，共需空间 $n+5 = \Theta(n)$



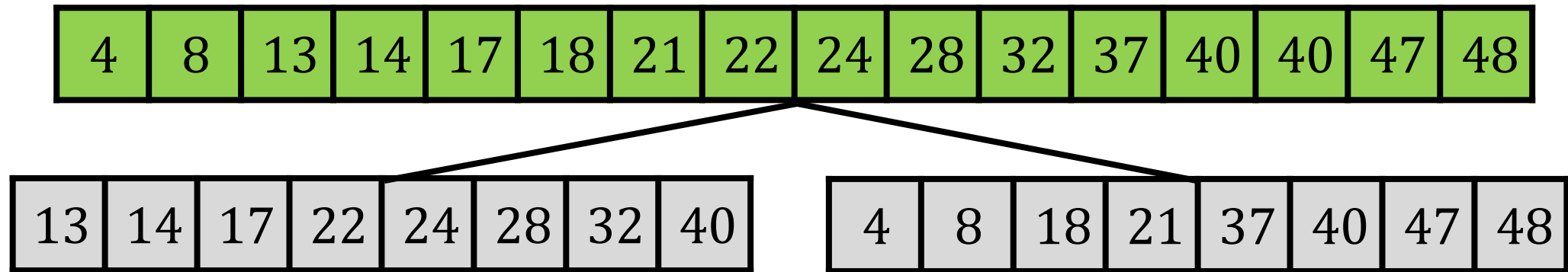
以比较为基础查找的时间下界

定理4.2: 设 $A[1..n]$ 含有 $n(n \geq 1)$ 个不同的元素, 排序为 $A(1) \leq A(2) \leq \dots \leq A(n)$ 。又设以比较为基础去判断是否 $x \in A(1:n)$ 的任何算法在最坏情况下所需的最小比较次数是 $\text{FIND}(n)$, 那么 $\text{FIND}(n) \geq \lceil \log(n+1) \rceil$

- 证明: 通过考察模拟求解查找问题的各种可能算法的比较树可知, $\text{FIND}(n)$ 为树中由根到一个叶子的最长路径的距离。在这所有的树中都必定有 n 个内节点与 x 在 A 中可能的 n 种出现情况相对应。如果一棵二元树的所有内节点所在的级数小于或等于级数 k , 则最多有 $2^k - 1$ 个内节点。因此 $n \leq 2^k - 1$, 即 $\text{FIND}(n) = k \geq \lceil \log(n+1) \rceil$ 。证毕。

不存在其最坏情况下计算时间低于 $\Theta(\log n)$ 的算法。
二分查找是解决查找问题最坏情况的最优算法

归并排序



● 算法流程

- 将数组 $A[1..n]$ 排序问题**分解**为 $A[1..\lfloor \frac{n}{2} \rfloor]$ 和 $A[\lfloor \frac{n}{2} \rfloor + 1..n]$
- **递归解决**子问题得到两个有序的子数组
- 将两个有序子数组**合并**为一个有序数组

分解原问题

解决子问题

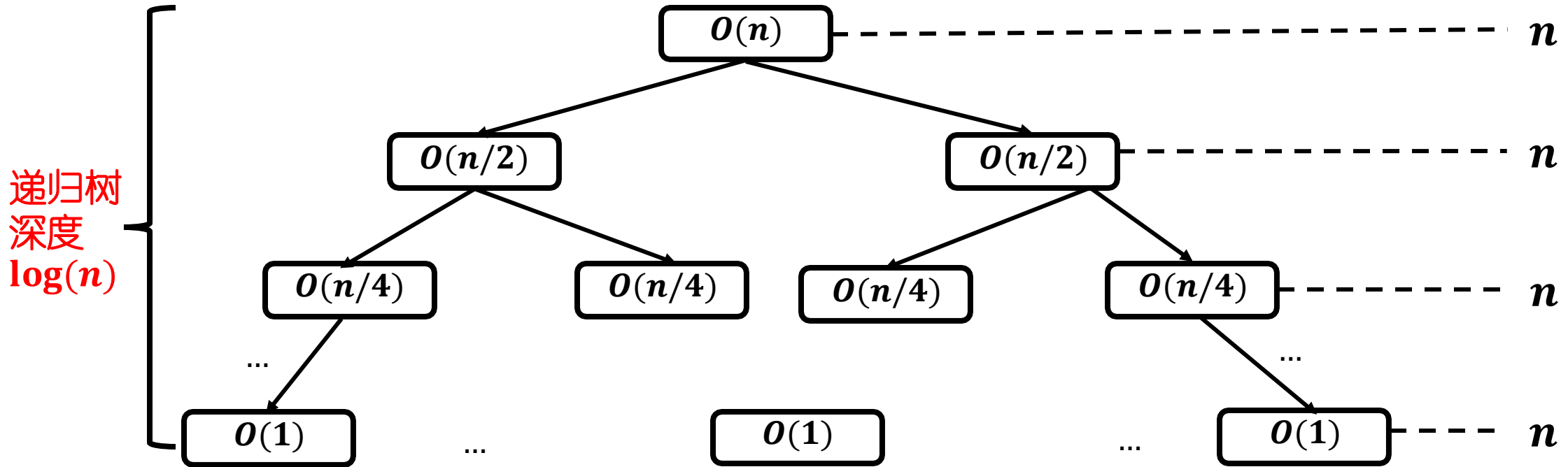
合并问题解

归并排序：分解数组，递归求解，合并排序

分而治之

归并排序：复杂度分析

- 递归树法：用树表示递归
- $$T(n) = \begin{cases} 2T(n/2) + O(n) & \text{if } n > 1 \\ O(1) & \text{if } n = 1 \end{cases}$$



$$T(n) = O(n \log n)$$



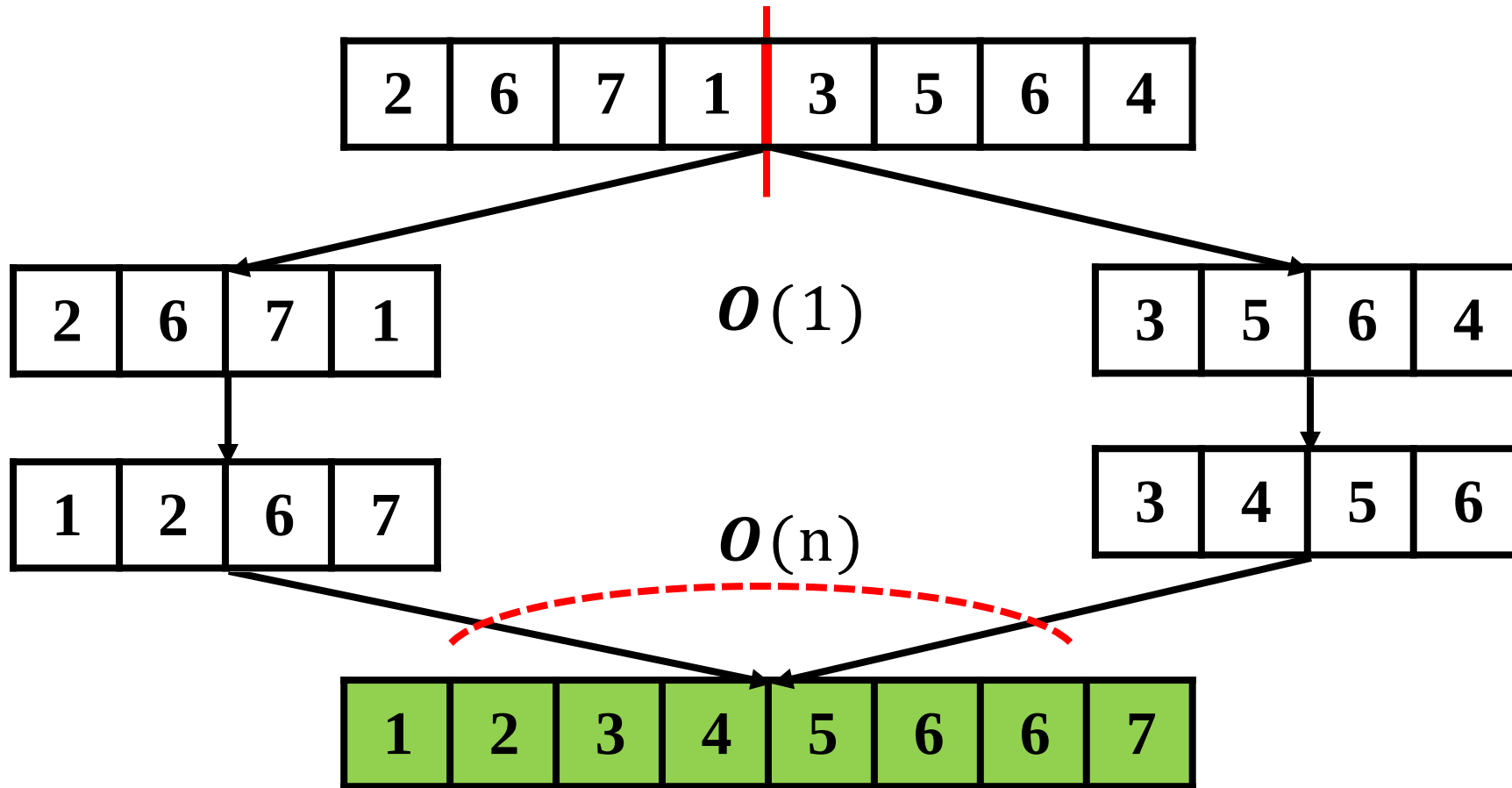
目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结





算法回顾：归并排序



分解原问题



解决子问题

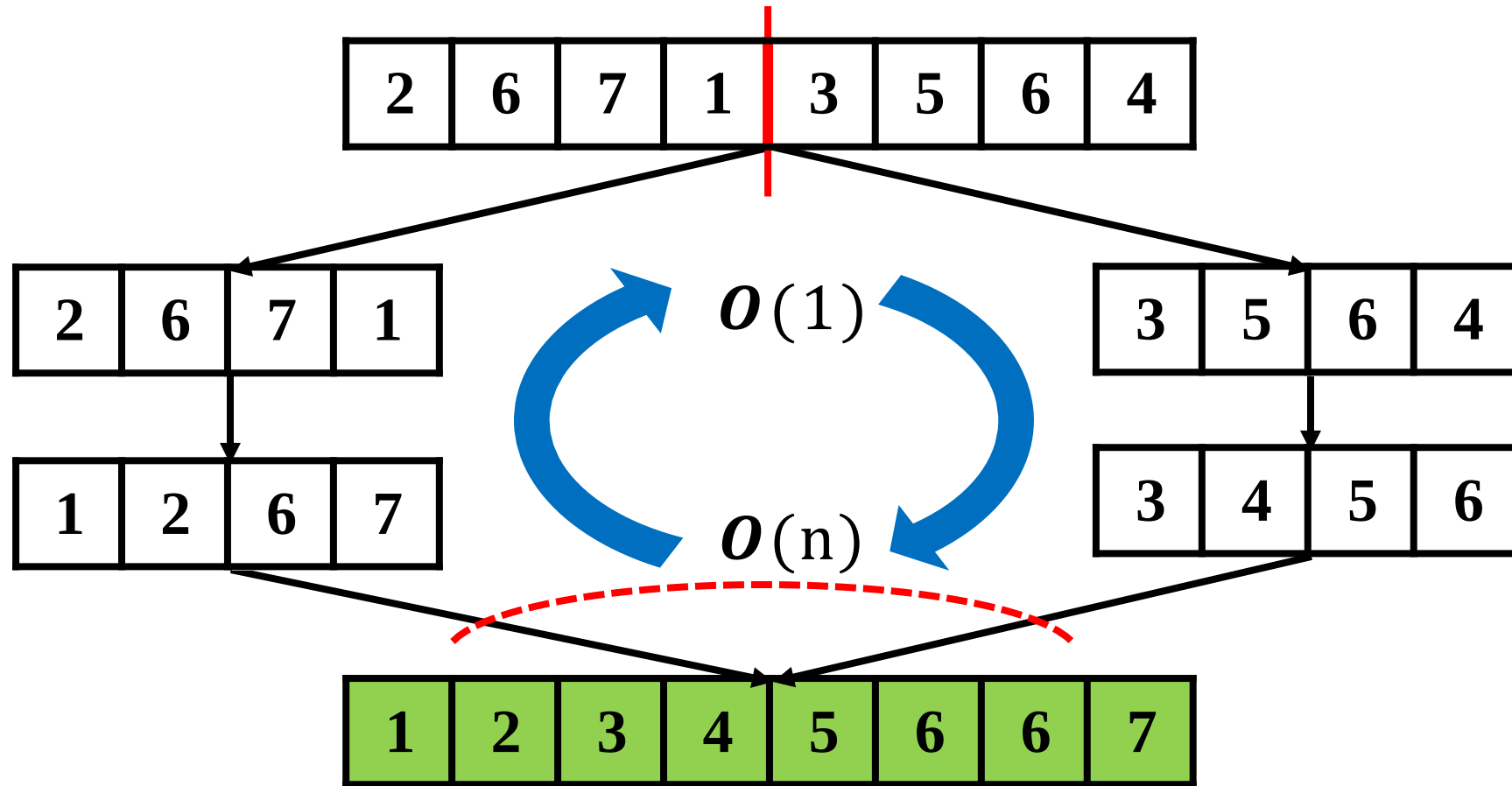


合并问题解

归并排序：简化分解，侧重合并



从归并排序到快速排序



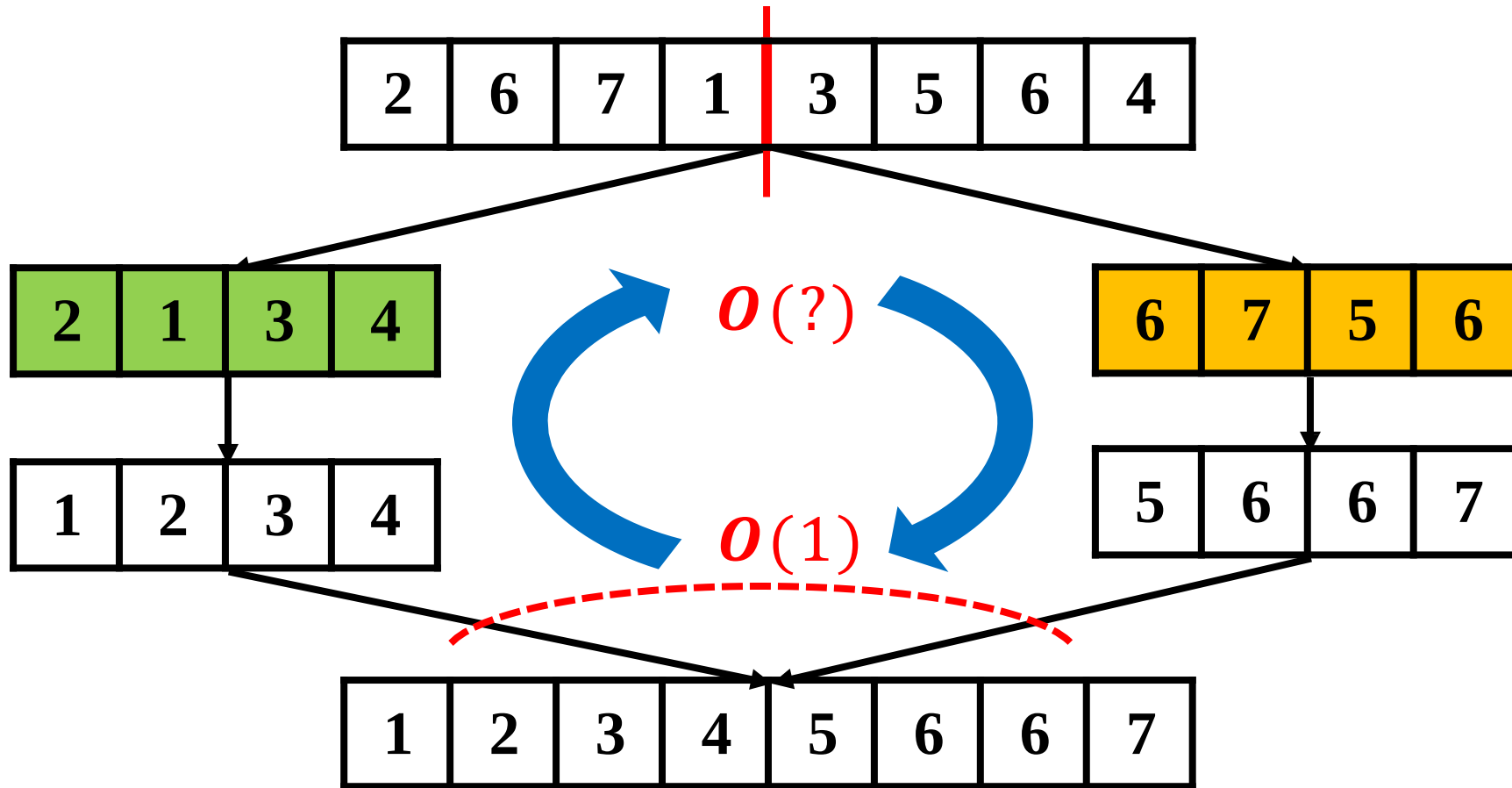
分解原问题

解决子问题

合并问题解



从归并排序到快速排序



分解原问题

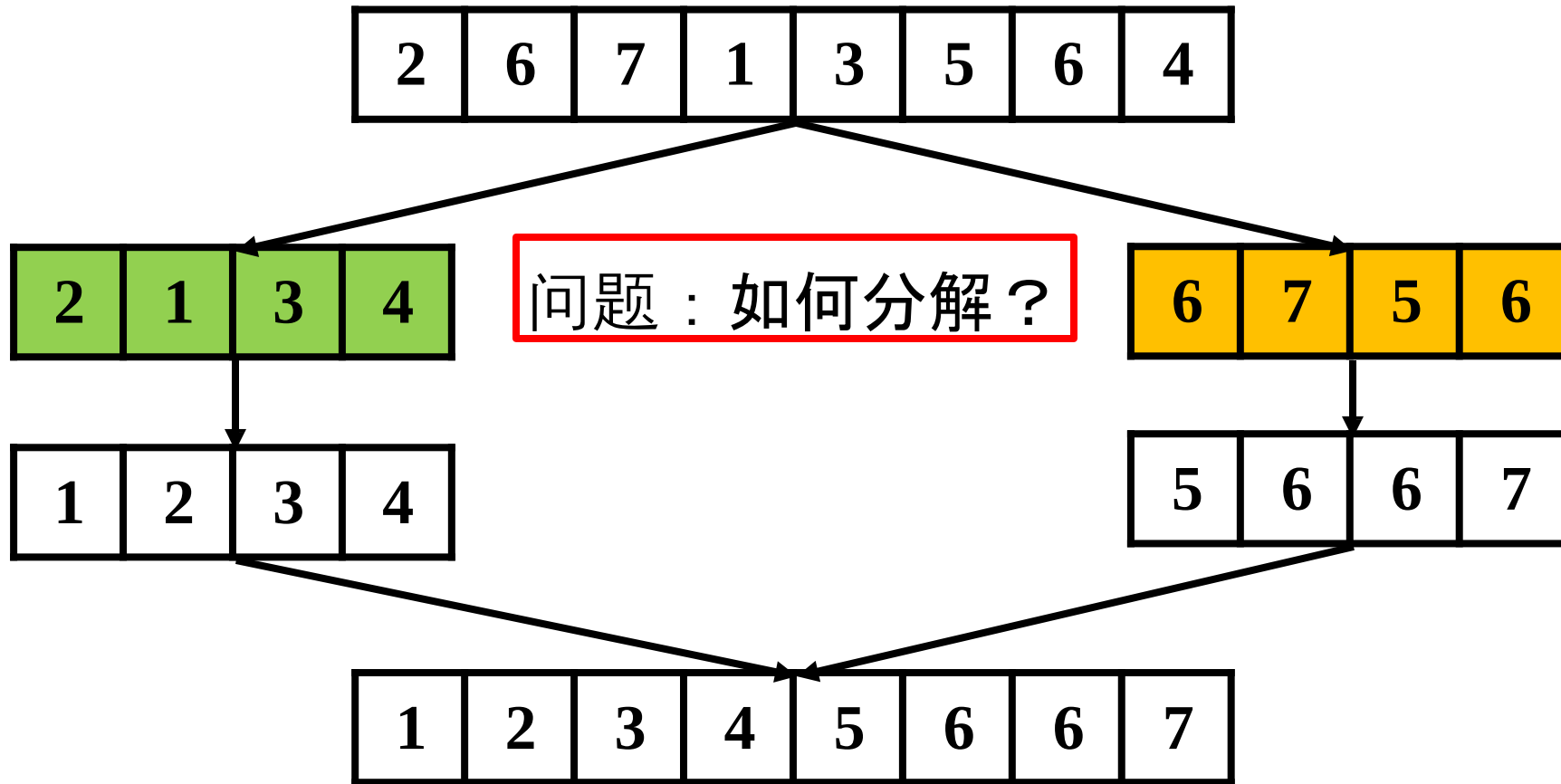
解决子问题

合并问题解

快速排序：侧重分解，简化合并



从归并排序到快速排序



分解原问题

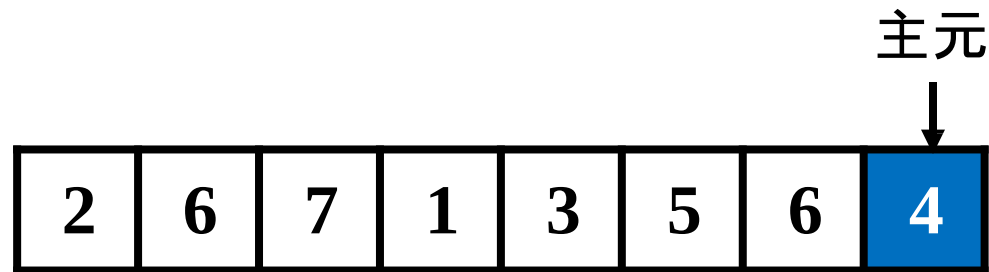
解决子问题

合并问题解

快速排序：侧重分解，简化合并

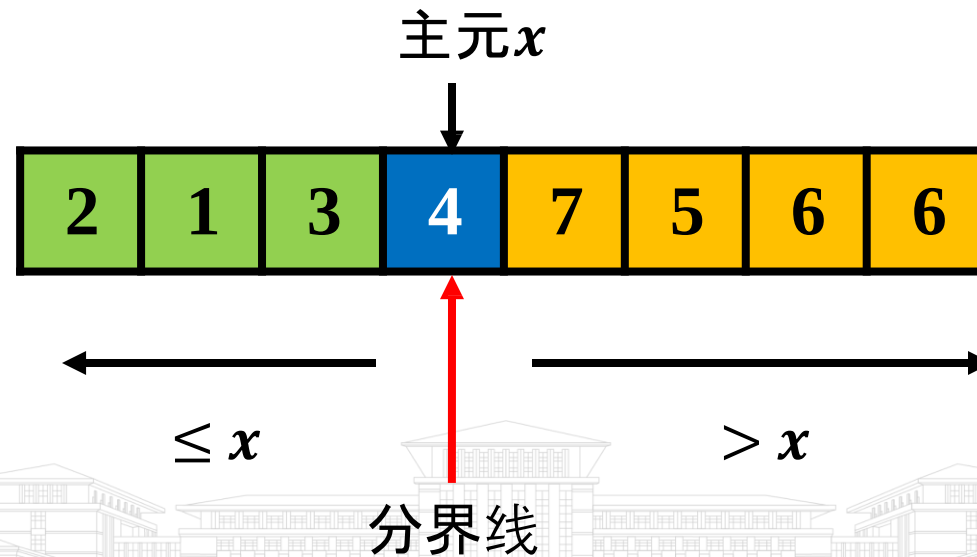
数组划分

- 基本思想
 - 任选元素 x 作为分界线，称为主元(pivot)



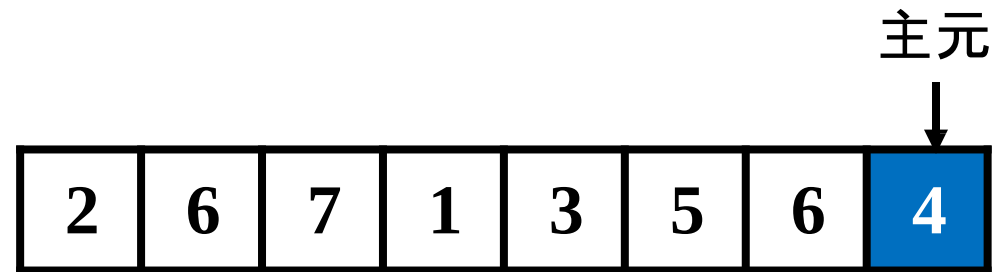
数组划分

- 基本思想
 - 任选元素 x 作为分界线，称为主元(pivot)
 - 交换重排，满足 x 左侧元素小于右侧



数组划分

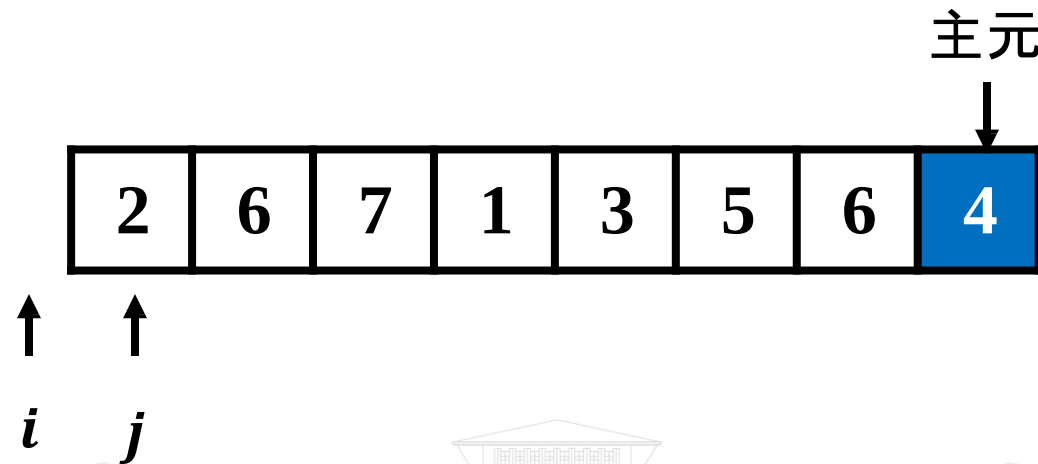
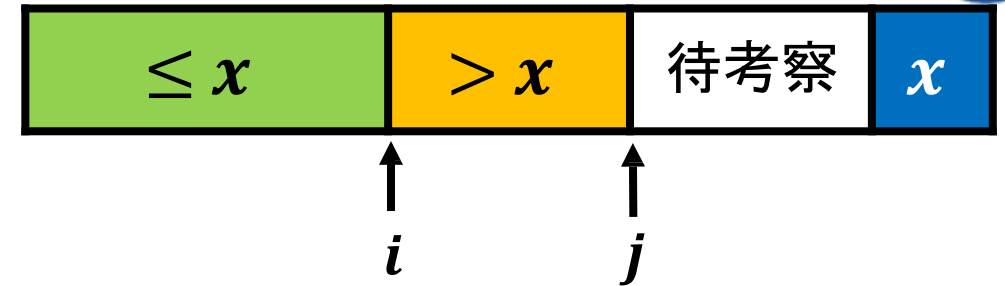
- 实现方法
 - 选取固定位置主元 x （如尾元素）



数组划分

- 实现方法

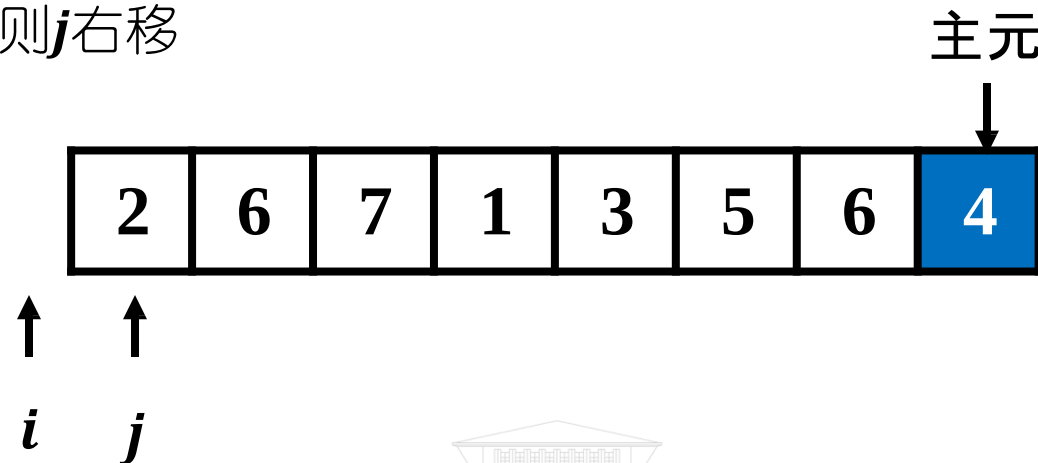
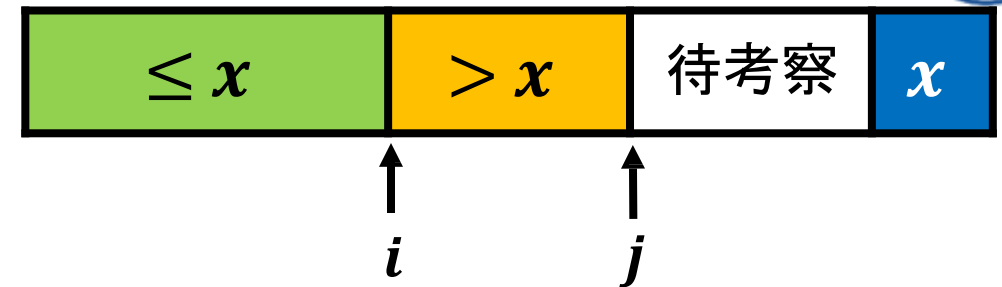
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较



数组划分

• 实现方法

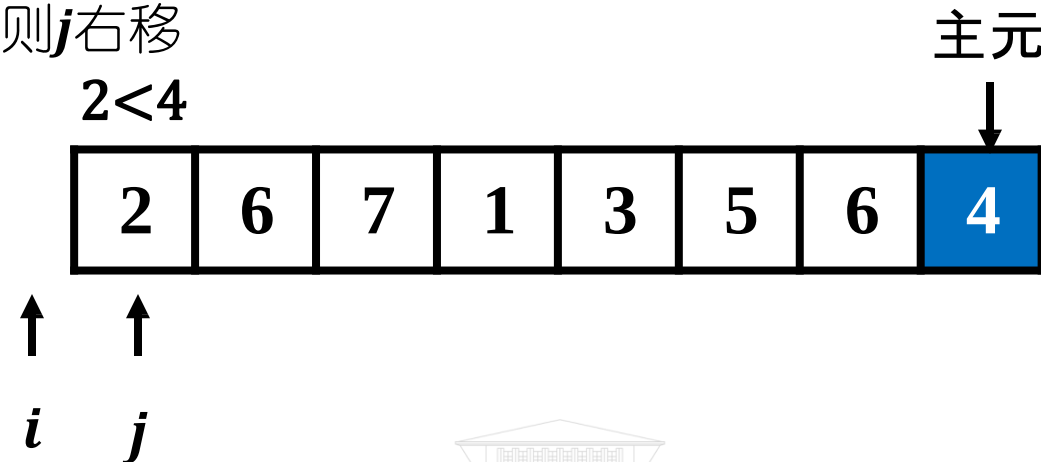
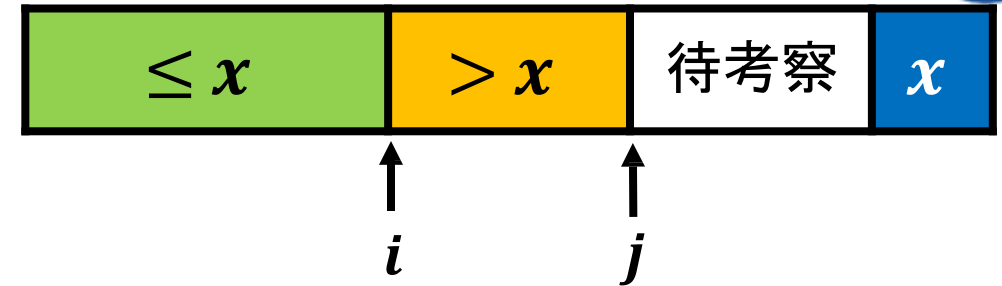
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

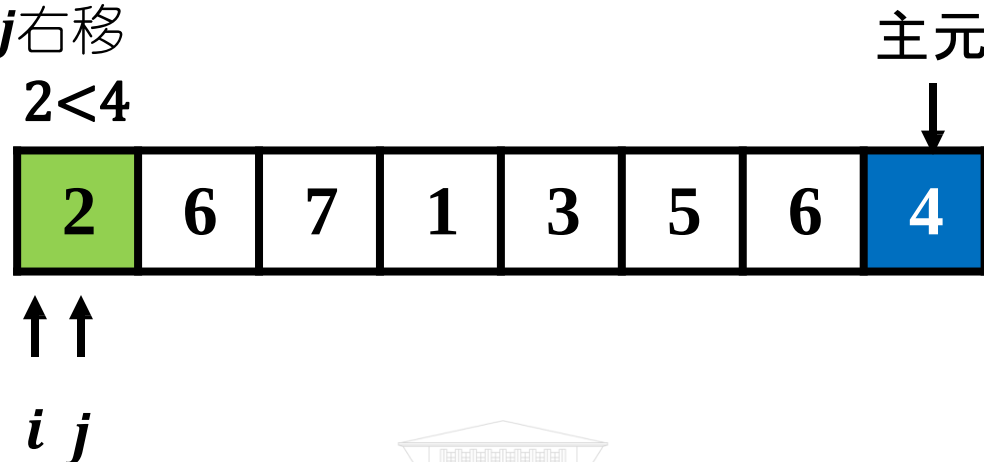
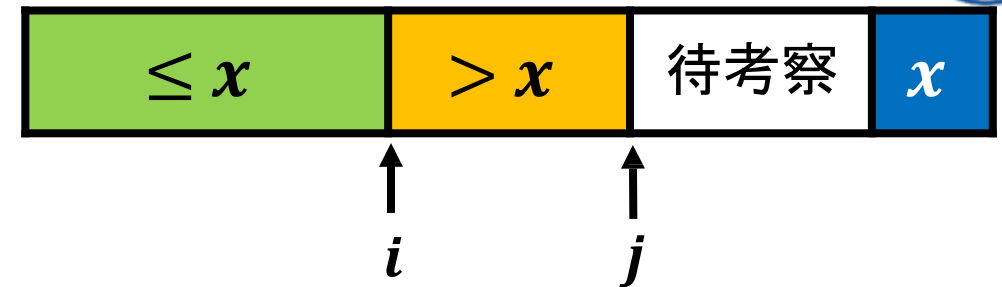
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

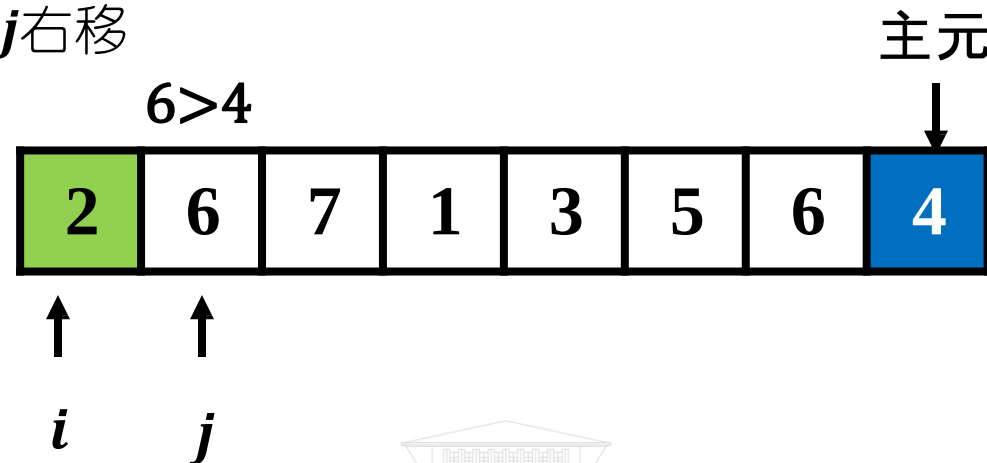
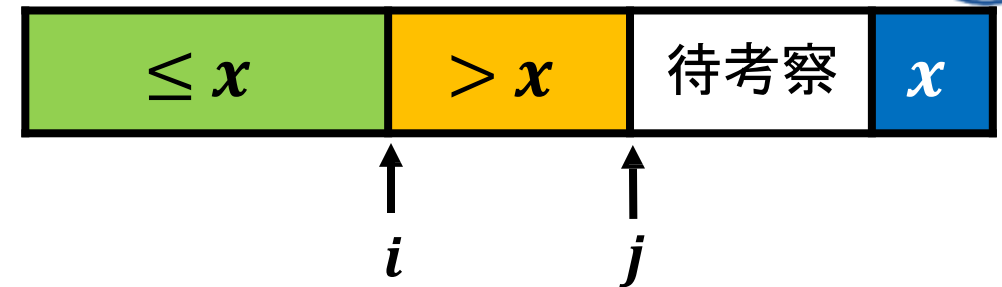
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

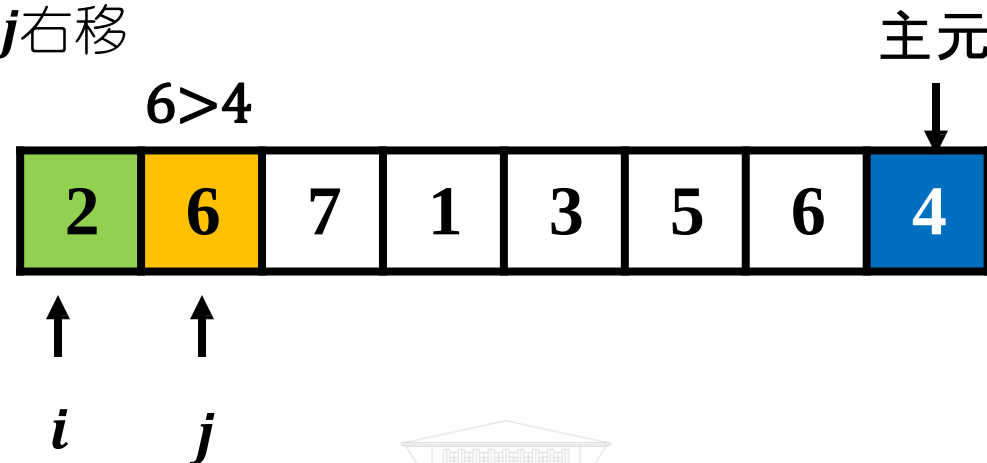
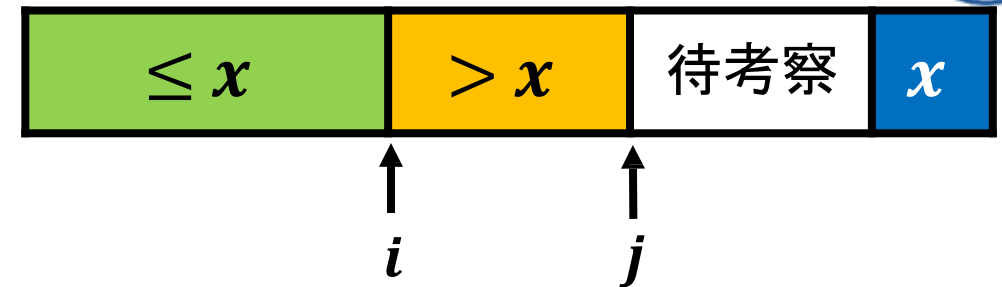
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

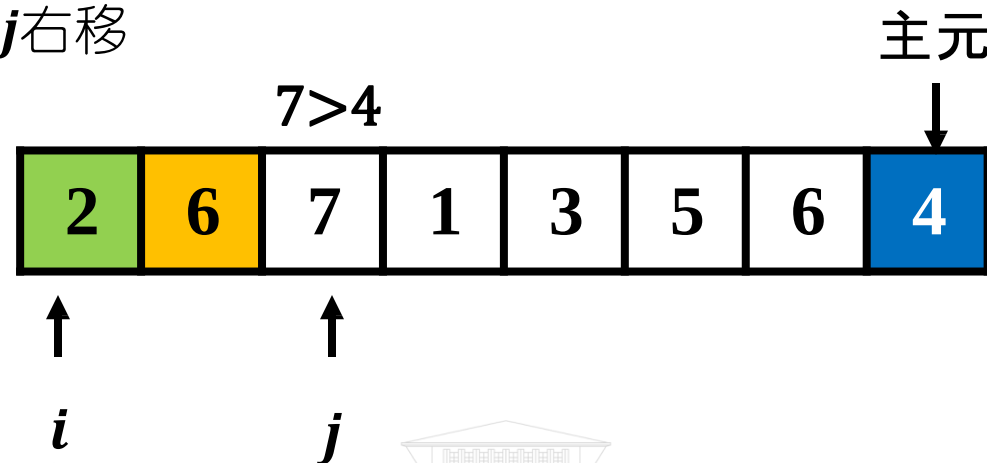
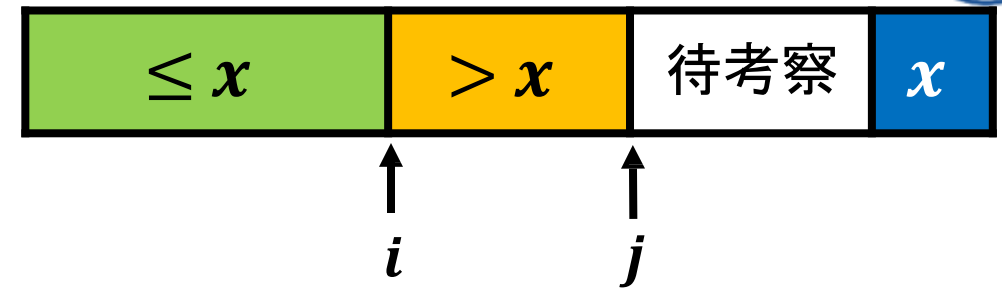
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

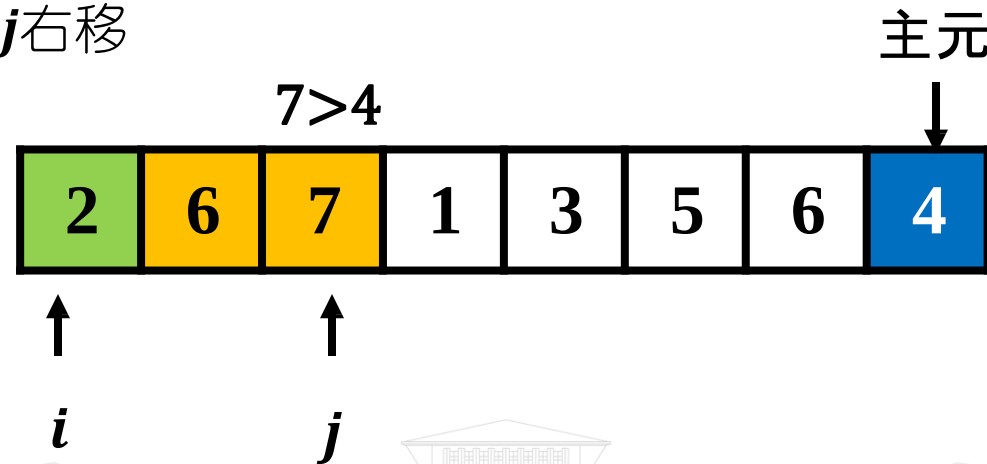
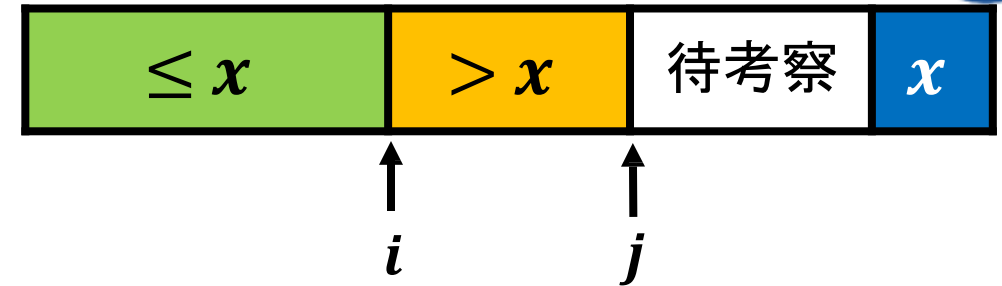
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

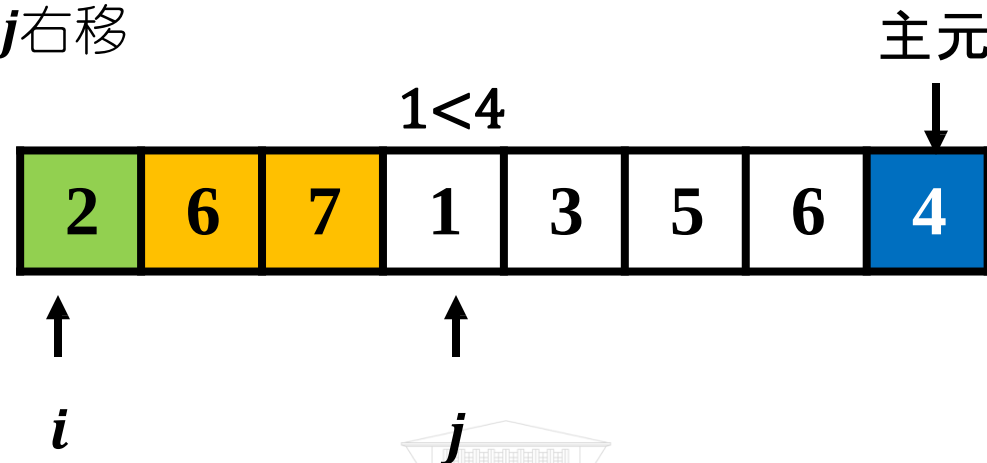
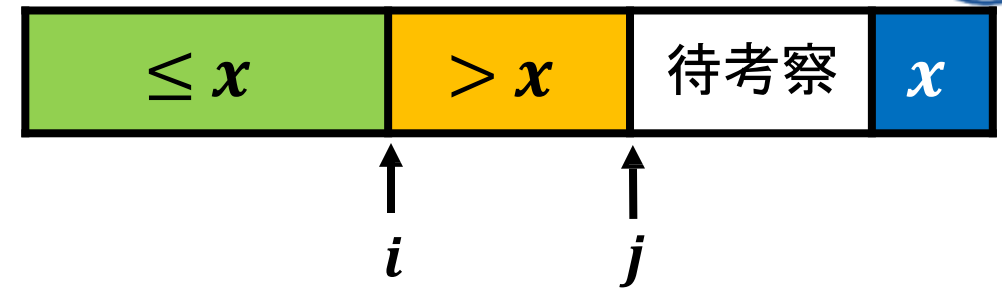
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

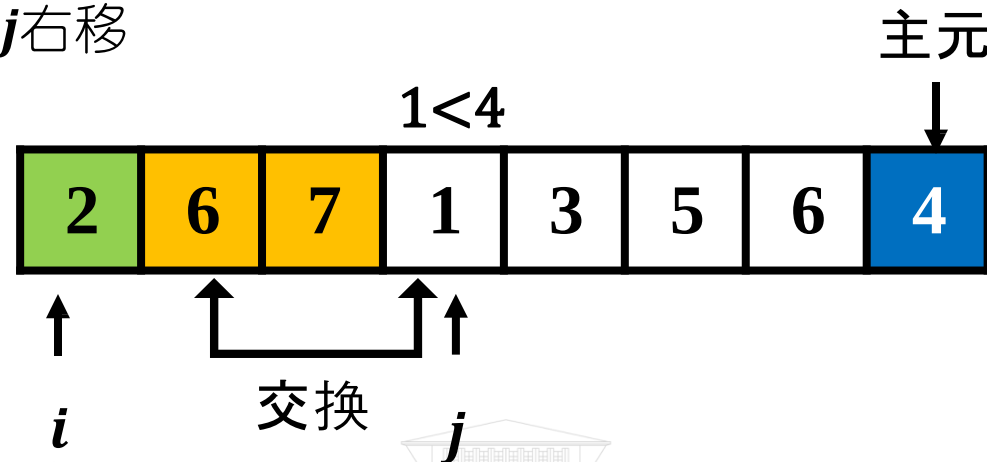
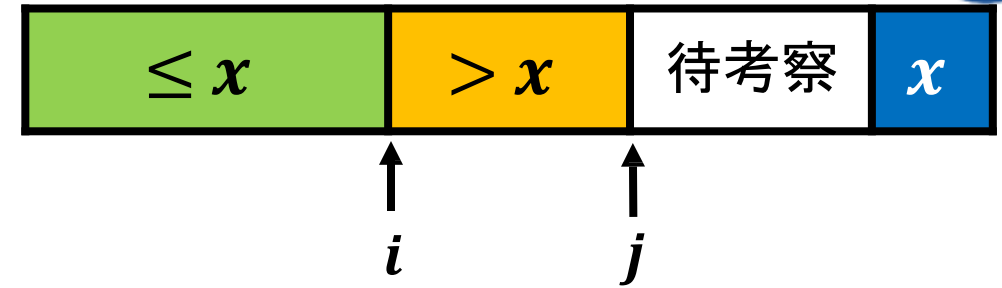
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

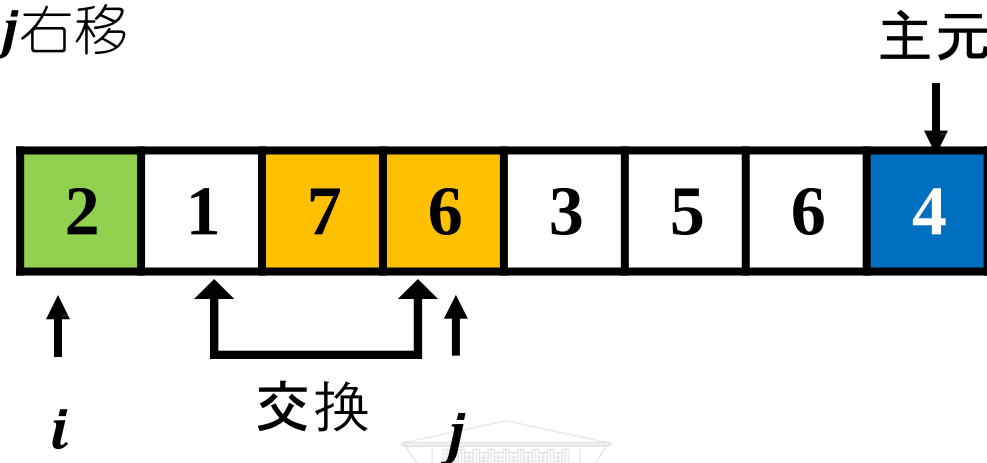
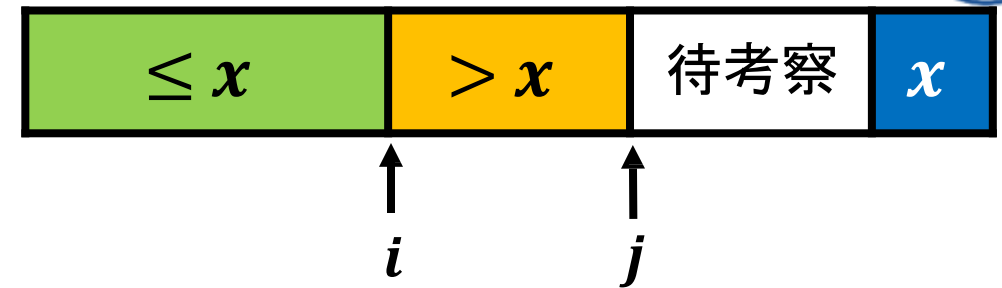
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

- 实现方法

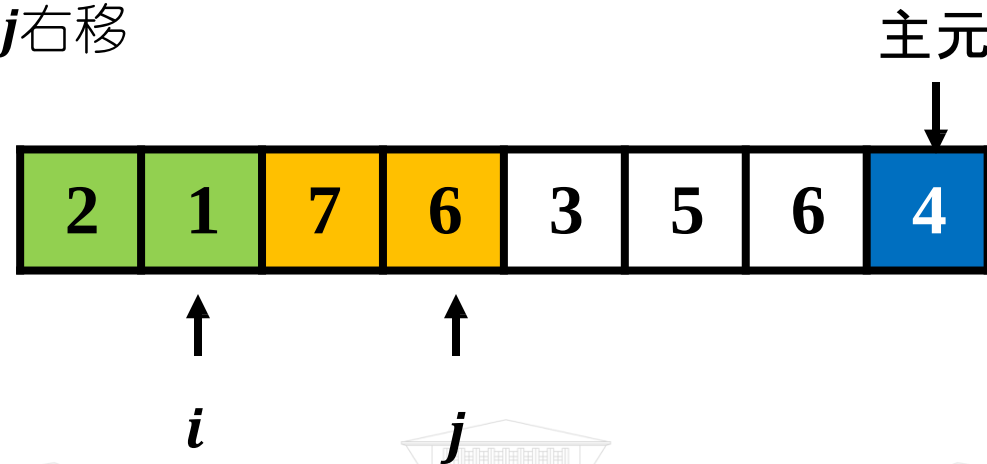
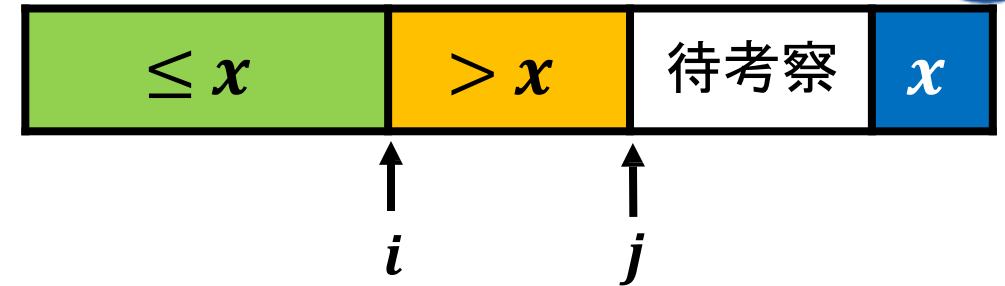
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

- 实现方法

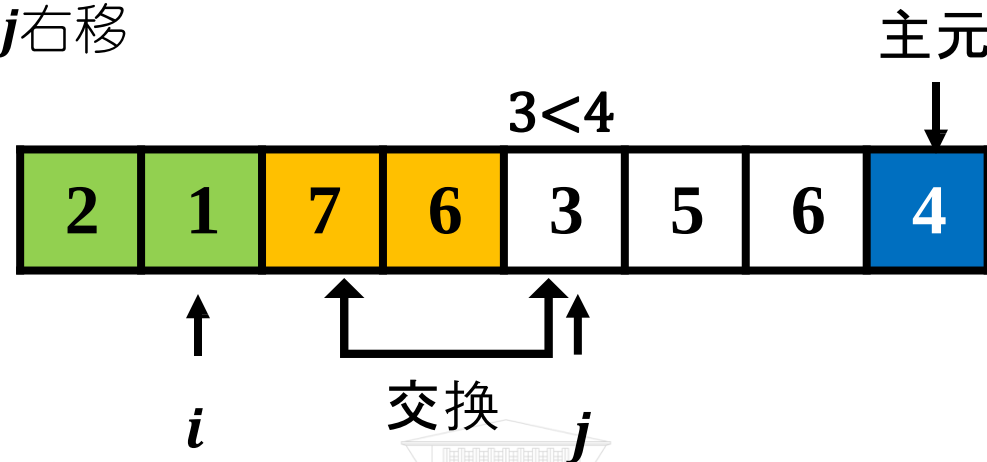
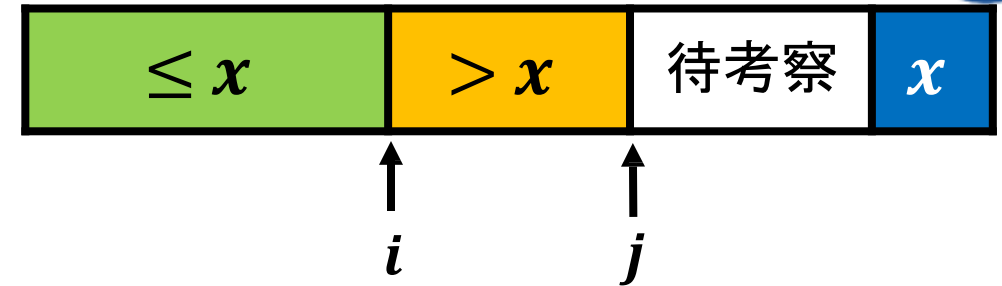
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

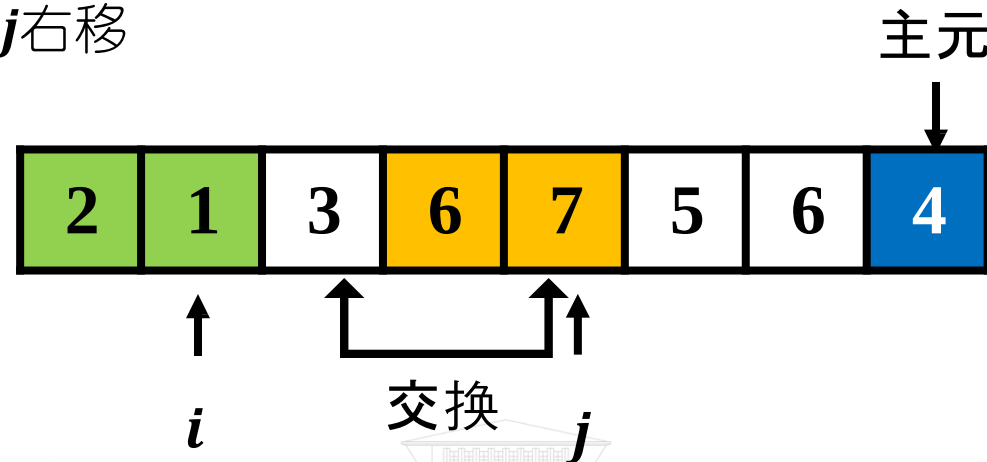
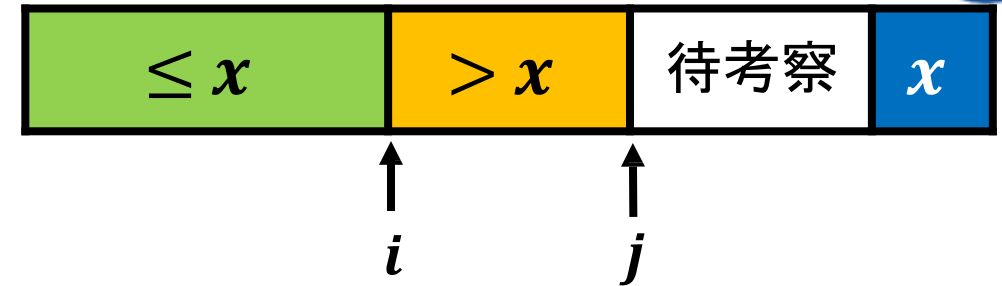
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

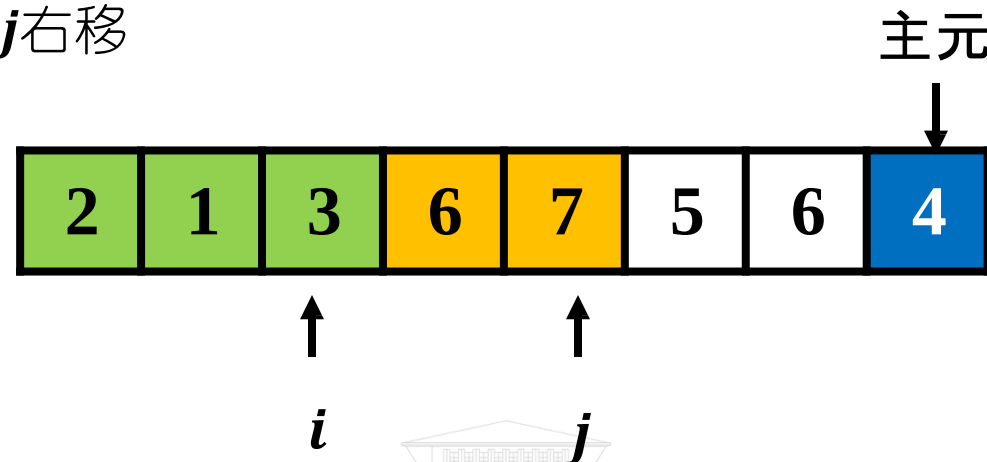
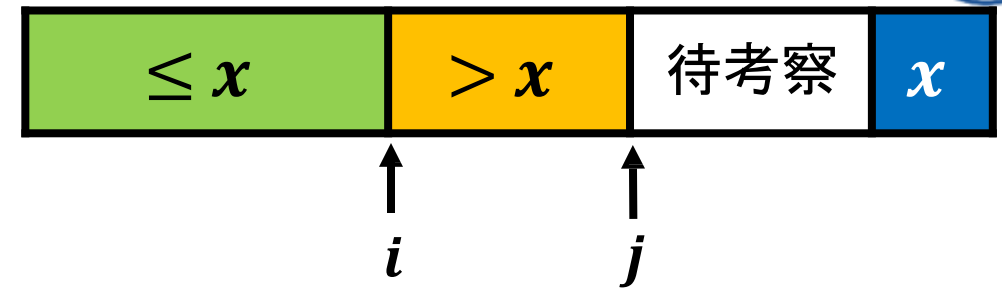
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

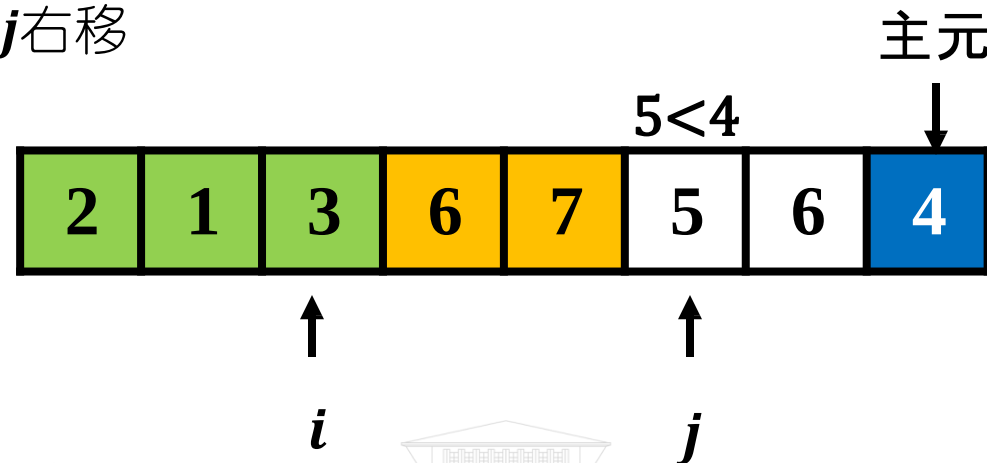
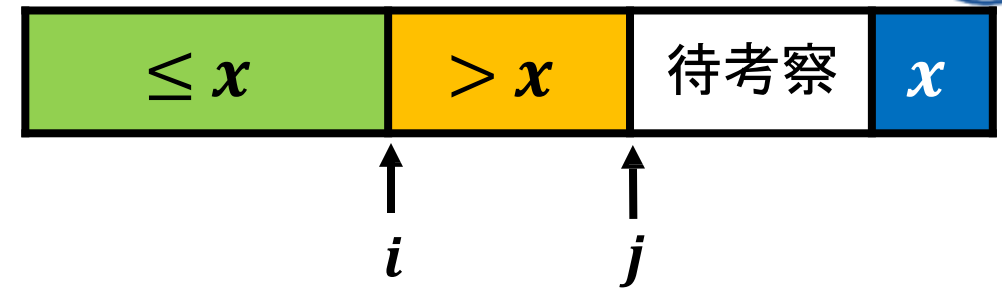
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

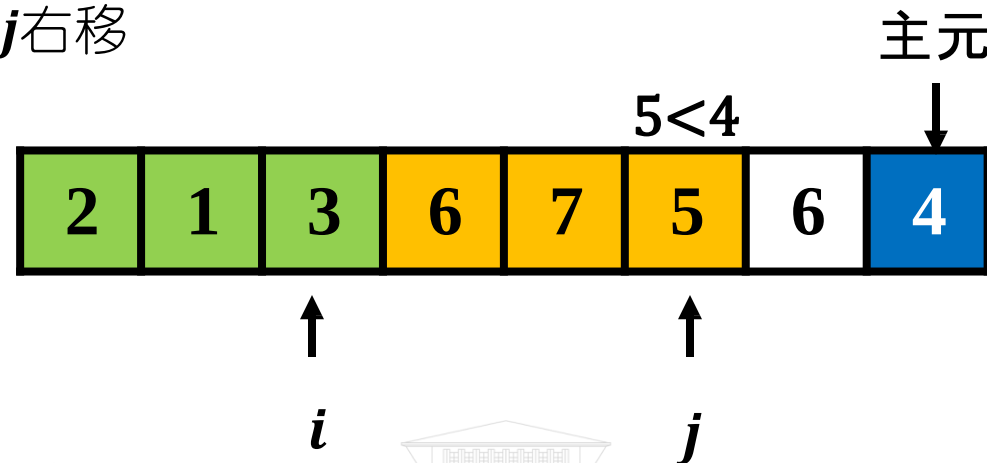
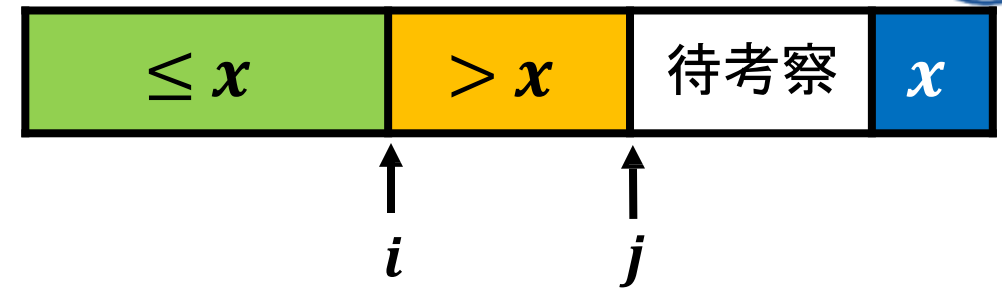
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

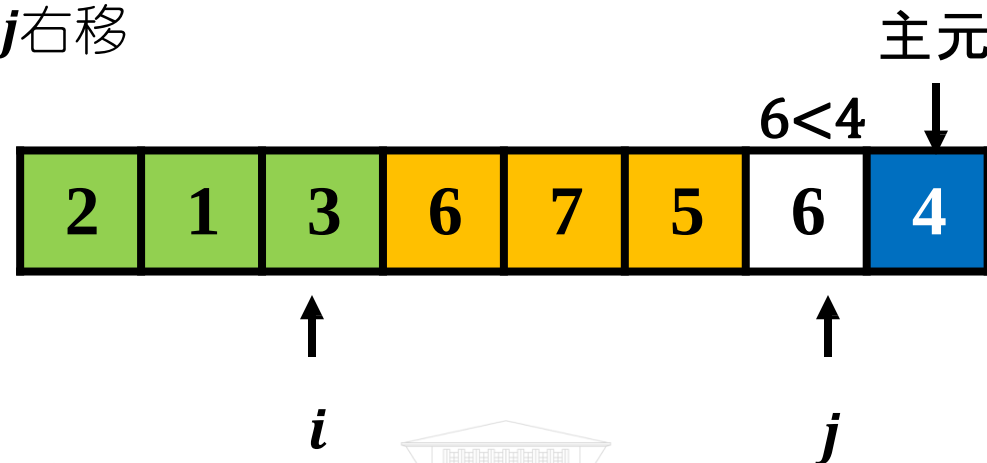
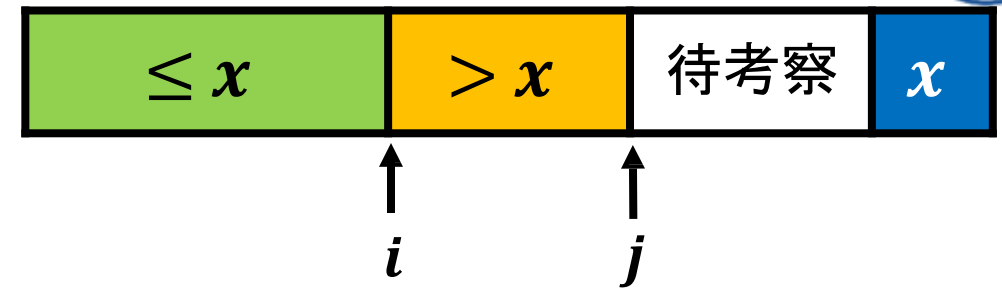
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

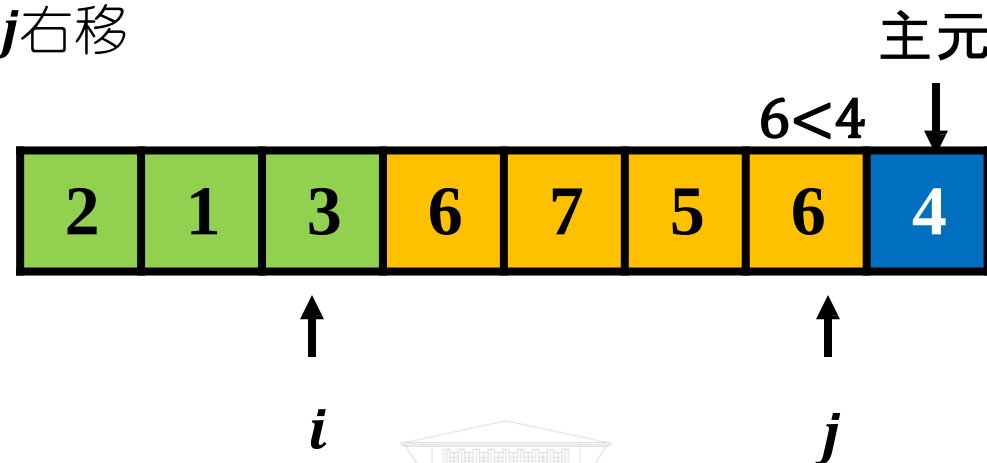
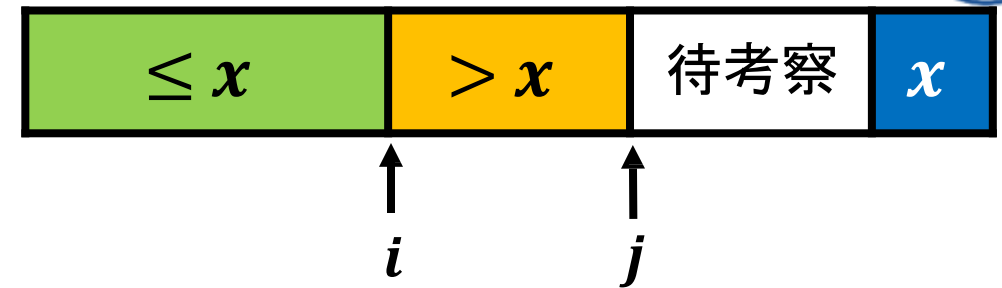
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

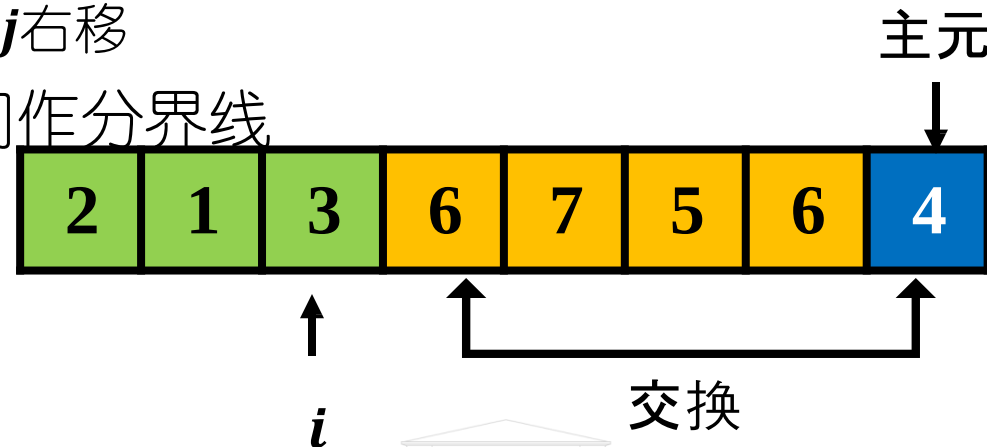
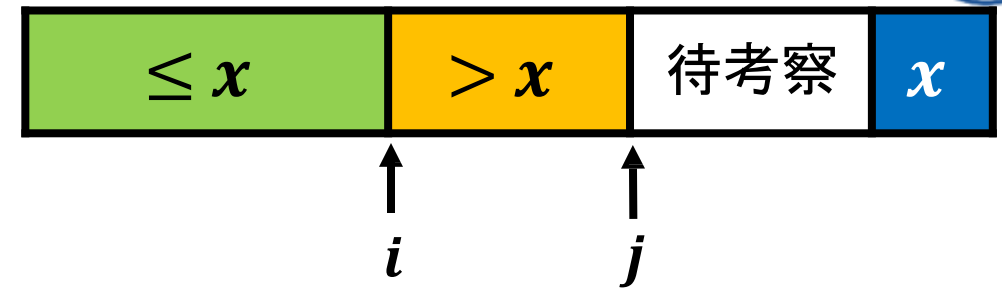
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移



数组划分

• 实现方法

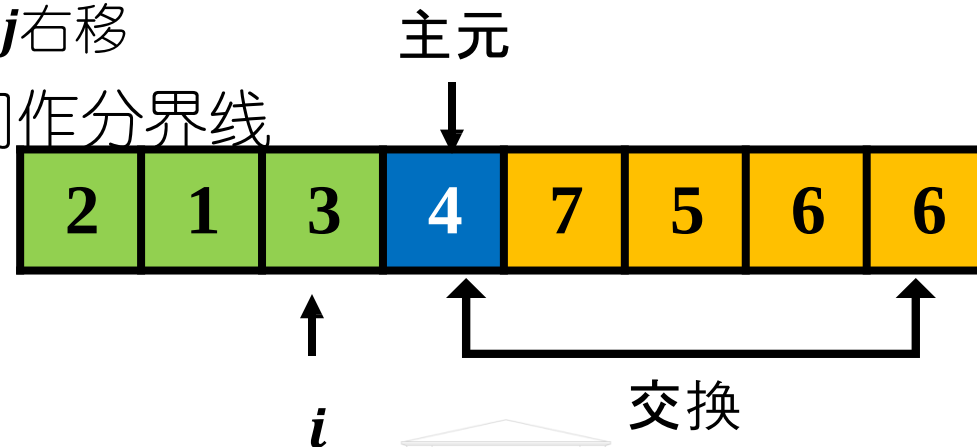
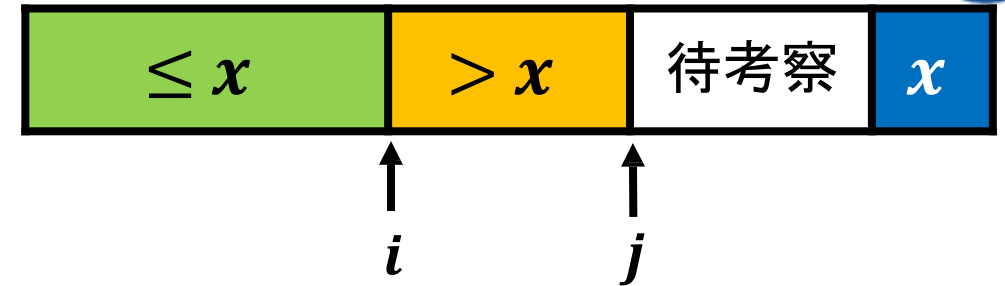
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移
- 把主元放在中间作分界线



数组划分

• 实现方法

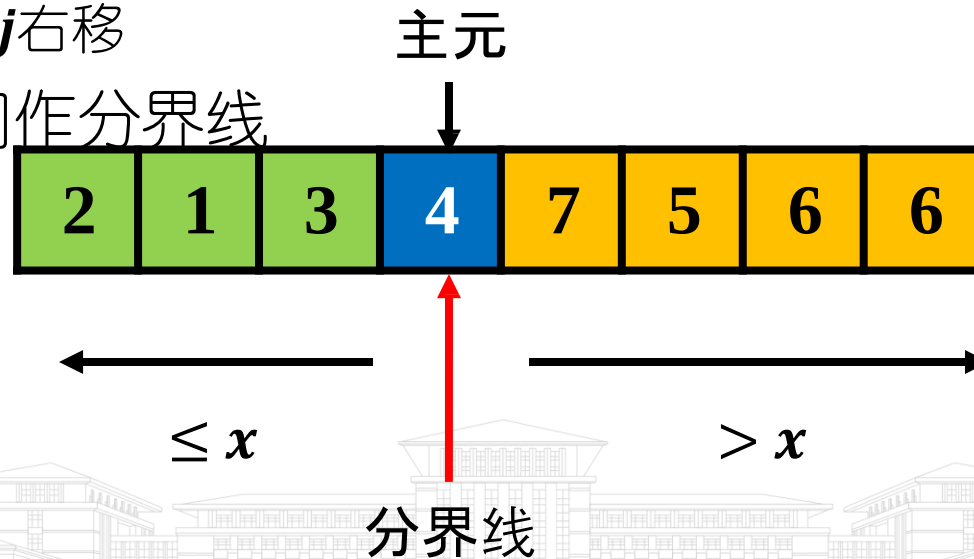
- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移
- 把主元放在中间作分界线



数组划分

- 实现方法

- 选取固定位置主元 x （如尾元素）
- 维护两个部分的右端点变量 i, j
- 考察数组元素 $A[j]$ ，只和主元比较
 - 若 $A[j] \leq x$ ，则交换 $A[j]$ 和 $A[i + 1]$ ， i, j 右移
 - 若 $A[j] > x$ ，则 j 右移
- 把主元放在中间作分界线



数组划分：伪代码

● Partition(A, p, r)

输入：数组 A ，起始位置 p ，终止位置 r

输出：划分位置 q

$x \leftarrow A[r]$

选取主元

$i \leftarrow p - 1$

for $j \leftarrow p$ **to** $r - 1$ **do**

if $A[j] \leq x$ **then**

 exchange $A[i + 1]$ with $A[j]$

$i \leftarrow i + 1$

end

end

exchange $A[i + 1]$ with $A[r]$

$q \leftarrow i + 1$

return q



数组划分：伪代码

● Partition(A, p, r)

输入：数组 A , 起始位置 p , 终止位置 r

输出：划分位置 q

$x \leftarrow A[r]$

$i \leftarrow p - 1$

for $j \leftarrow p$ **to** $r - 1$ **do**

if $A[j] \leq x$ **then**

 exchange $A[i + 1]$ with $A[j]$

$i \leftarrow i + 1$

end

end

exchange $A[i + 1]$ with $A[r]$

$q \leftarrow i + 1$

return q

比主元小的元素交换到前面



数组划分：伪代码

● Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r

输出: 划分位置 q

$x \leftarrow A[r]$

$i \leftarrow p - 1$

for $j \leftarrow p$ **to** $r - 1$ **do**

if $A[j] \leq x$ **then**

 exchange $A[i + 1]$ with $A[j]$

$i \leftarrow i + 1$

end

end

exchange $A[i + 1]$ with $A[r]$

$q \leftarrow i + 1$

return q

主元作分界线



数组划分：复杂度分析

● Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r

输出: 划分位置 q

$x \leftarrow A[r]$

$i \leftarrow p - 1$

for $j \leftarrow p$ **to** $r - 1$ **do**

if $A[j] \leq x$ **then**

 exchange $A[i + 1]$ with $A[j]$

$i \leftarrow i + 1$

end

end

exchange $A[i + 1]$ with $A[r]$

$q \leftarrow i + 1$

return q

} $O(n)$

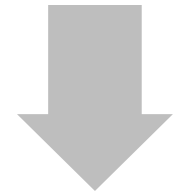
时间复杂度 : $O(n)$



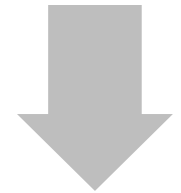
快速排序：算法框架

2	6	7	1	3	5	6	4
---	---	---	---	---	---	---	---

分解原问题



解决子问题

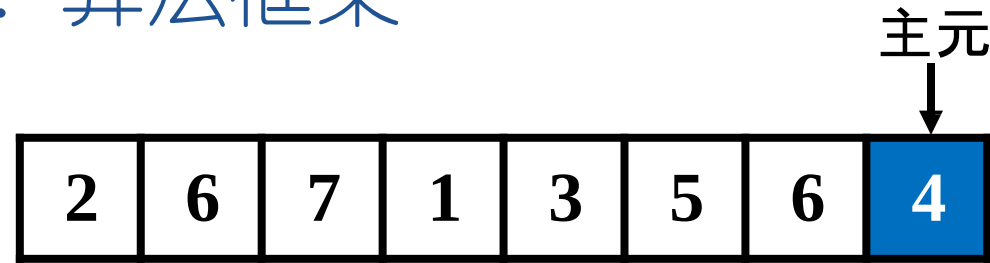


合并问题解

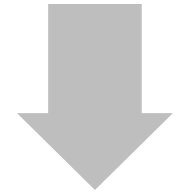




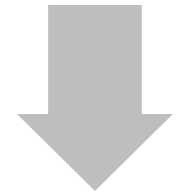
快速排序：算法框架



分解原问题



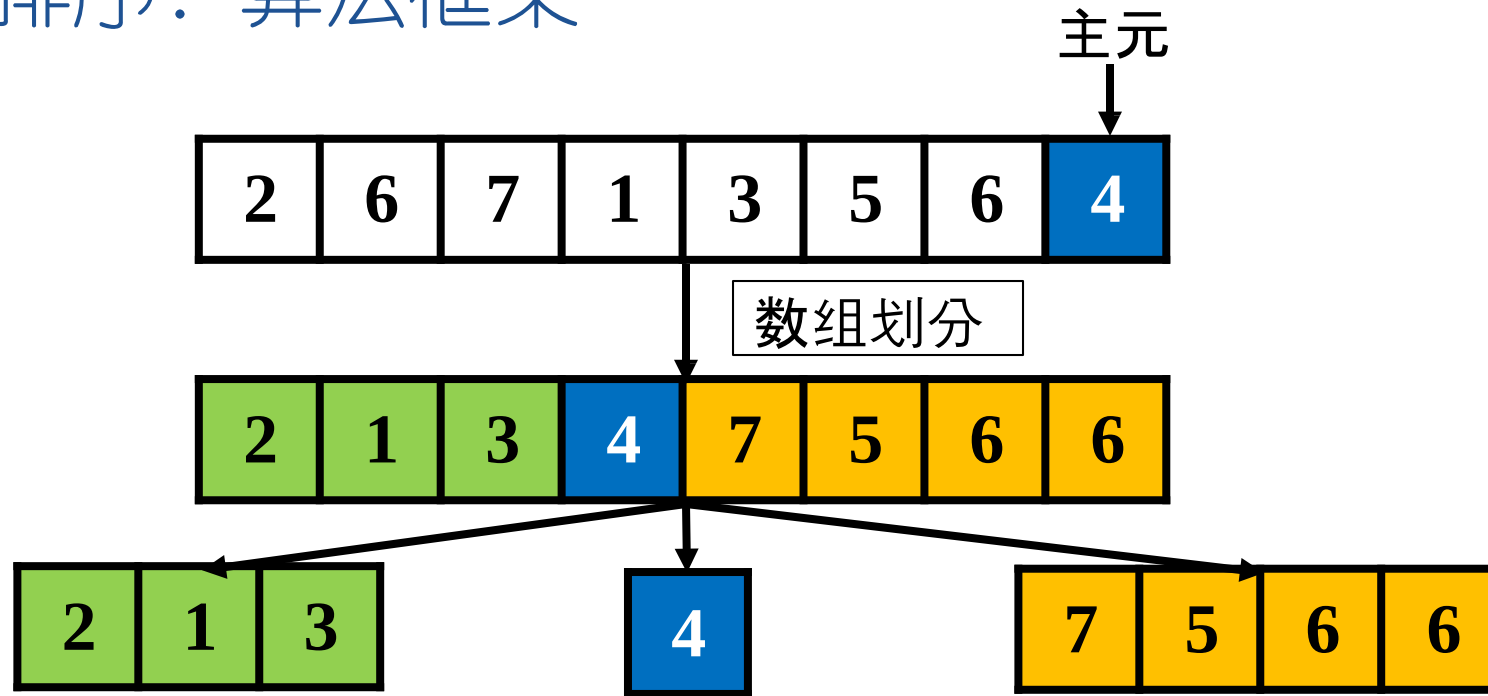
解决子问题



合并问题解



快速排序：算法框架



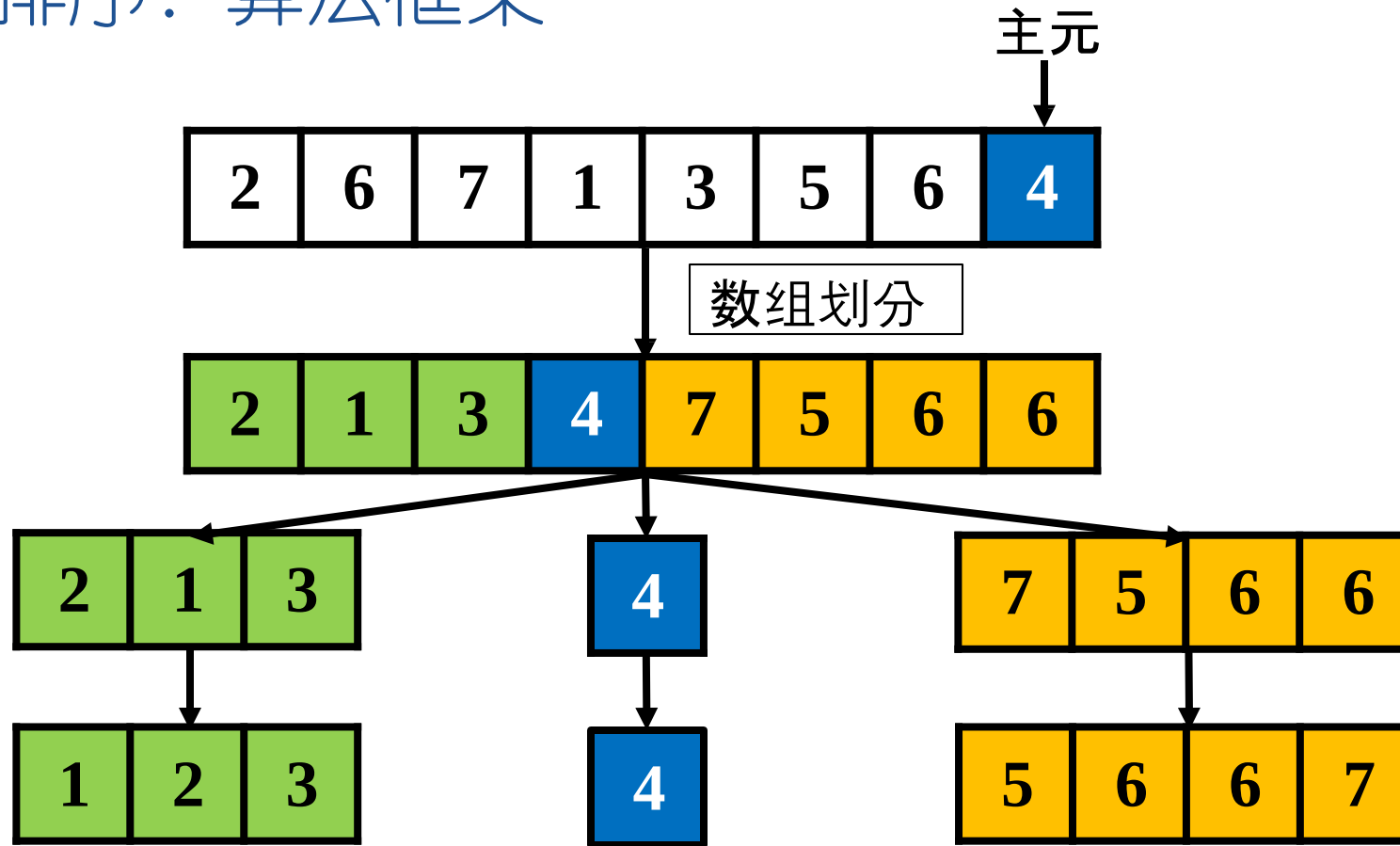
分解原问题

解决子问题

合并问题解



快速排序：算法框架



分解原问题

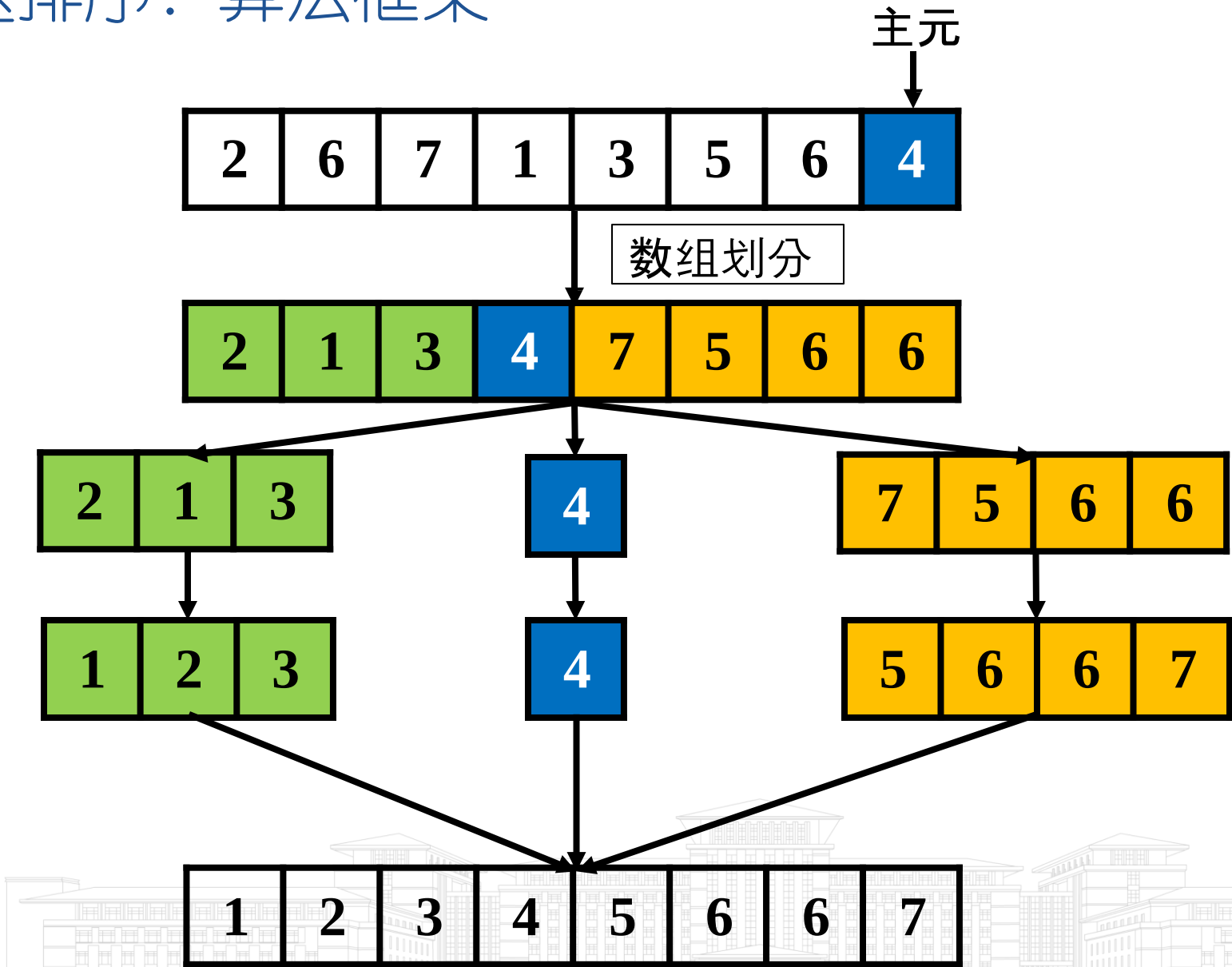
解决子问题

合并问题解





快速排序：算法框架



分解原问题

解决子问题

合并问题解

算法实例



2	6	7	1	3	5	6	4
---	---	---	---	---	---	---	---



算法实例



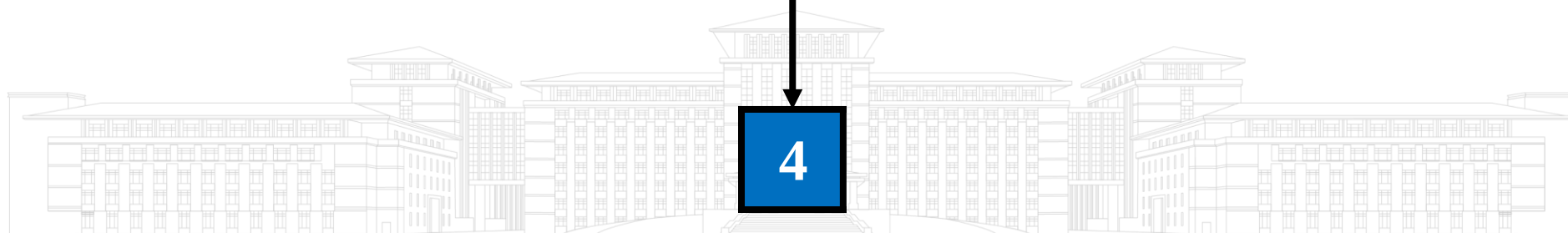
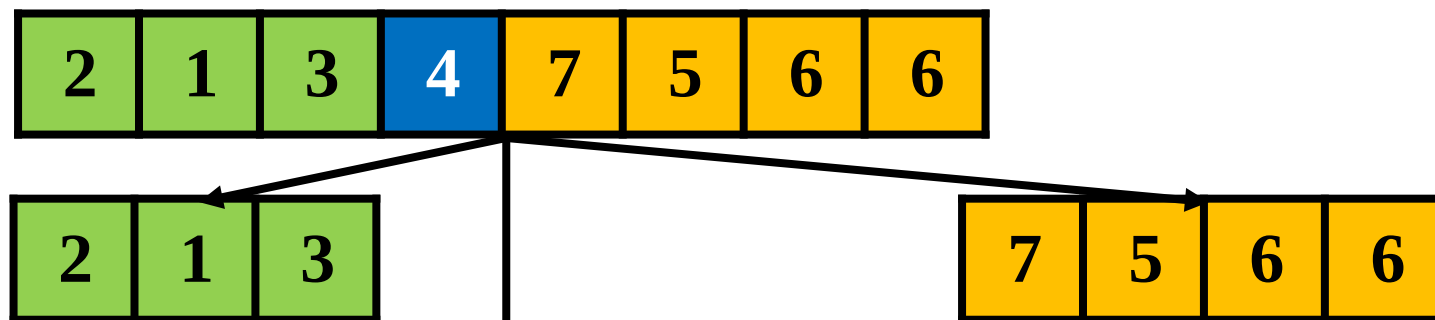
2	6	7	1	3	5	6	4
---	---	---	---	---	---	---	---



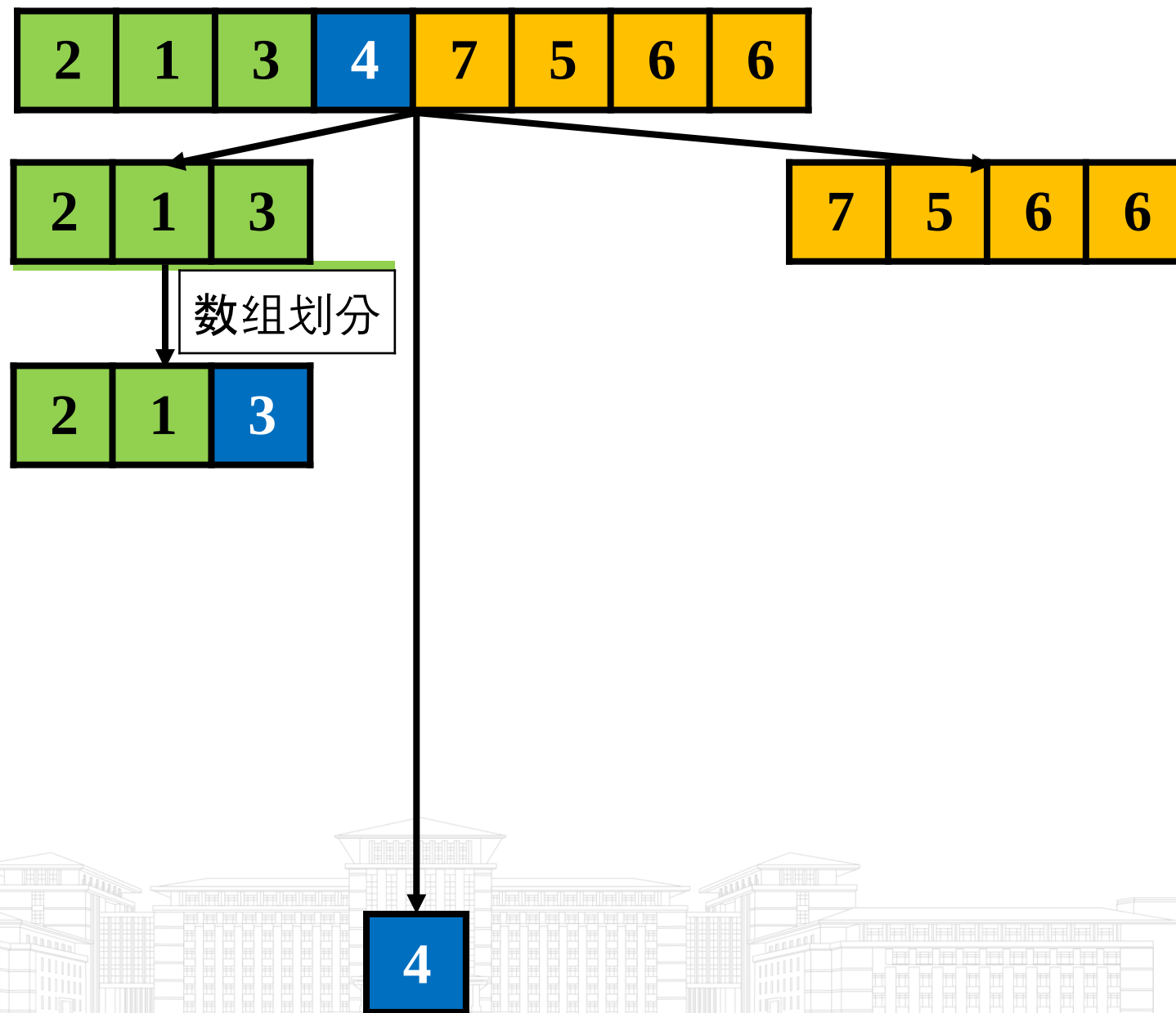
算法实例



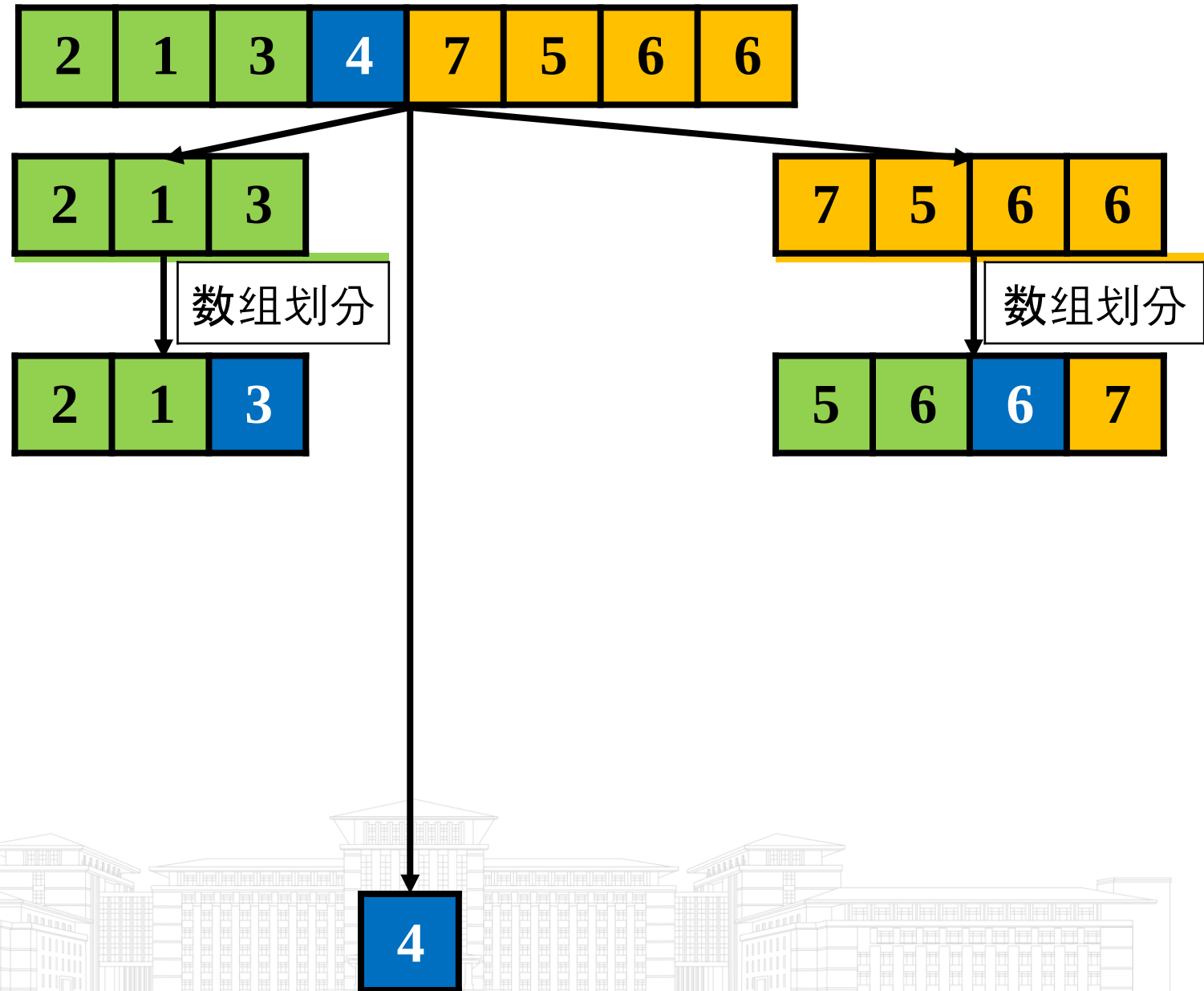
算法实例



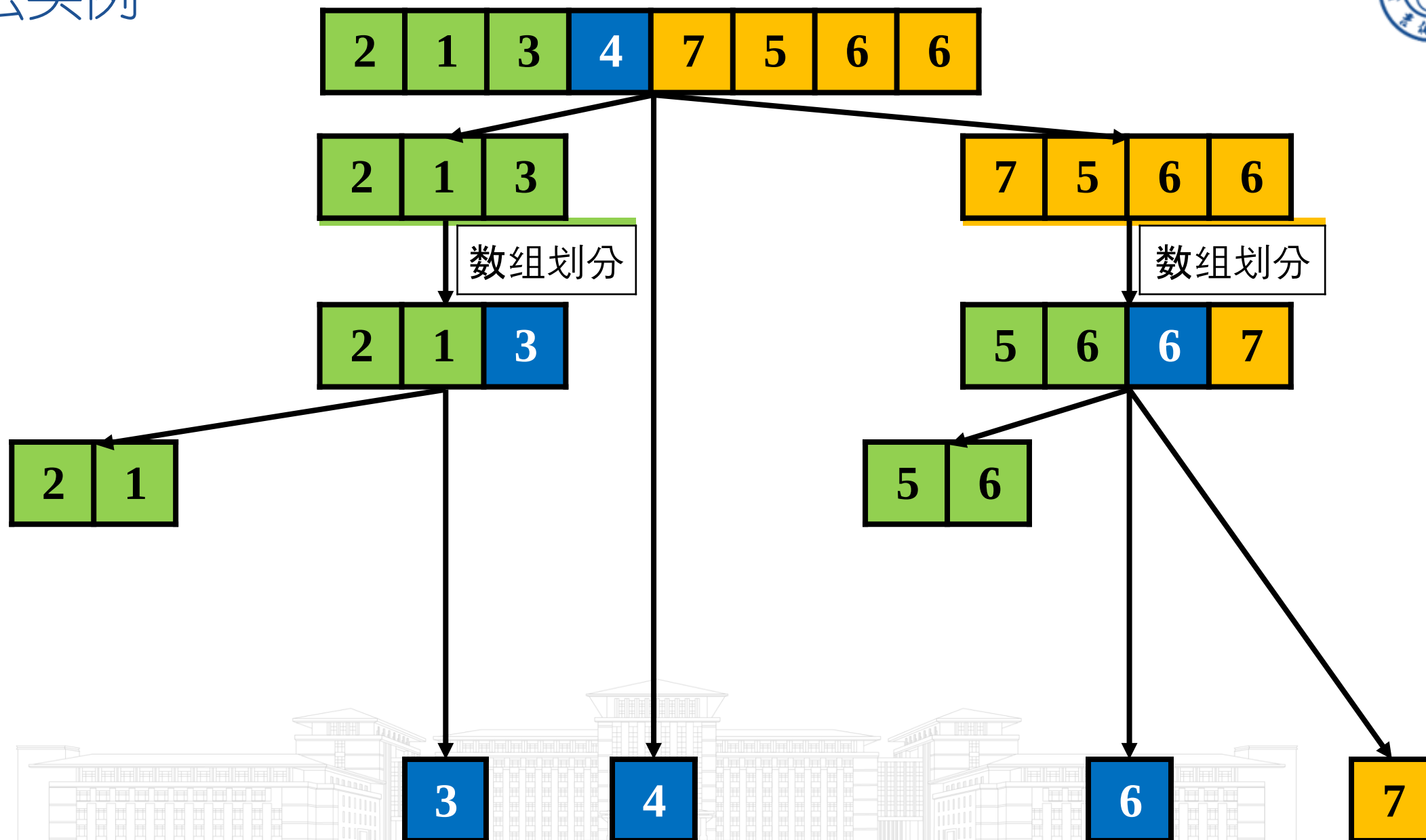
算法实例



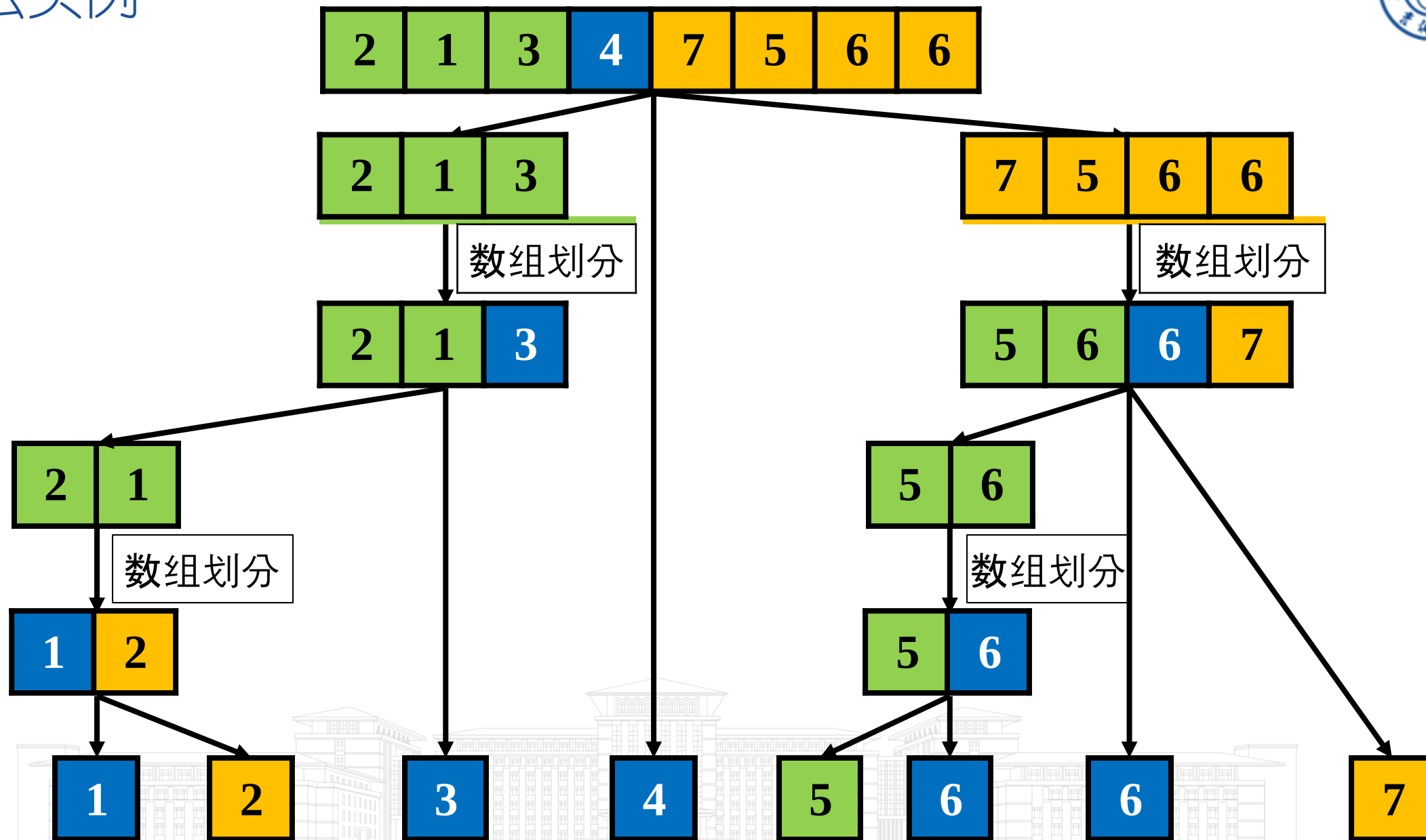
算法实例



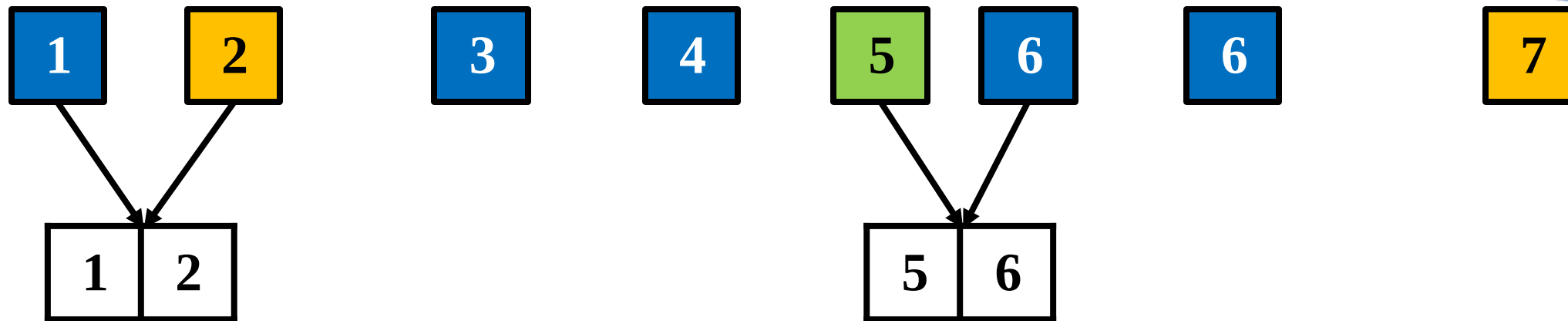
算法实例



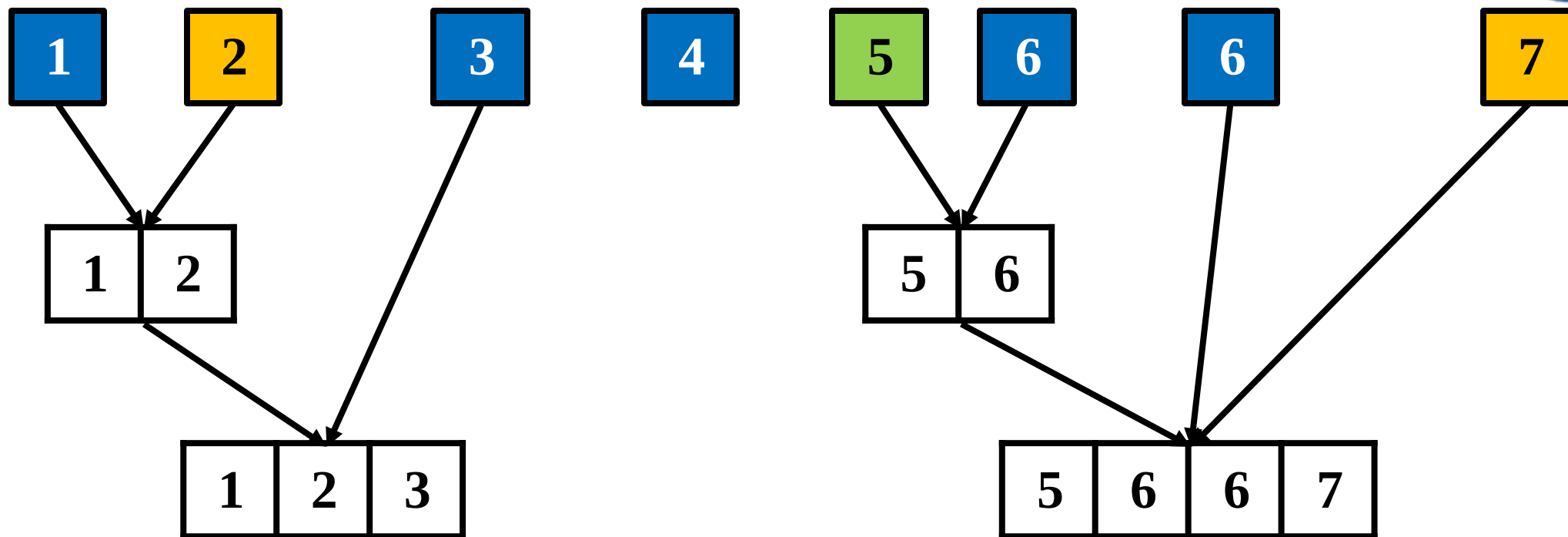
算法实例



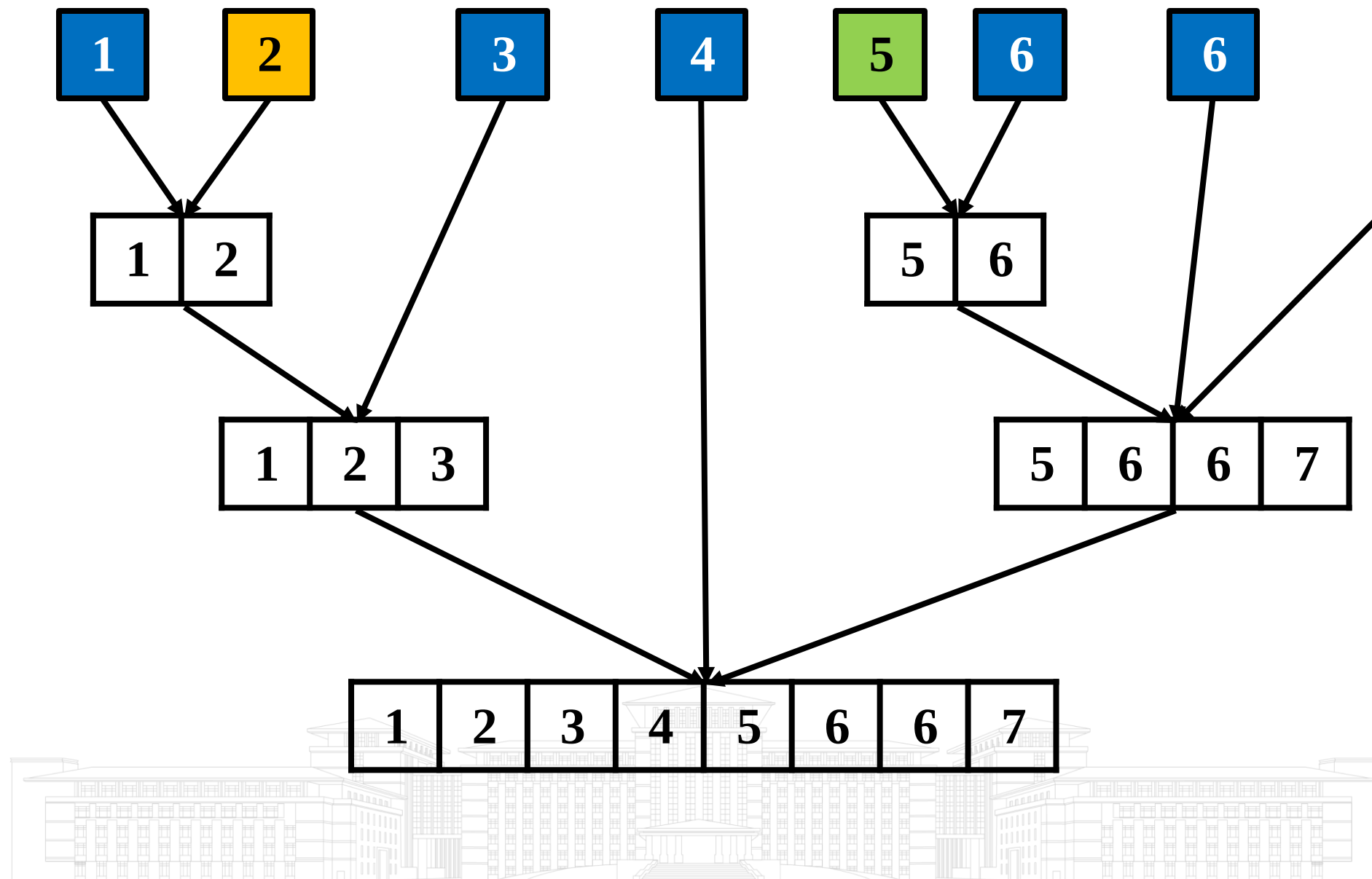
算法实例



算法实例



算法实例



快速排序：伪代码

- **QuickSort(A, p, r)**

初始调用：QuickSort($A, 1, n$)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$

数组划分

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

end



快速排序：伪代码

- **QuickSort(A, p, r)**

初始调用：QuickSort($A, 1, n$)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

end

左右分治



快速排序：复杂度分析

- **QuickSort(A, p, r)**

初始调用：QuickSort($A, 1, n$)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

end

$O(n)$



快速排序：复杂度分析

- **QuickSort(A, p, r)**

初始调用：QuickSort($A, 1, n$)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$) ----- ?

 QuickSort($A, q + 1, r$) ----- ?

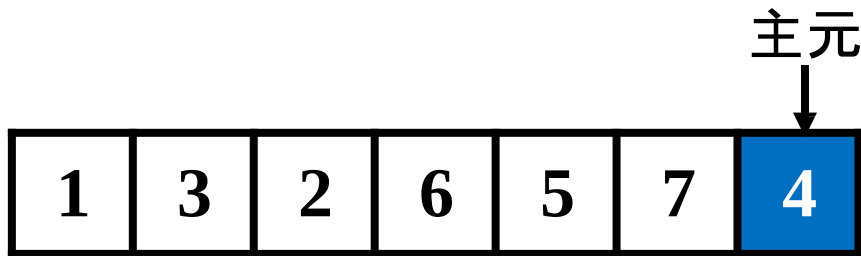
end

问题：子问题规模不确定，如何分析时间复杂度？



快速排序：复杂度分析

- 最好情况
 - 数组划分后，每次主元都在中间



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

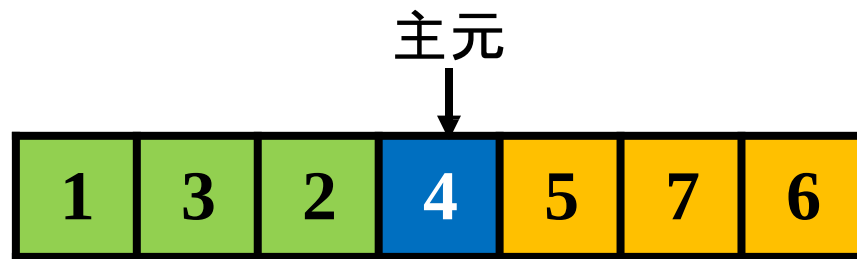
 QuickSort($A, q + 1, r$)

end



快速排序：复杂度分析

- 最好情况
 - 数组划分后，每次主元都在中间



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

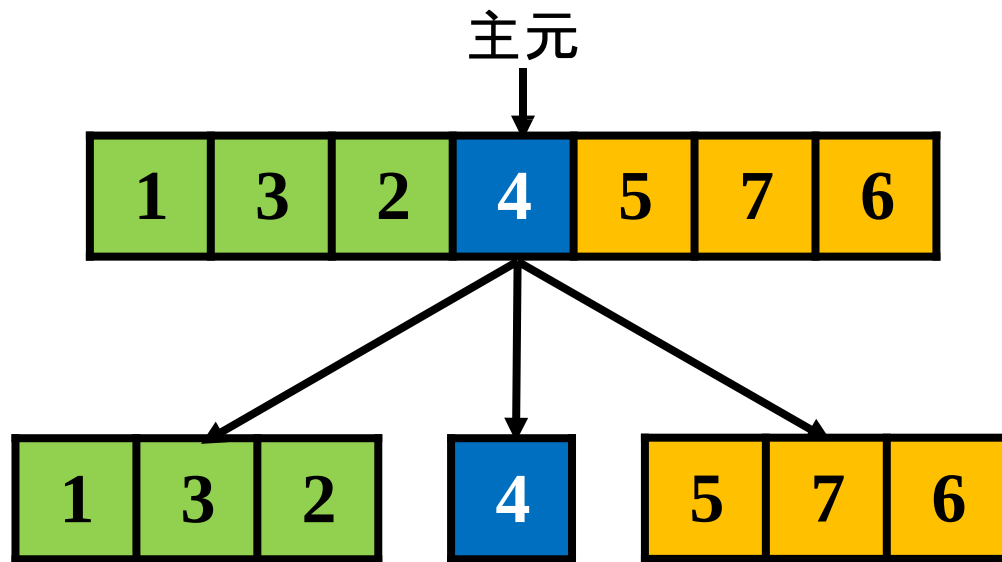
 QuickSort($A, q + 1, r$)

end



快速排序：复杂度分析

- 最好情况
 - 数组划分后，每次主元都在中间



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

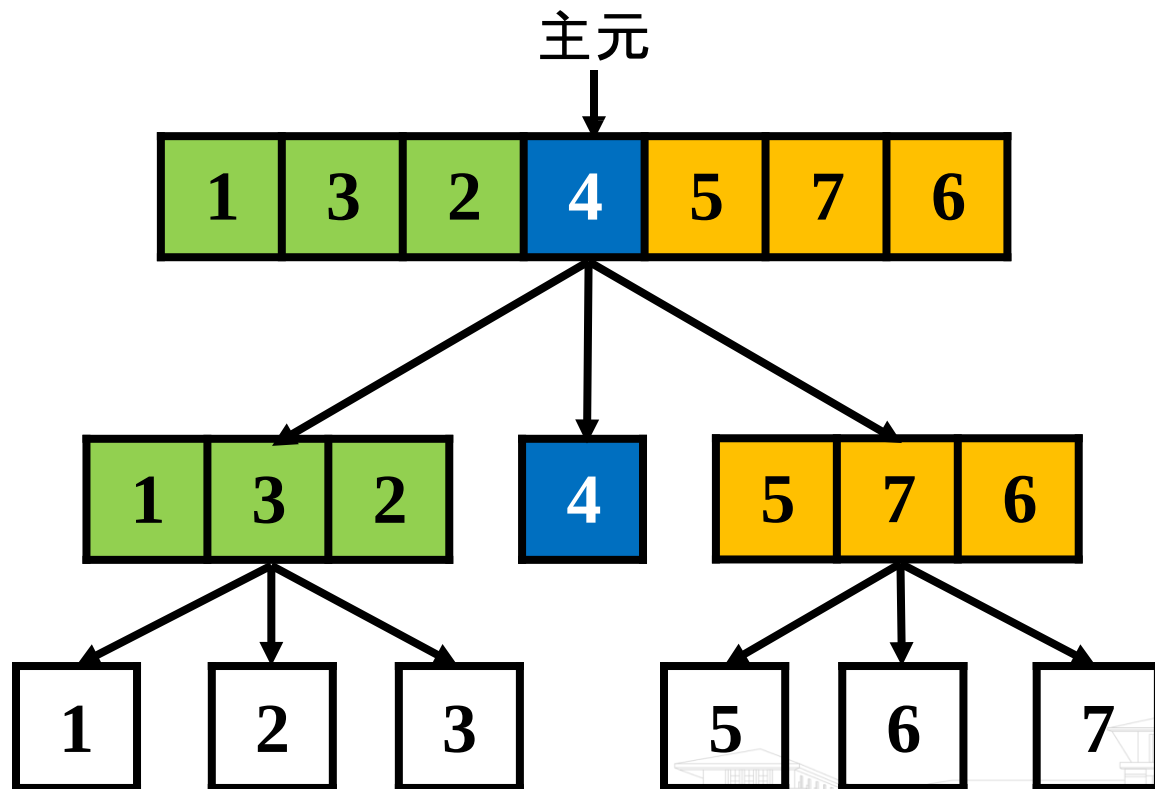
 QuickSort($A, q + 1, r$)

end



快速排序：复杂度分析

- 最好情况
 - 数组划分后，每次主元都在中间



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

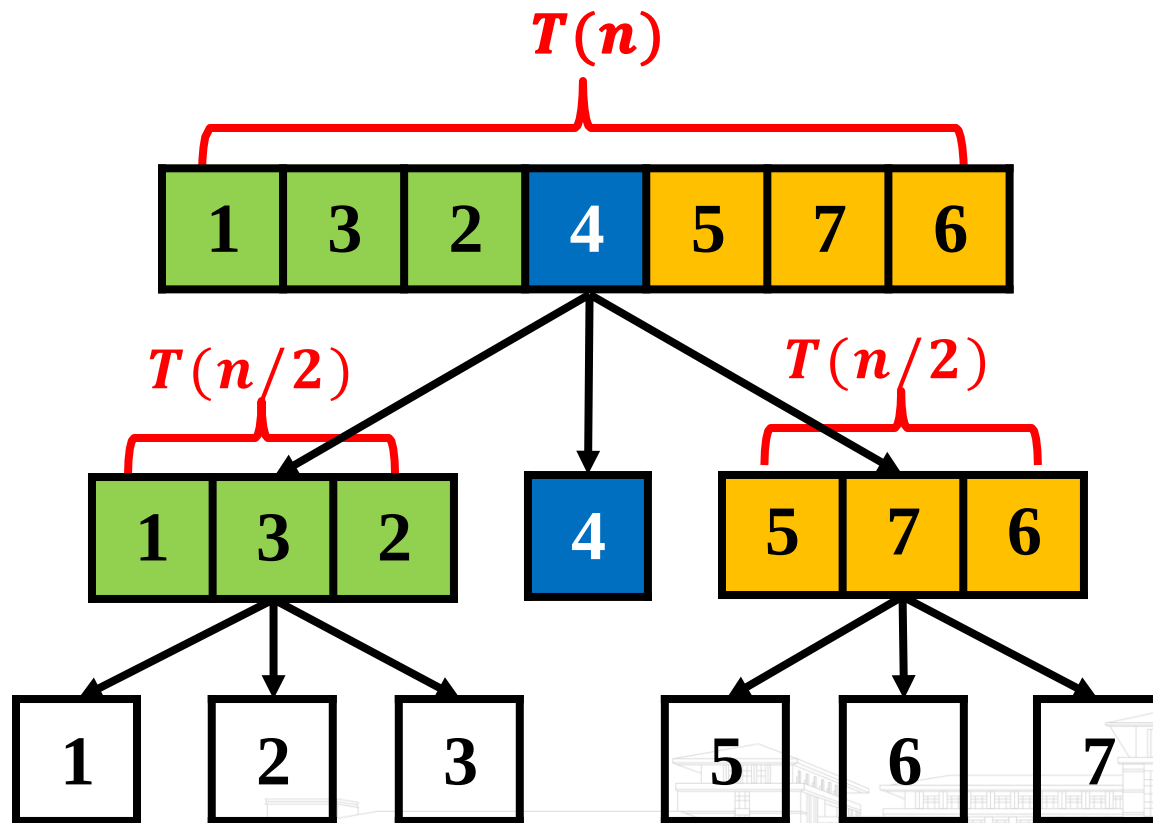
 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

end

快速排序：复杂度分析

- 最好情况
 - 数组划分后，每次主元都在中间

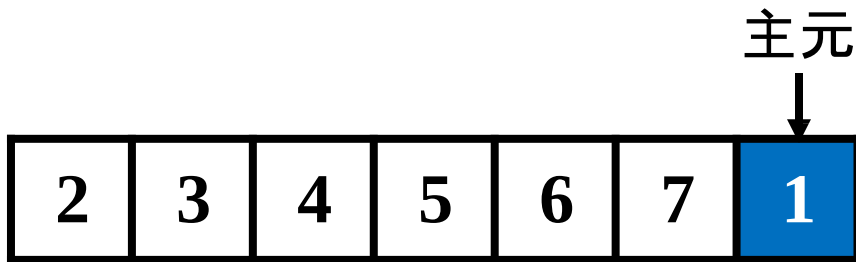


输入: 数组 A , 起始位置 p , 终止位置 r
输出: 有序数组 A
if $p < r$ then
 $q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$
 QuickSort($A, p, q - 1$) ----- $T(n/2)$
 QuickSort($A, q + 1, r$) ----- $T(n/2)$
end

$$T(n) = 2T(n/2) + O(n) \\ = O(n \log n)$$

快速排序：复杂度分析

- 最坏情况
 - 数组划分后，每次主元都在一侧



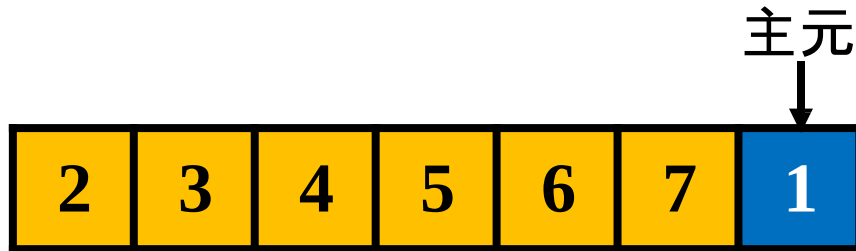
输入: 数组 A , 起始位置 p , 终止位置 r
输出: 有序数组 A

```
if  $p < r$  then
     $q \leftarrow \text{Partition}(A, p, r)$  -----  $O(n)$ 
    QuickSort( $A, p, q - 1$ )
    QuickSort( $A, q + 1, r$ )
end
```



快速排序：复杂度分析

- 最坏情况
 - 数组划分后，每次主元都在一侧



输入: 数组 A , 起始位置 p , 终止位置 r
输出: 有序数组 A

```
if  $p < r$  then
     $q \leftarrow \text{Partition}(A, p, r)$  -----  $O(n)$ 
    QuickSort( $A, p, q - 1$ )
    QuickSort( $A, q + 1, r$ )
end
```



快速排序：复杂度分析

- 最坏情况

- 数组划分后，每次主元都在一侧

主元



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

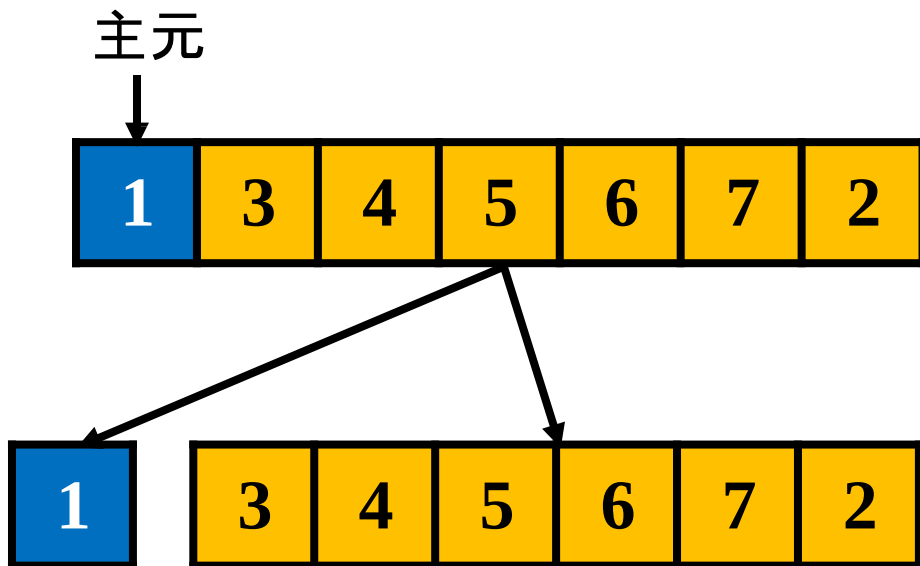
end



快速排序：复杂度分析

- 最坏情况

- 数组划分后，每次主元都在一侧



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

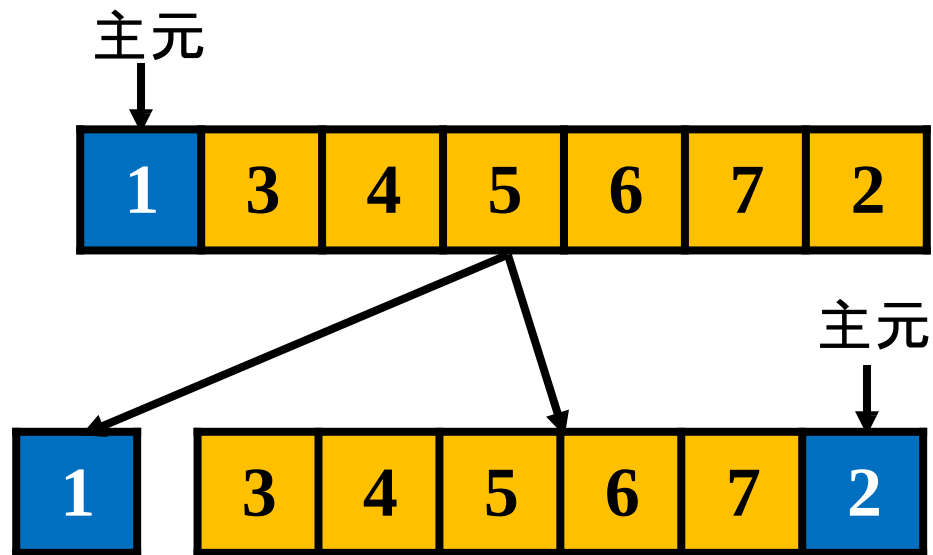
end



快速排序：复杂度分析

- 最坏情况

- 数组划分后，每次主元都在一侧



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

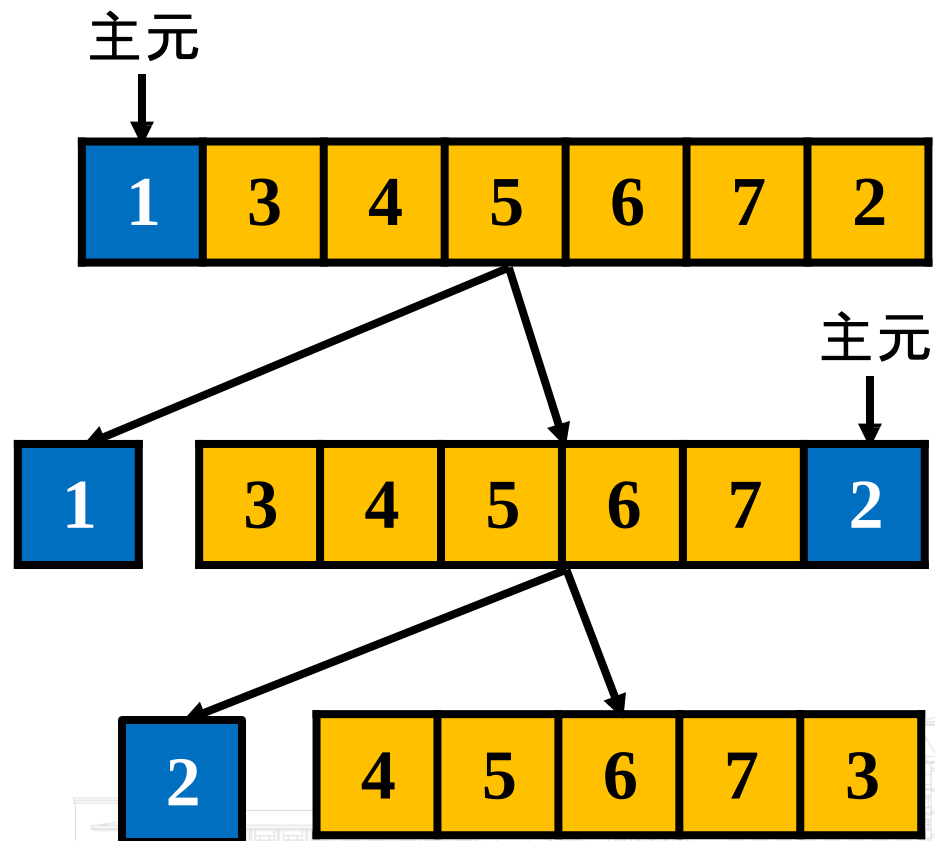
end



快速排序：复杂度分析

- 最坏情况

- 数组划分后，每次主元都在一侧



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$)

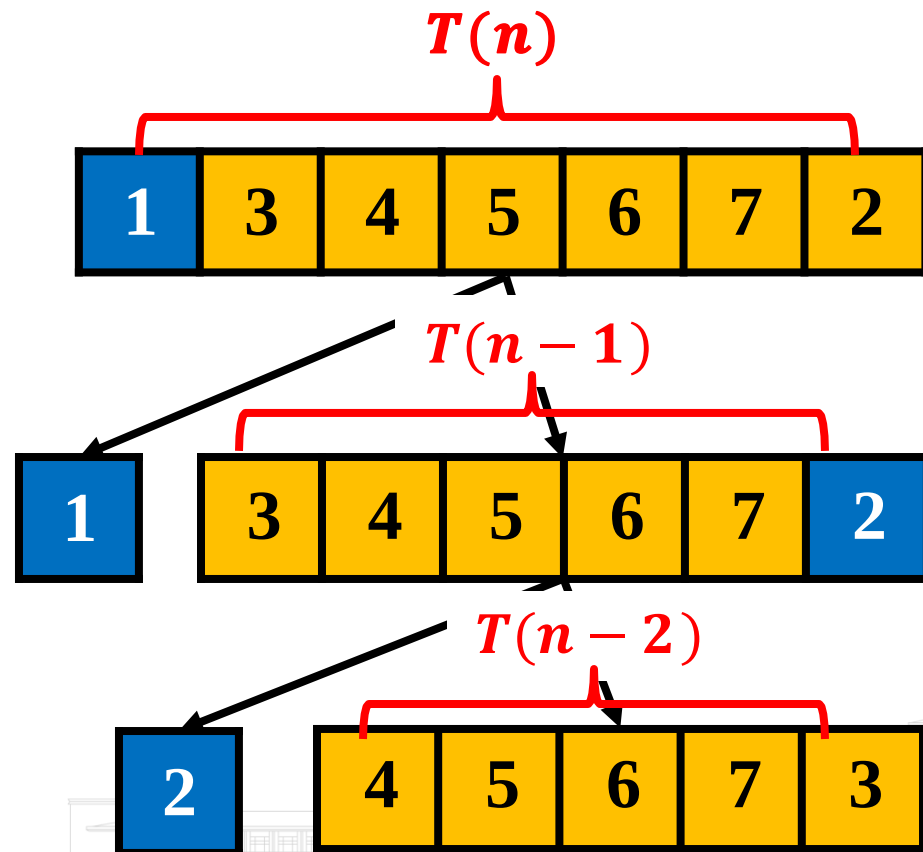
 QuickSort($A, q + 1, r$)

end

快速排序：复杂度分析

- 最坏情况

- 数组划分后，每次主元都在一侧



输入: 数组 A , 起始位置 p , 终止位置 r

输出: 有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$ ----- $O(n)$

 QuickSort($A, p, q - 1$) ----- $T(0)$

 QuickSort($A, q + 1, r$) ----- $T(n-1)$

end

$$T(n) = T(n-1) + T(1)O(n) = O(n^2)$$

快速排序：复杂度分析

初始调用：QuickSort($A, 1, n$)

- QuickSort(A, p, r)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

end

- 最好情况： $O(n \log n)$
- 最坏情况： $O(n^2)$

快速排序看似不优于归并排序



快速排序：复杂度分析

初始调用：QuickSort($A, 1, n$)

- QuickSort(A, p, r)

输入：数组 A ，起始位置 p ，终止位置 r

输出：有序数组 A

if $p < r$ then

$q \leftarrow \text{Partition}(A, p, r)$

 QuickSort($A, p, q - 1$)

 QuickSort($A, q + 1, r$)

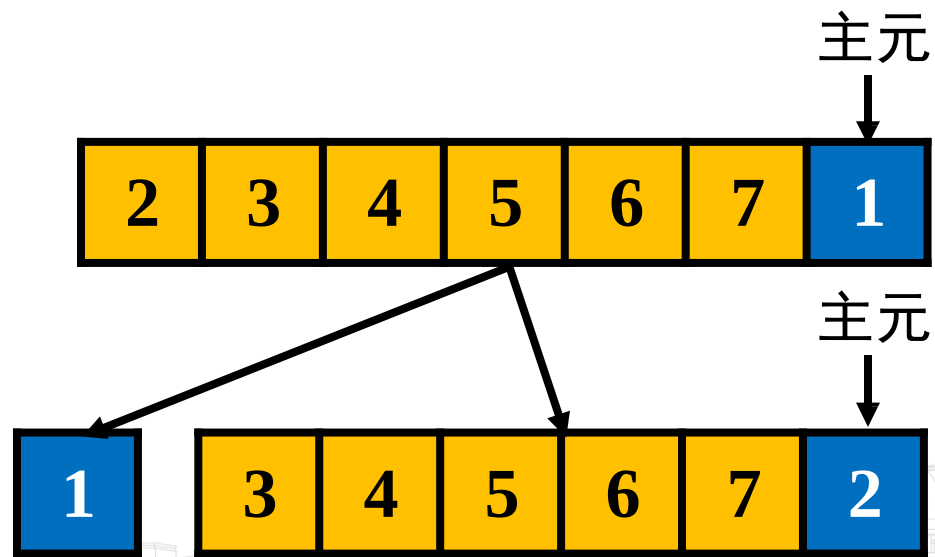
end

- 最好情况： $O(n \log n)$
- 最坏情况： $O(n^2)$

思考：如何摆脱输入导致最坏情况的困境？

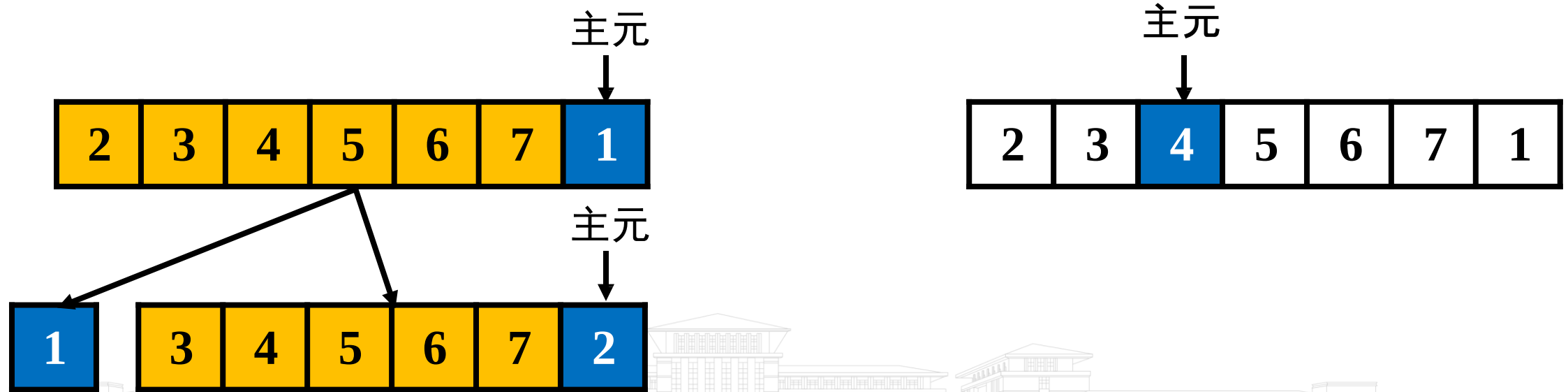
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况



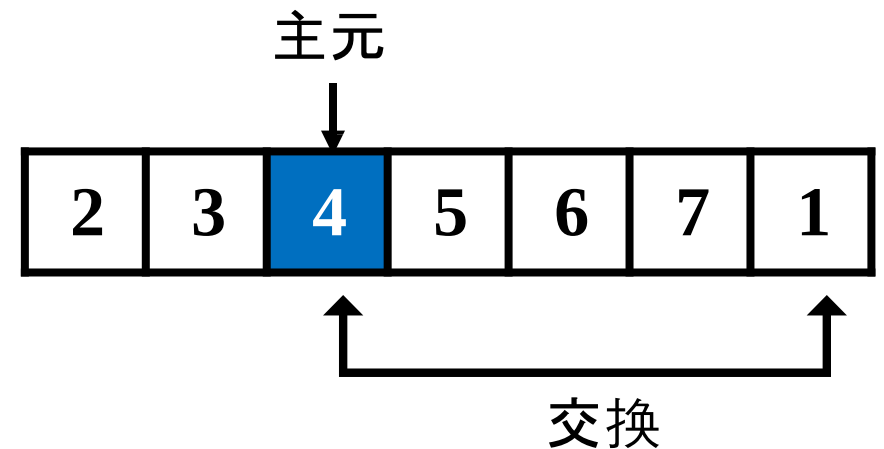
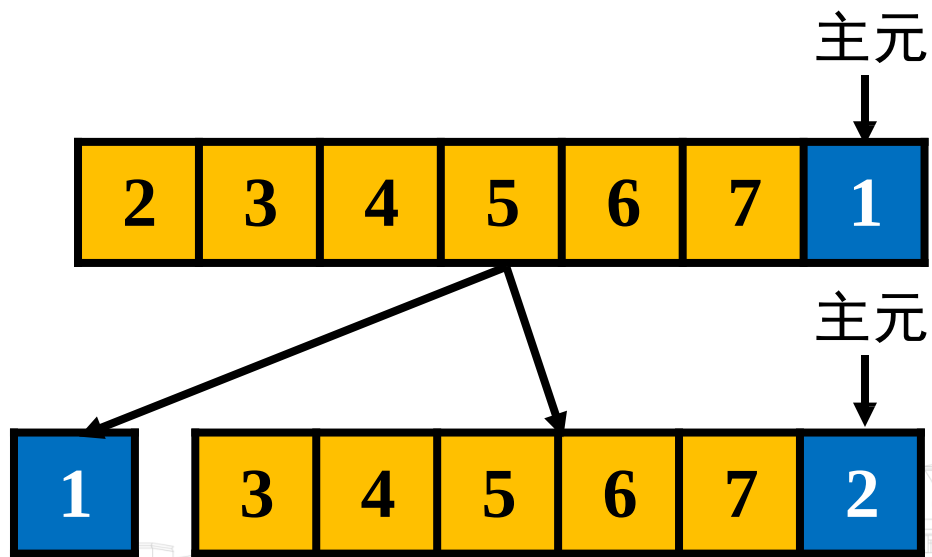
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



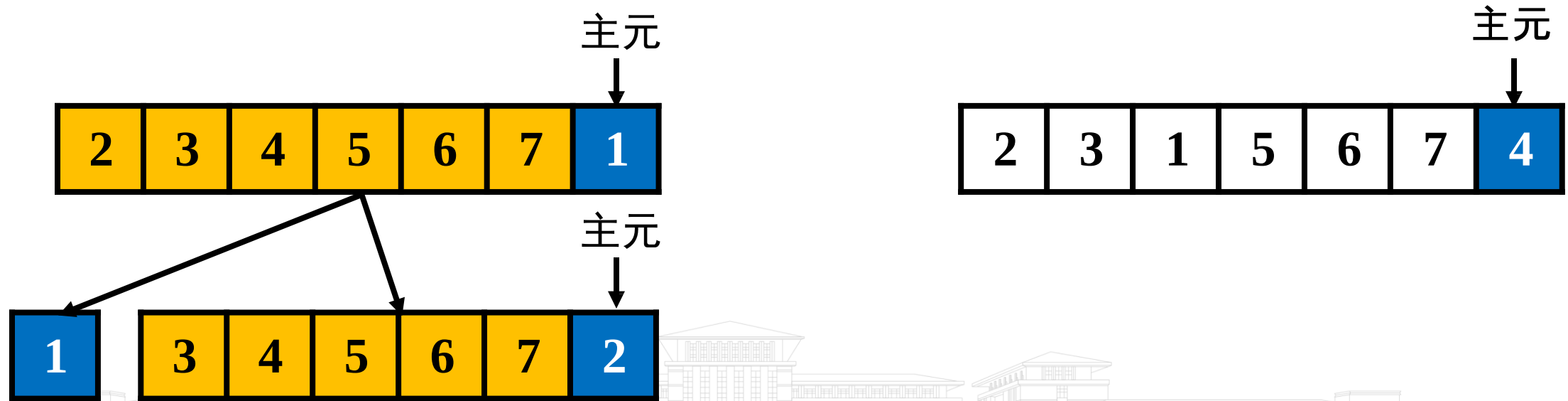
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



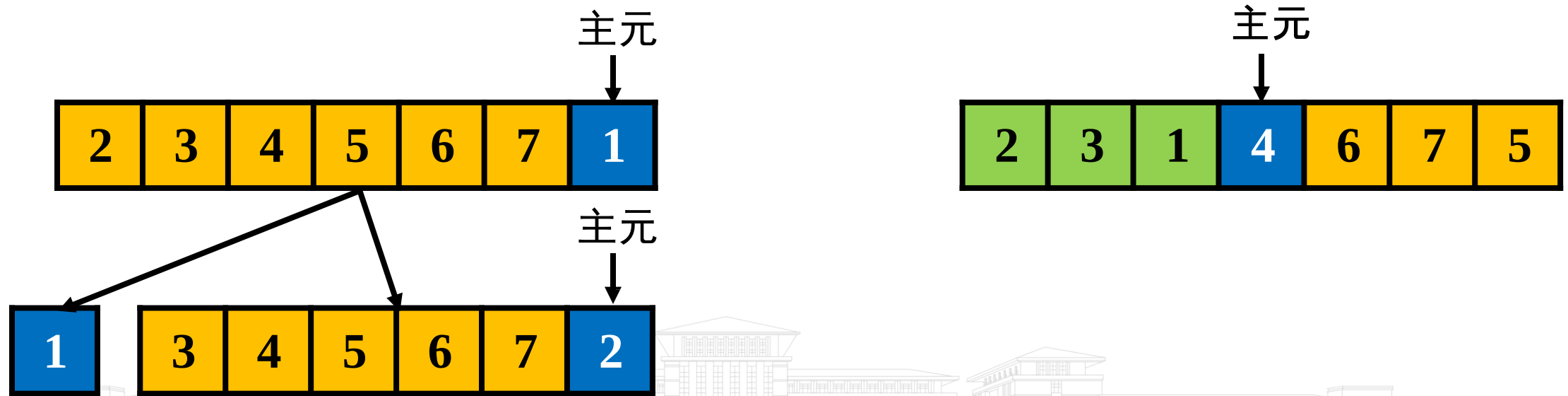
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



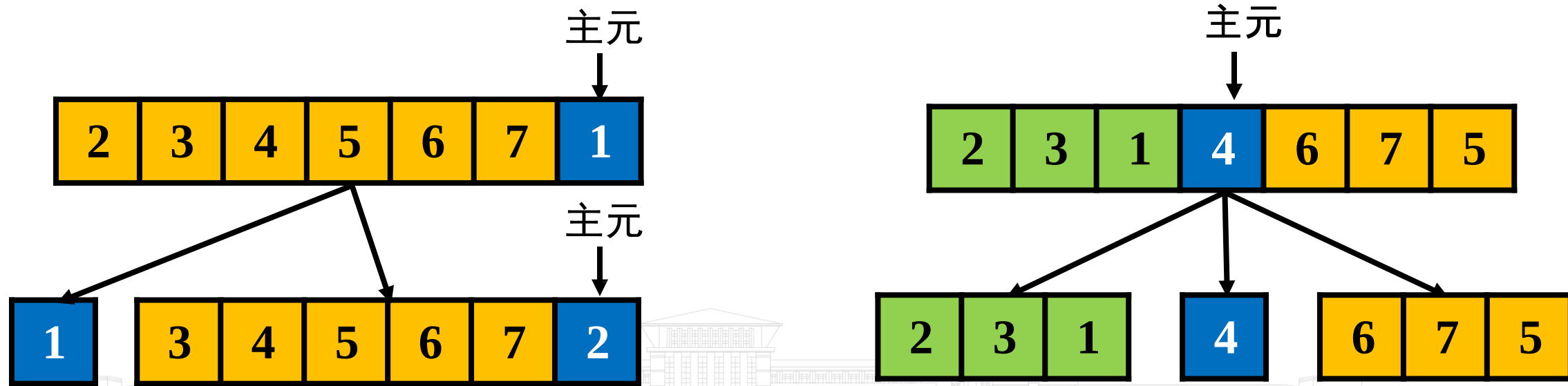
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



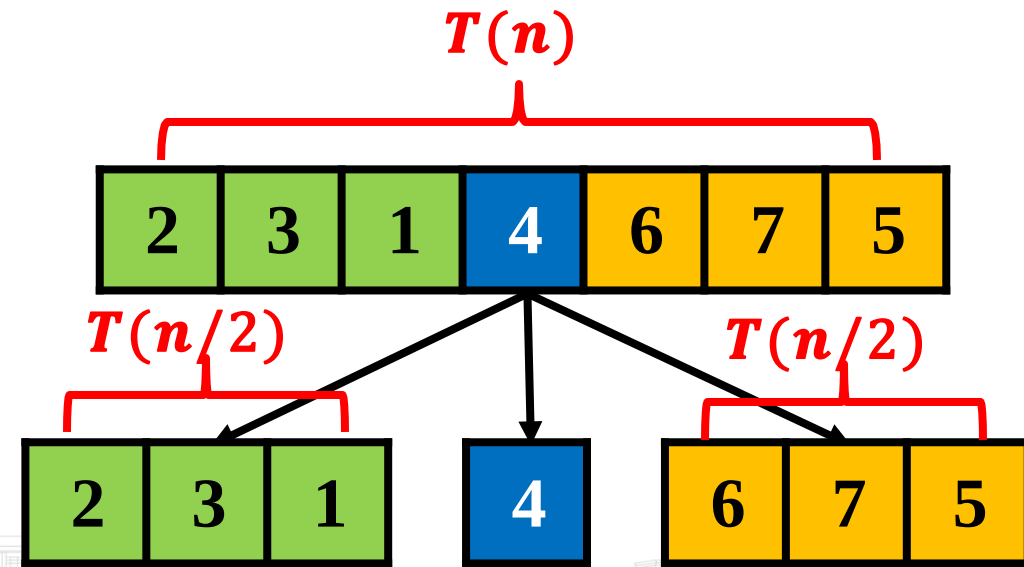
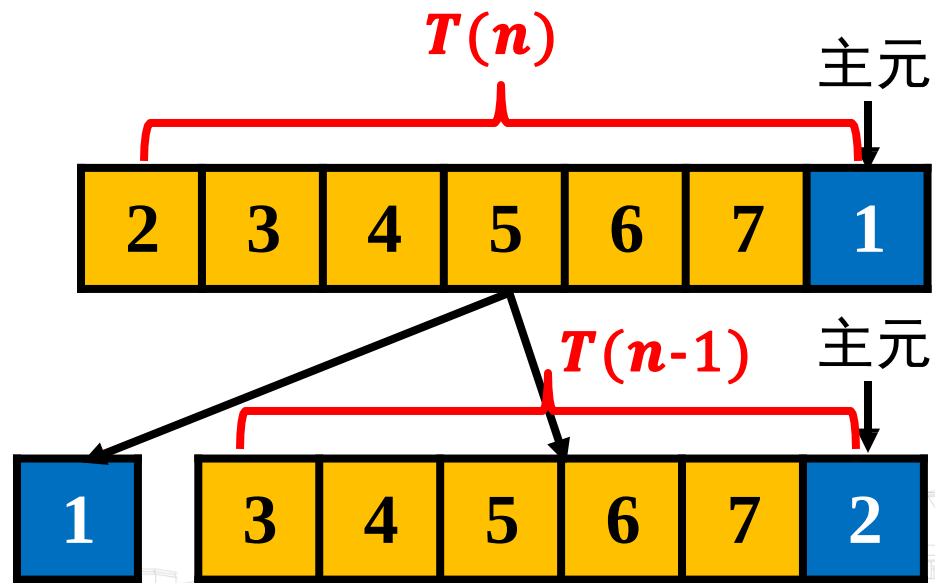
随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



随机划分

- 反思最差情况
 - 数组划分时选取**固定**位置主元，可以针对性构造最差情况
- 解决方案
 - 数组划分时选取**随机**位置主元，无法针对性构造最差情况



随机划分：伪代码

• Randomized-Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r

输出: 划分位置 q

$s \leftarrow \text{Random}(p, r)$

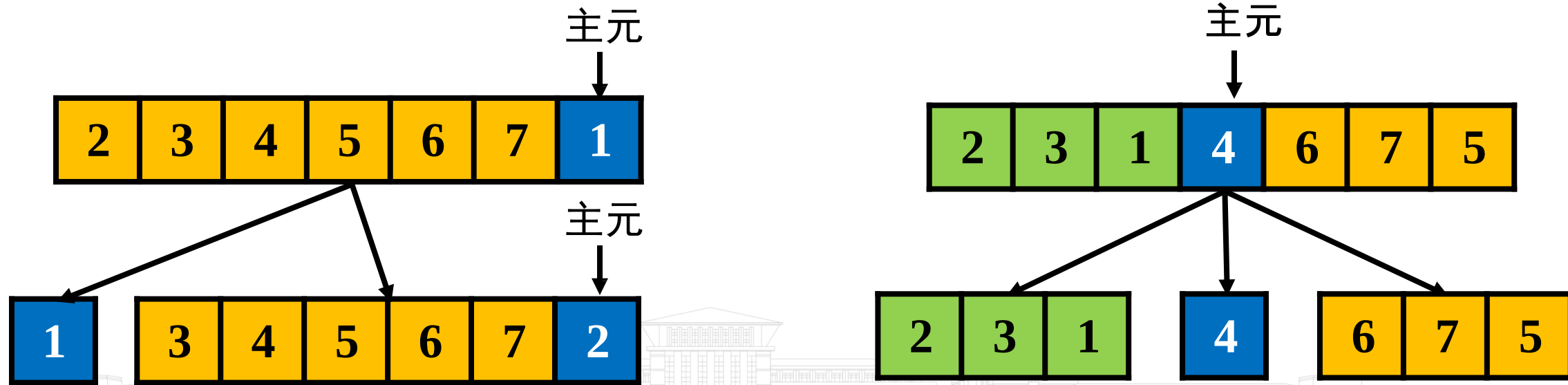
exchange $A[s]$ with $A[r]$

$q \leftarrow \text{Partition}(A, p, r)$

return q

随机选取主元

思考：满足
算法的能行
性吗？



随机划分：伪代码

• Randomized-Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r

输出: 划分位置 q

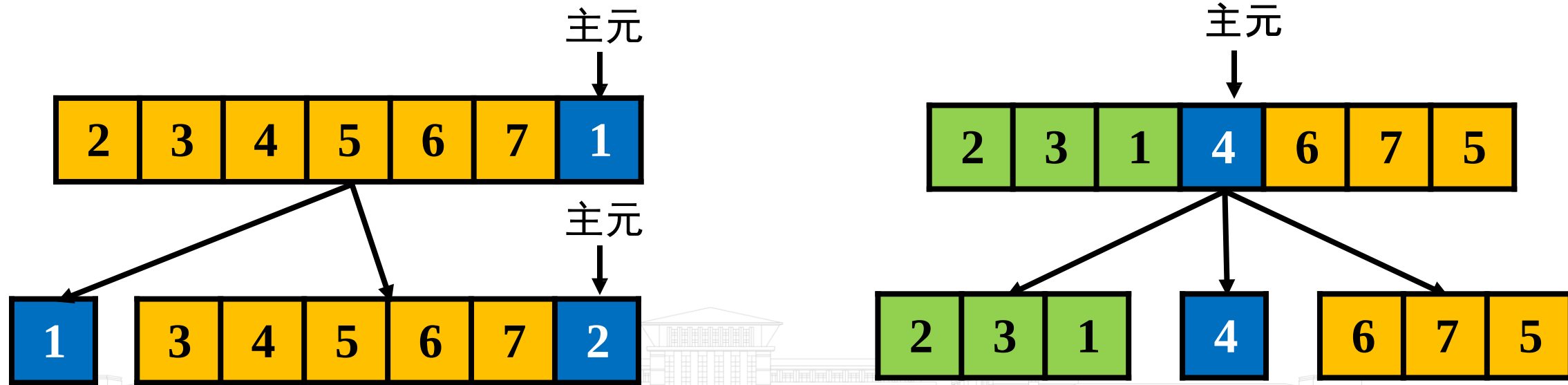
$s \leftarrow \text{Random}(p, r)$

exchange $A[s]$ with $A[r]$

$q \leftarrow \text{Partition}(A, p, r)$

return q

交换到末尾



随机划分：伪代码

• Randomized-Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r

输出: 划分位置 q

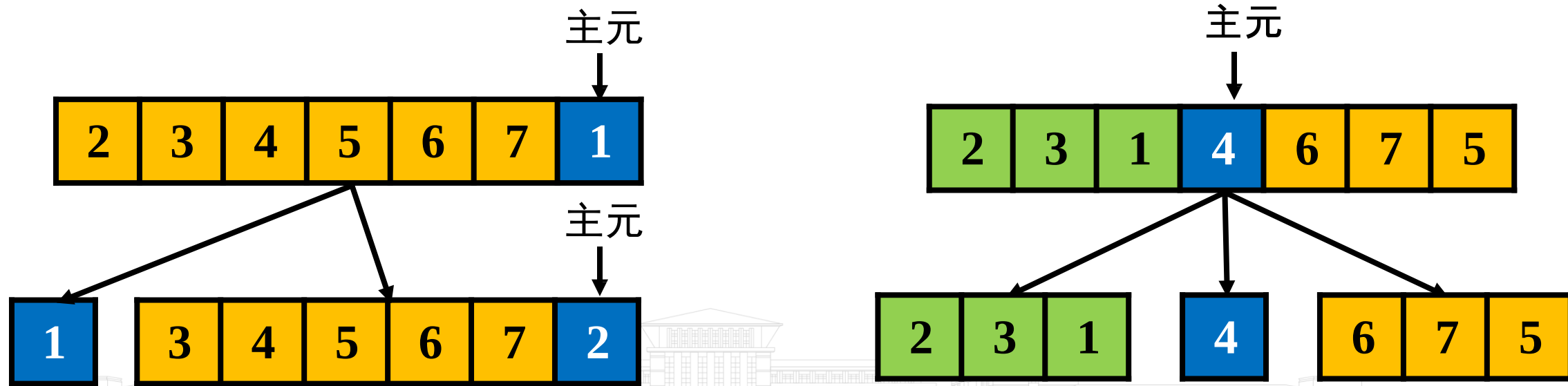
$s \leftarrow \text{Random}(p, r)$

exchange $A[s]$ with $A[r]$

$q \leftarrow \text{Partition}(A, p, r)$

return q

执行数组划分



随机化快速排序：伪代码

• Randomized-Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r
输出: 划分位置 q
 $s \leftarrow \text{Random}(p, r)$
exchange $A[s]$ with $A[r]$
 $q \leftarrow \text{Partition}(A, p, r)$
return q

• Randomized-QuickSort(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r
输出: 有序数组 A
if $p < r$ then
 $q \leftarrow \text{Randomized-Partition}(A, p, r)$
 Randomized-QuickSort($A, p, q - 1$)
 Randomized-QuickSort($A, q + 1, r$)
end

随机选取主元





随机化快速排序：伪代码

• Randomized-Partition(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r
输出: 划分位置 q
 $s \leftarrow \text{Random}(p, r)$
exchange $A[s]$ with $A[r]$
 $q \leftarrow \text{Partition}(A, p, r)$
return q

思考：如何分析
时间复杂度？

• Randomized-QuickSort(A, p, r)

输入: 数组 A , 起始位置 p , 终止位置 r
输出: 有序数组 A
if $p < r$ then
 $q \leftarrow \text{Randomized-Partition}(A, p, r)$
 Randomized-QuickSort($A, p, q - 1$)
 Randomized-QuickSort($A, q + 1, r$)
end

左右分治





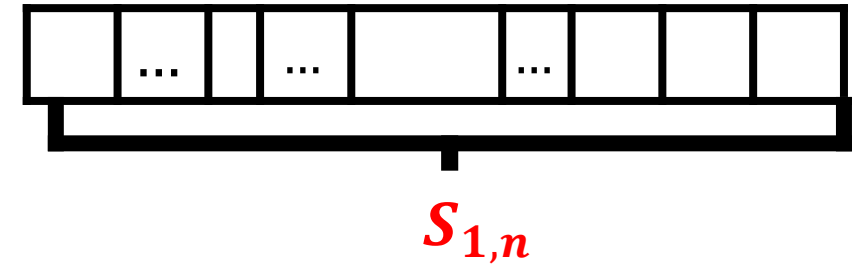
随机化快速排序：复杂度分析

- 分析目标：期望复杂度
 - 计算元素期望比较次数 $E[X]$



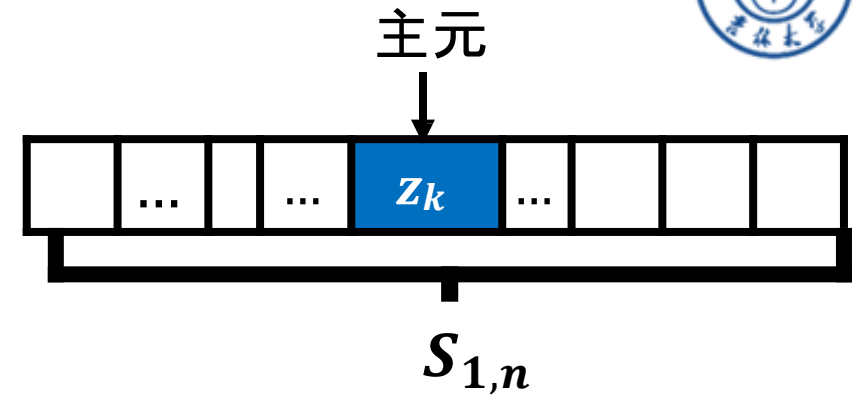
随机化快速排序：复杂度分析

- 分析目标：期望复杂度
 - 计算元素期望比较次数 $E[X]$
- 符号表示
 - z_k ：数组 A 中第 k 小的元素
 - 集合 $S_{i,j} : \{z_i, \dots, z_j\}$



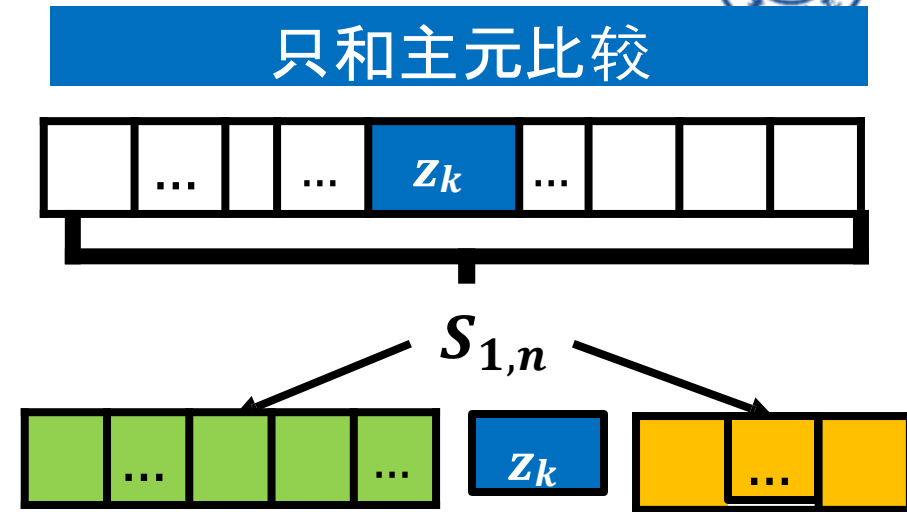
随机化快速排序：复杂度分析

- 分析目标：期望复杂度
 - 计算元素期望比较次数 $E[X]$
- 符号表示
 - z_k ：数组 A 中第 k 小的元素
 - 集合 $S_{i,j} : \{z_i, \dots, z_j\}$



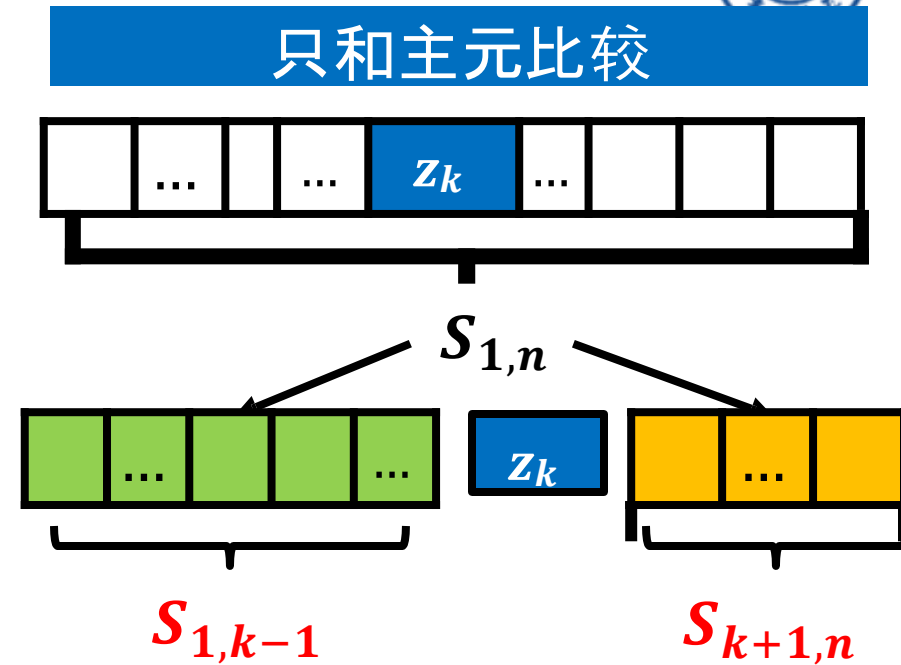
随机化快速排序：复杂度分析

- 分析目标：期望复杂度
 - 计算元素期望比较次数 $E[X]$
- 符号表示
 - z_k ：数组 A 中第 k 小的元素
 - 集合 $S_{i,j} : \{z_i, \dots, z_j\}$



随机化快速排序：复杂度分析

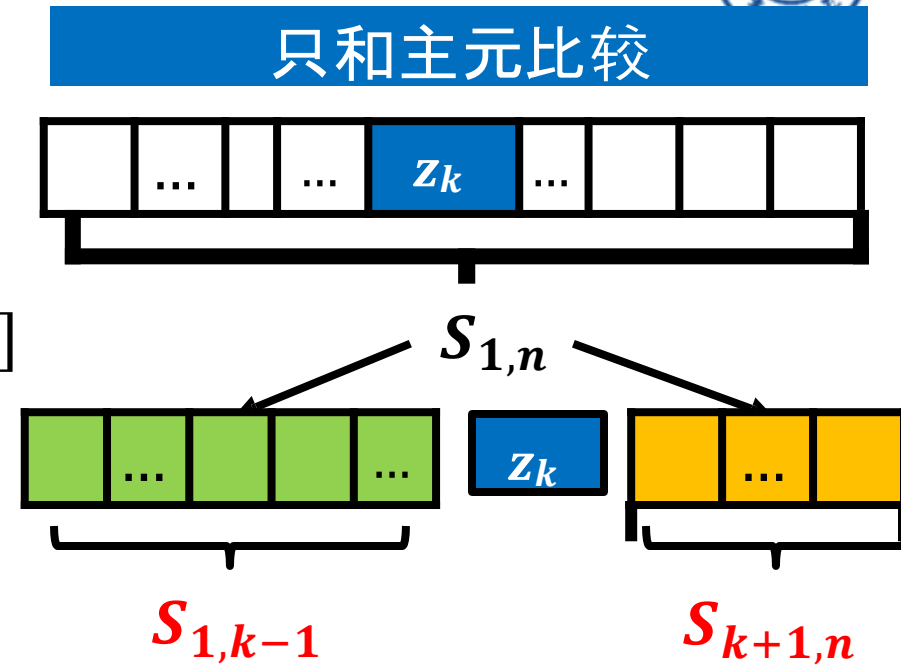
- 分析目标：期望复杂度
 - 计算元素期望比较次数 $E[X]$
- 符号表示
 - z_k ：数组 A 中第 k 小的元素
 - 集合 $S_{i,j} : \{z_i, \dots, z_j\}$



随机化快速排序：复杂度分析

• 推导过程

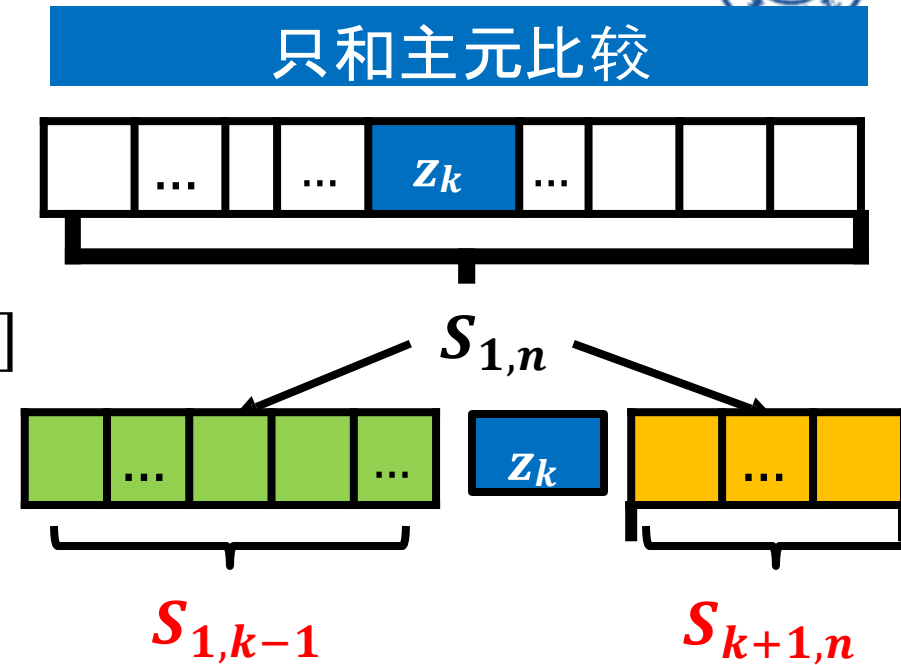
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$



随机化快速排序：复杂度分析

• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$

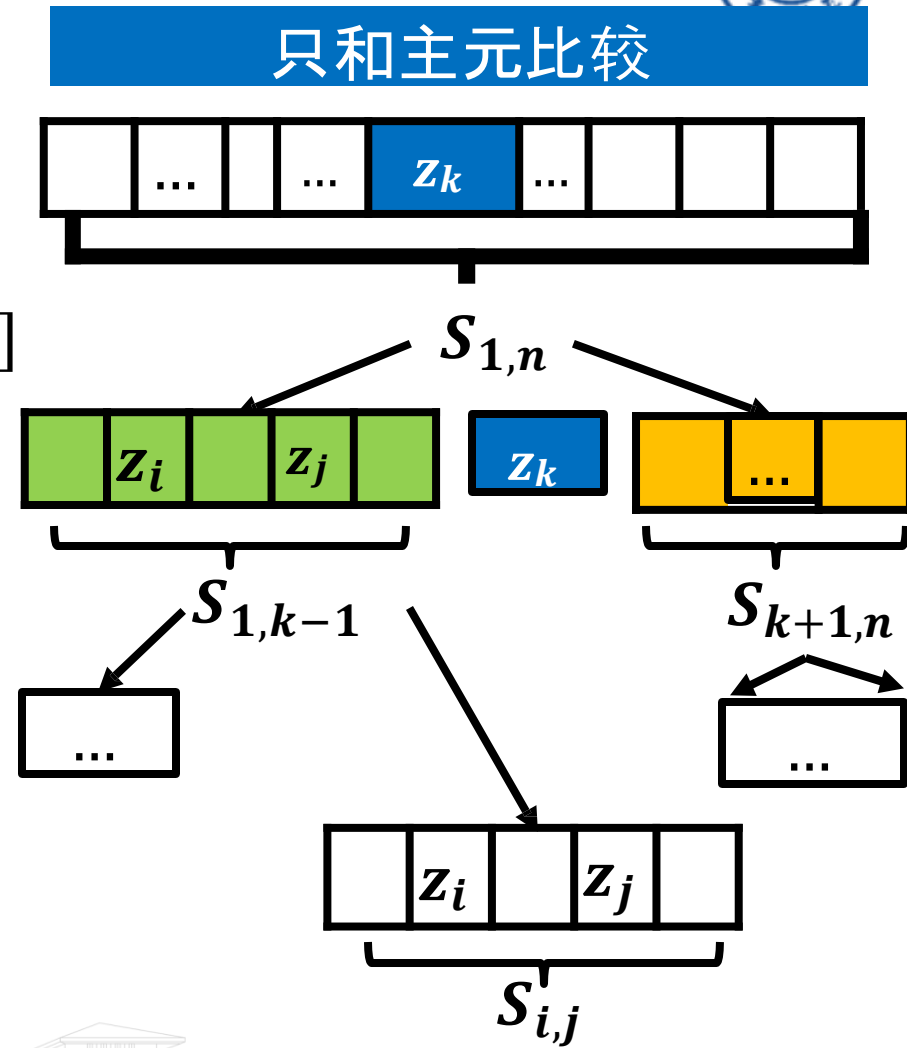


随机化快速排序：复杂度分析

• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$

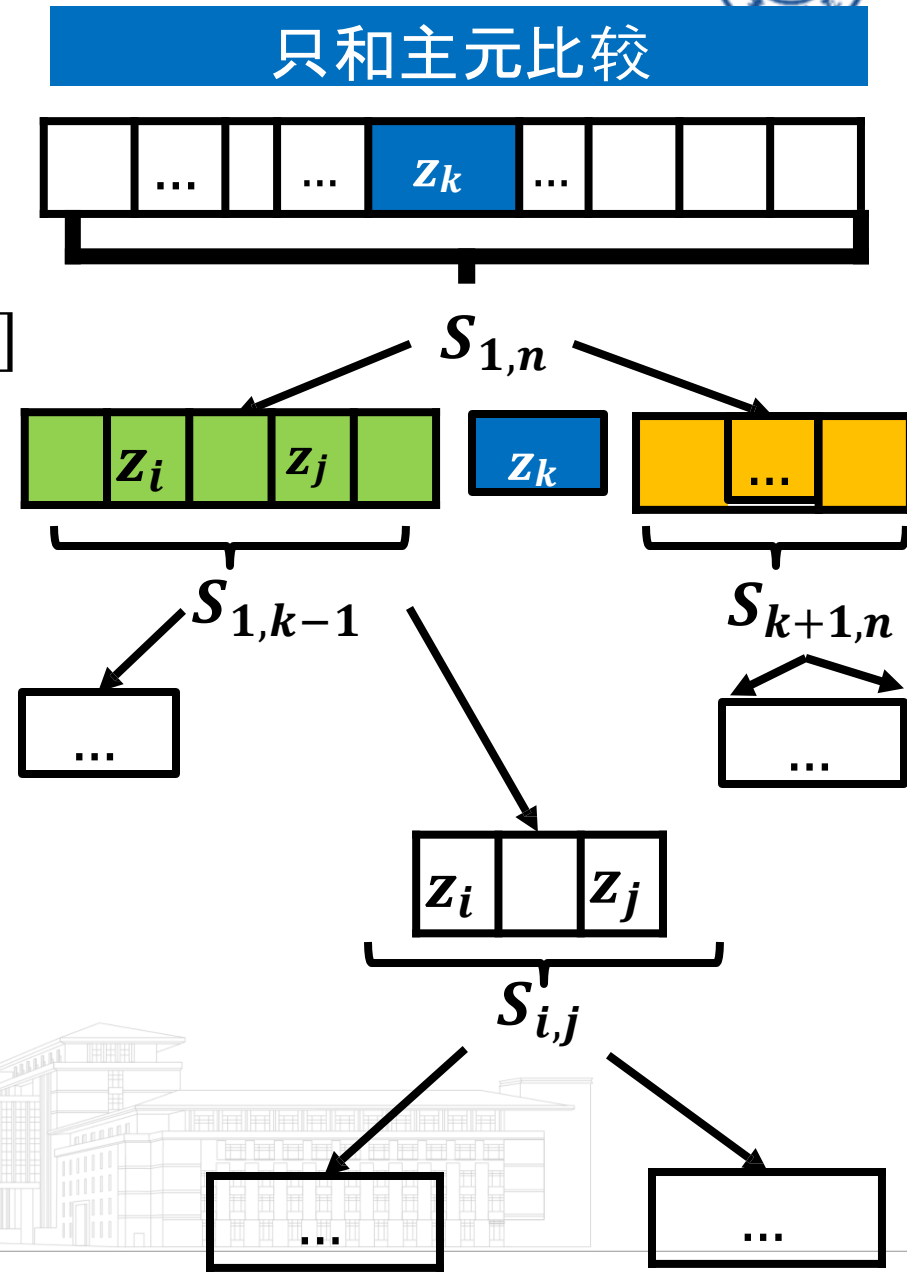
思考：如果 z_i 和 z_j 不在一侧？



随机化快速排序：复杂度分析

• 推导过程

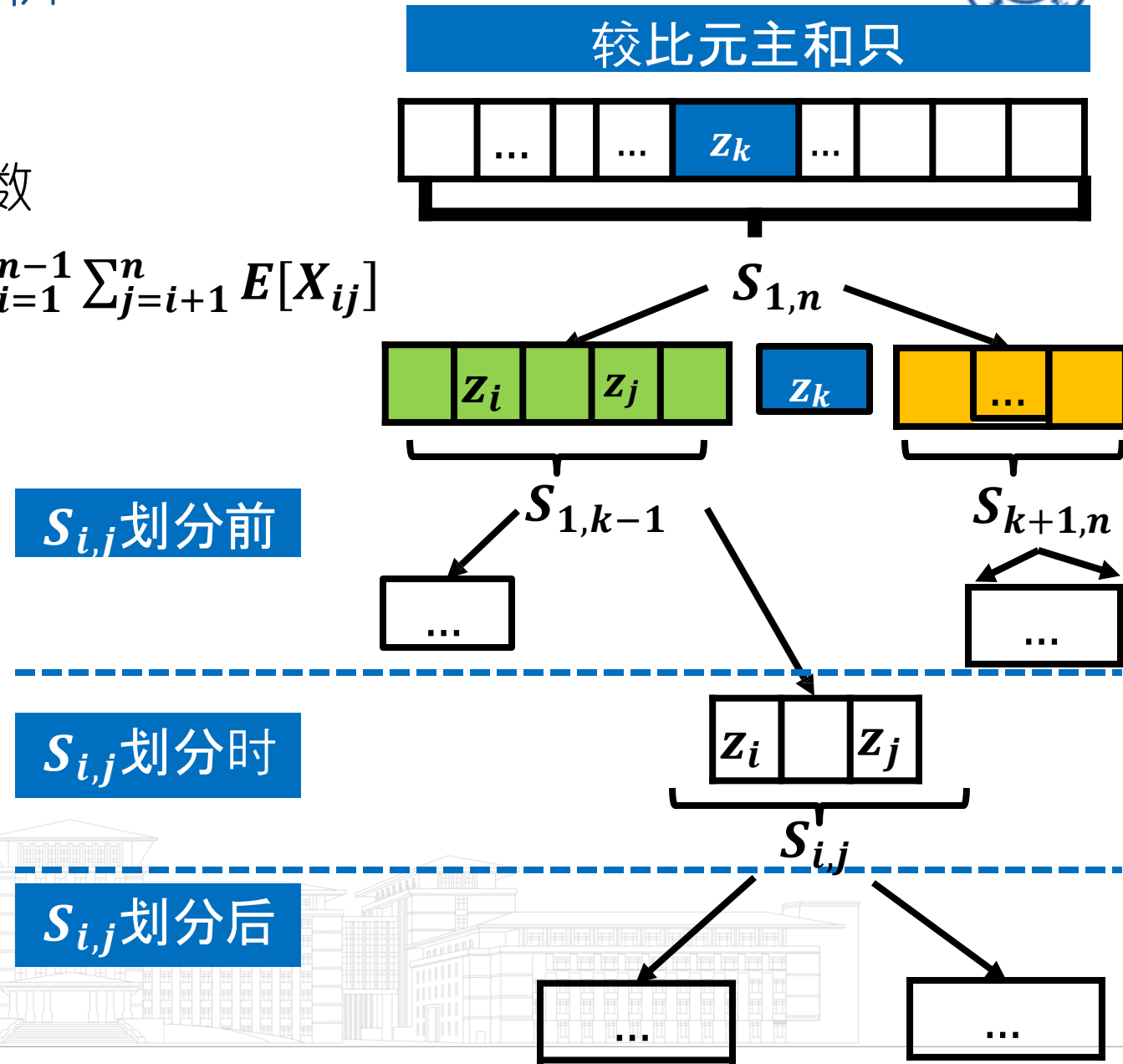
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

• 推导过程

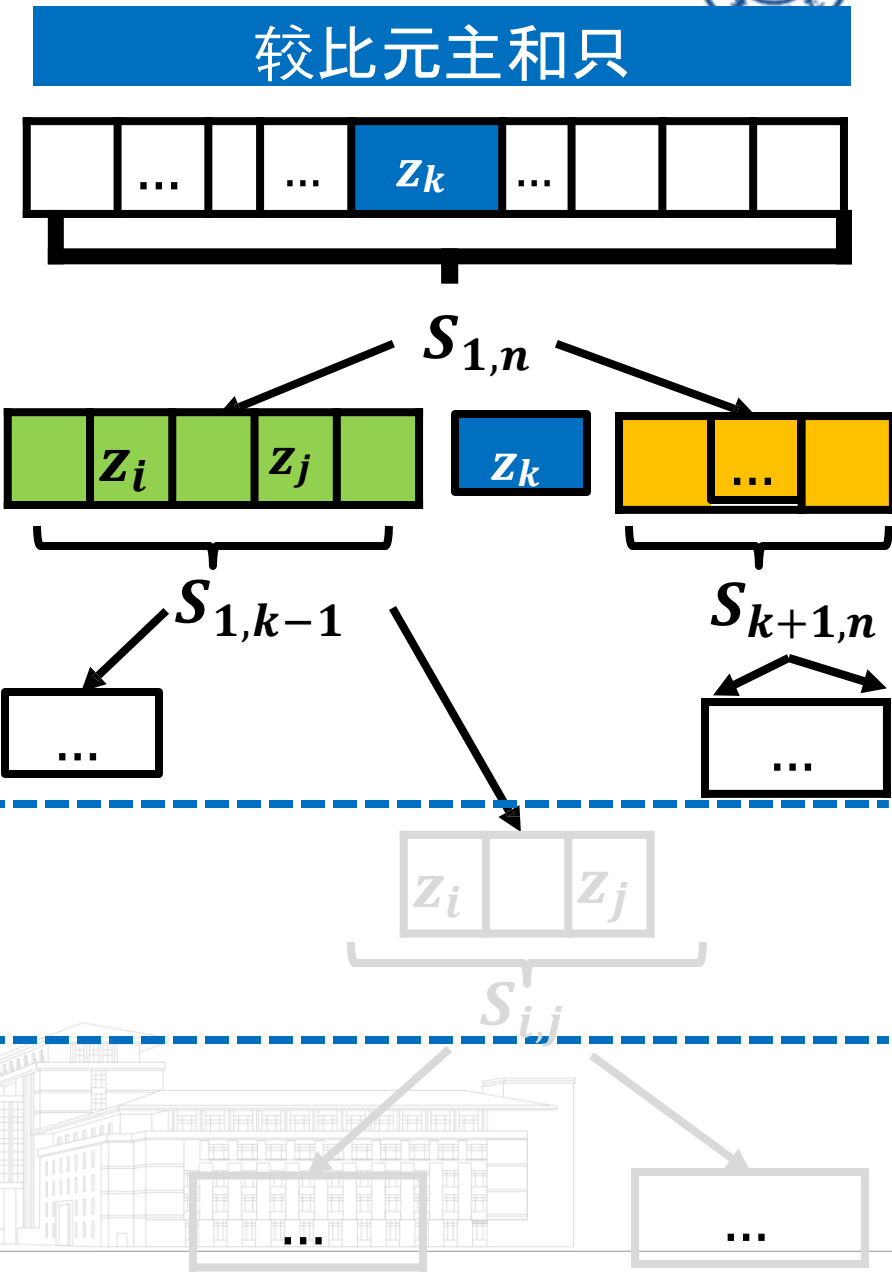
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

• 推导过程

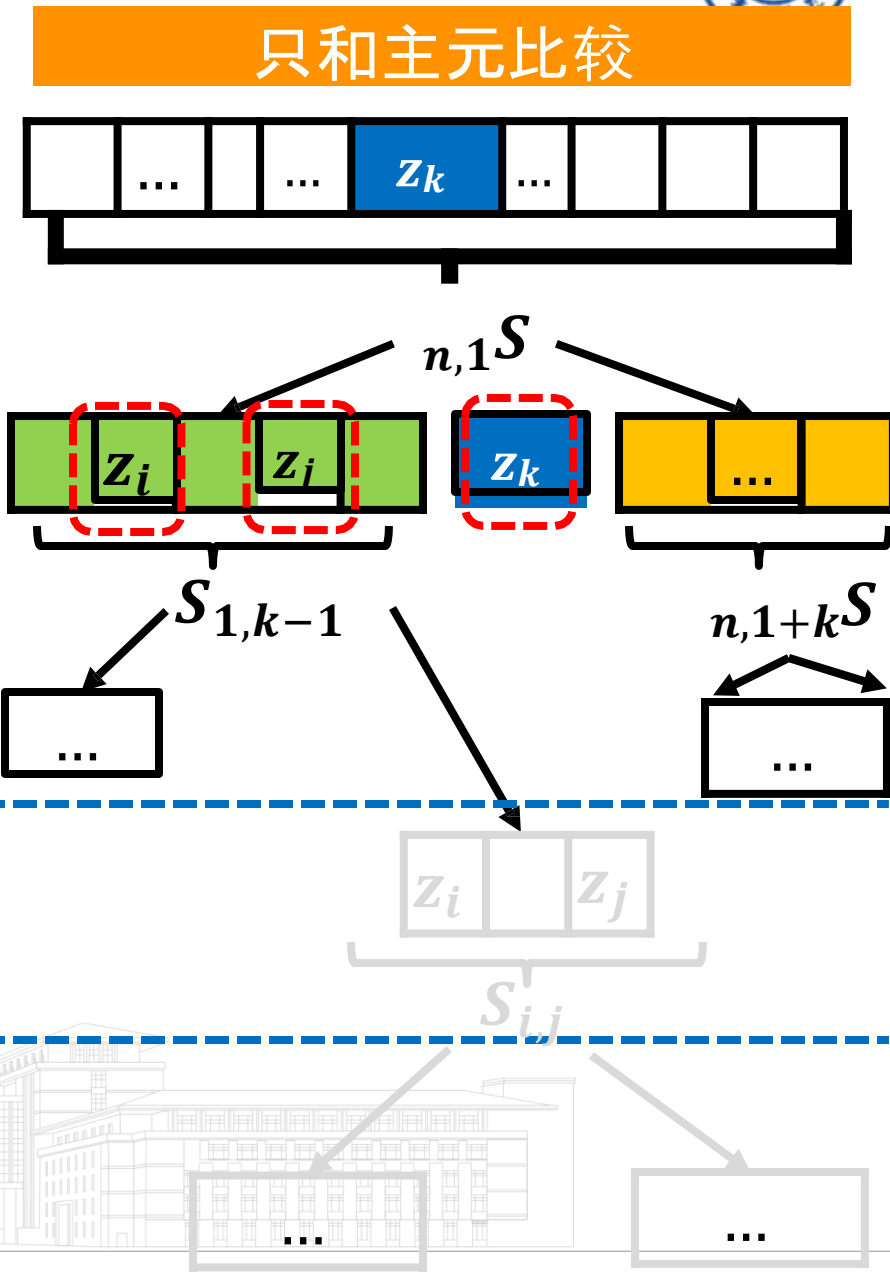
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$

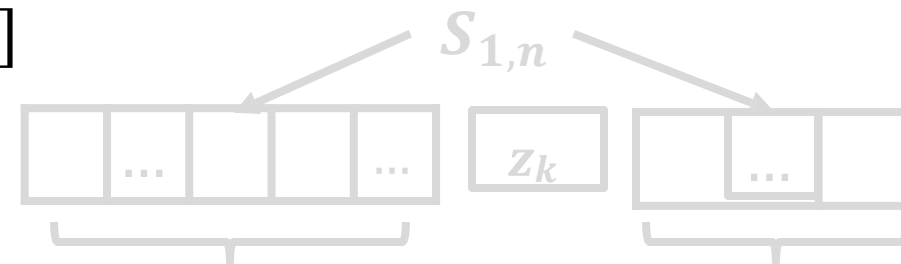


随机化快速排序：复杂度分析

• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$

只和主元比较



$S_{i,j}$ 划分前

主元不在 $S_{i,j}$ 中, z_i 和 z_j 不比较



$S_{i,j}$ 划分时



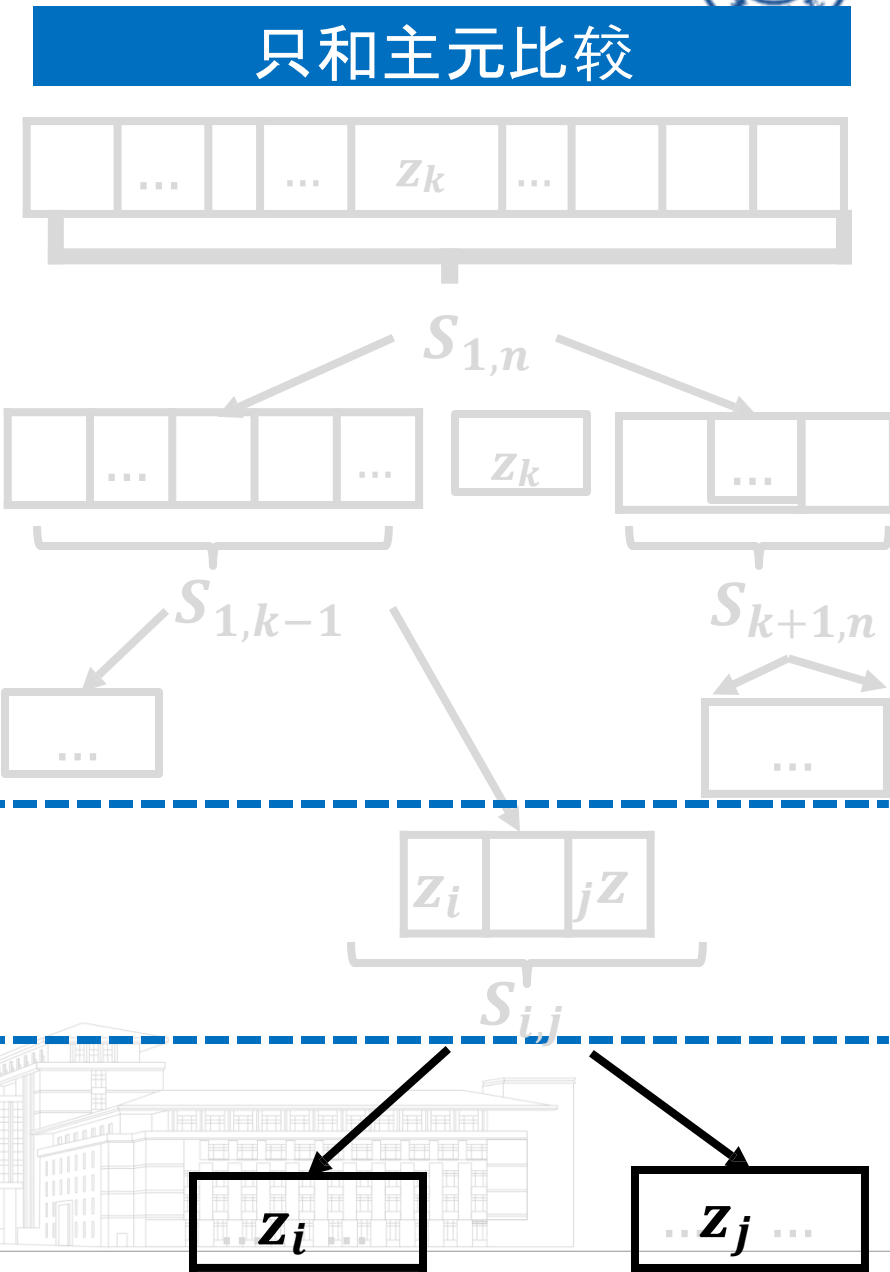
$S_{i,j}$ 划分后



随机化快速排序：复杂度分析

• 推导过程

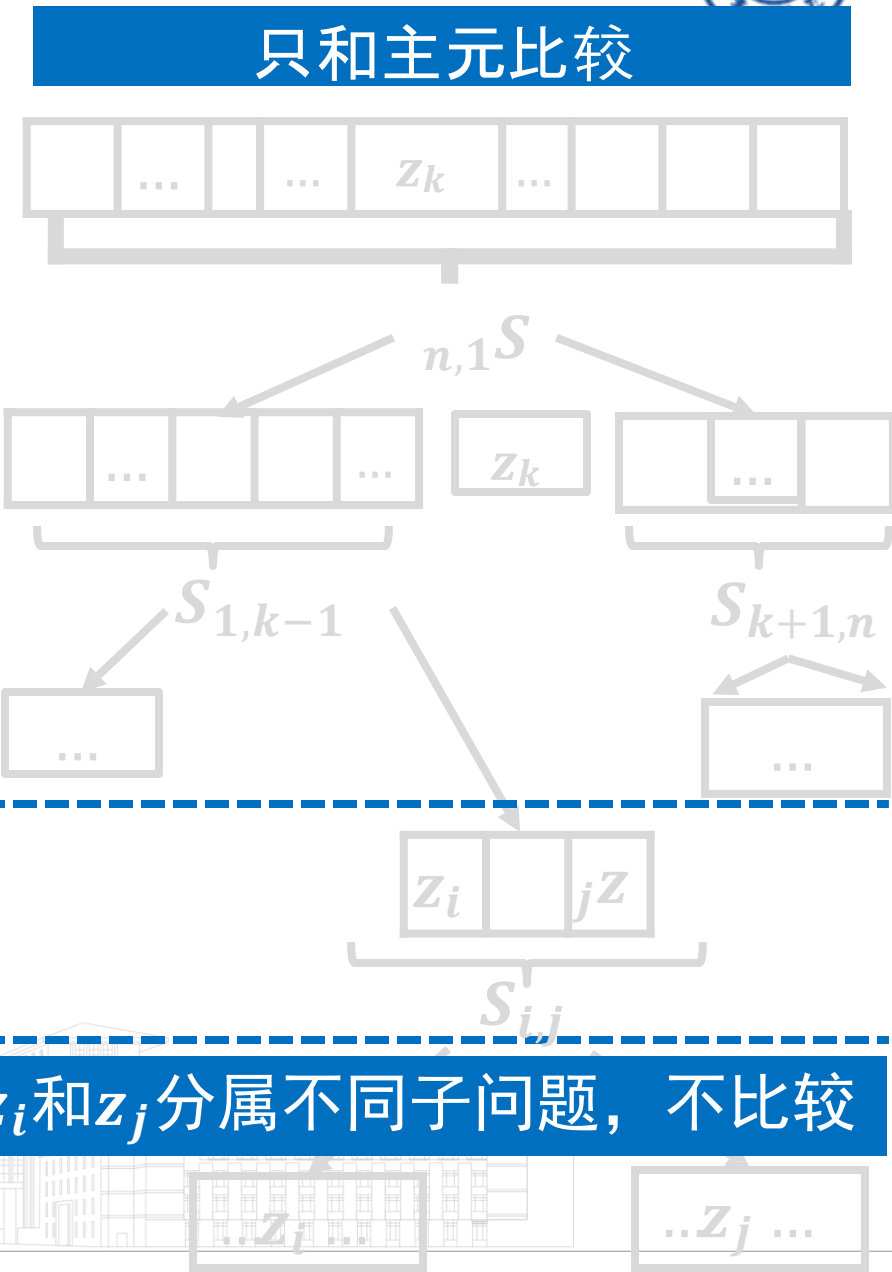
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

• 推导过程

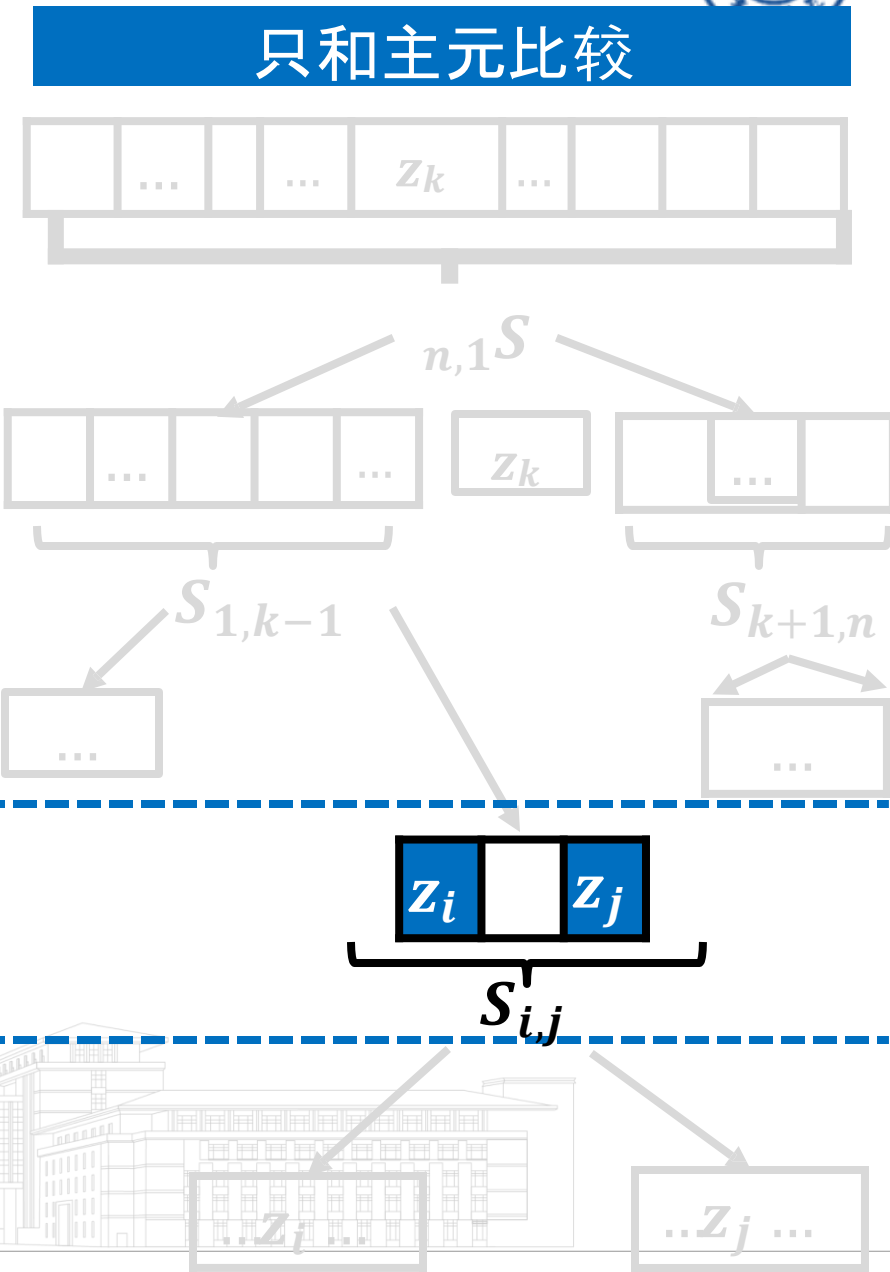
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

• 推导过程

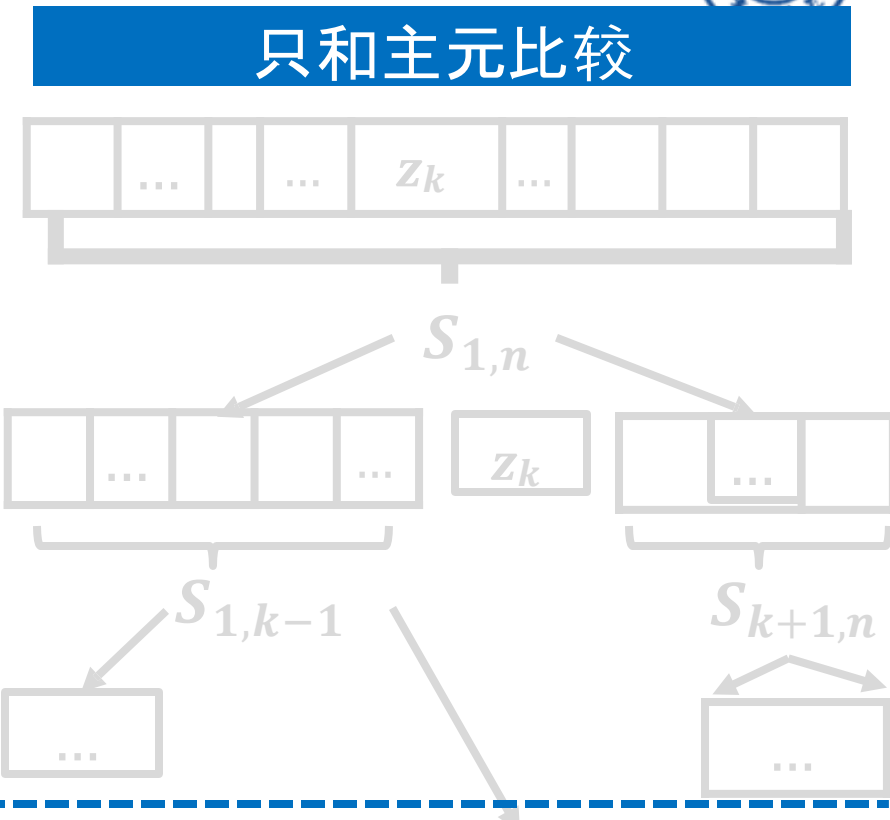
- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



随机化快速排序：复杂度分析

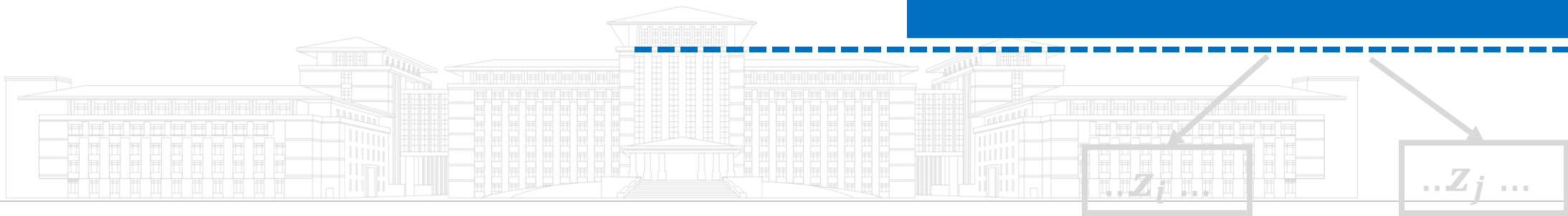
• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = ?$



$S_{i,j}$ 划分时

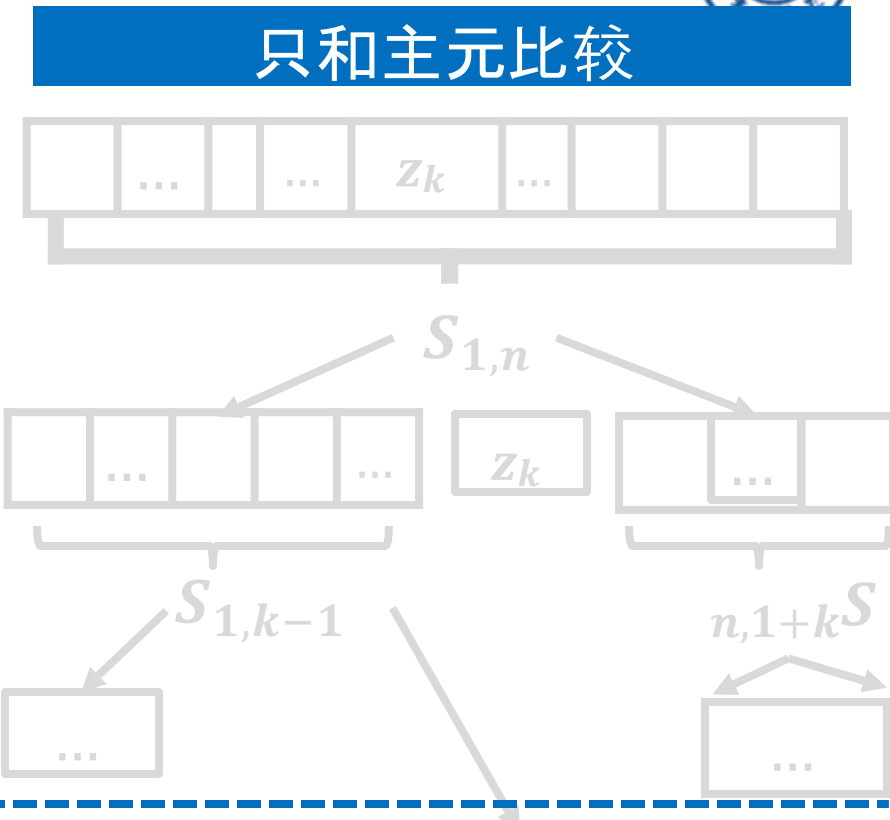
当 z_i 或 z_j 被选为主元时比较



随机化快速排序：复杂度分析

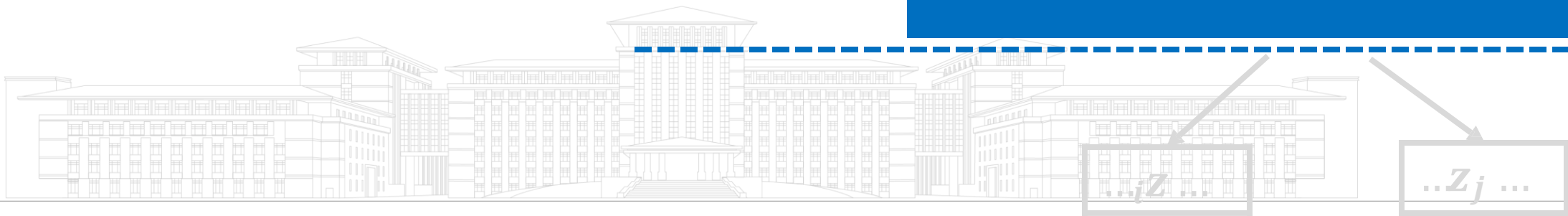
• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = \Pr[z_i \text{ 或者 } z_j \text{ 被选为主元}]$



$S_{i,j}$ 划分时

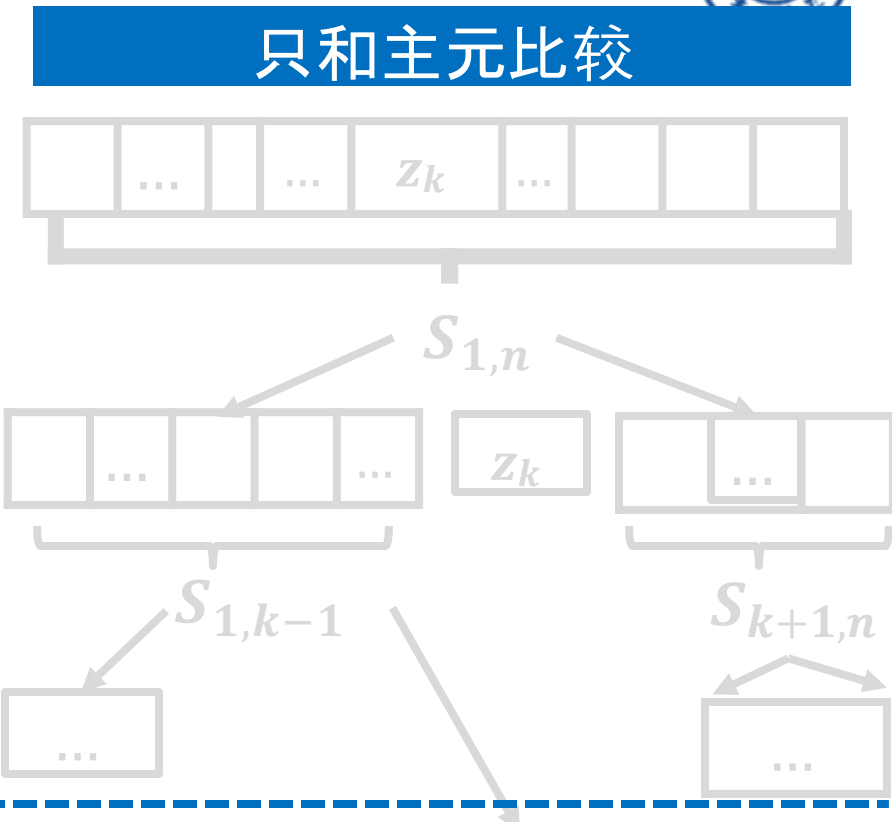
当 z_i 或 z_j 被选为主元时比较



随机化快速排序：复杂度分析

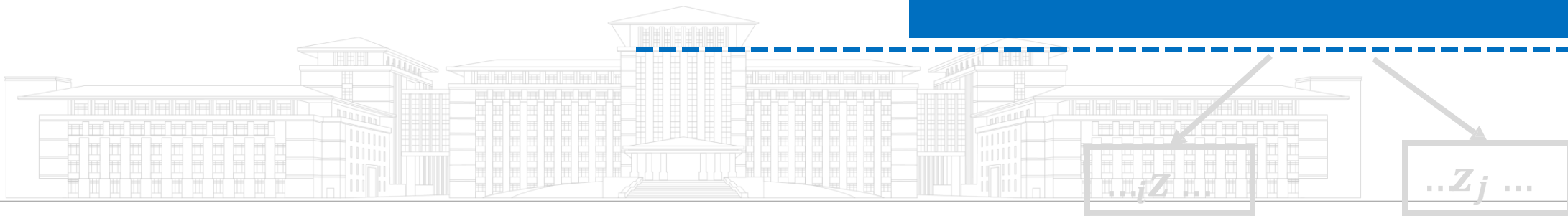
• 推导过程

- 随机变量 X_{ij} : z_i 和 z_j 比较的次数
- $E[X] = E[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$
- $E[X_{ij}] = \Pr[z_i \text{ 或者 } z_j \text{ 被选为主元}]$
 $= \Pr[z_i \text{ 被选为主元}] + \Pr[z_j \text{ 被选为主元}]$
 $= \frac{1}{j-i+1} + \frac{1}{j-i+1}$
 $= \frac{2}{j-i+1}$



$S_{i,j}$ 划分时

当 z_i 或 z_j 被选为主元时比较



随机化快速排序：复杂度分析

- 推导过程

- 随机变量 X_{ij} ： z_i 和 z_j 比较的次数

- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$

- $E[X_{ij}] = \frac{2}{j-i+1}$

- $\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} =$



随机化快速排序：复杂度分析

- 推导过程

- 随机变量 X_{ij} ： z_i 和 z_j 比较的次数

- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$

- $E[X_{ij}] = \frac{2}{j-i+1}$

- $\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1}$

$k = j - i$



随机化快速排序：复杂度分析

- 推导过程

- 随机变量 X_{ij} ： z_i 和 z_j 比较的次数

- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$

- $E[X_{ij}] = \frac{2}{j-i+1}$

- $\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k}$



随机化快速排序：复杂度分析

- 推导过程

- 随机变量 X_{ij} ： z_i 和 z_j 比较的次数

- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$

- $E[X_{ij}] = \frac{2}{j-i+1}$

- $\sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k}$
 $= \sum_i^{n-1} O(\log n)$

调和级数





随机化快速排序：复杂度分析

- 推导过程

- 随机变量 X_{ij} ： z_i 和 z_j 比较的次数

- $E[X] = E\left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}\right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n E[X_{ij}]$

- $E[X_{ij}] = \frac{2}{j-i+1}$

- $$\begin{aligned} \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} < \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} \\ &= \sum_i^{n-1} O(\log n) \\ &= O(n \log n) \end{aligned}$$

期望时间复杂度： $O(n \log n)$

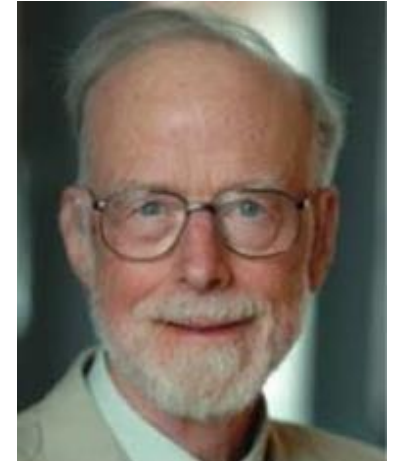
排序算法比较



约翰 • 冯 • 诺伊曼
John von Neumann

1945年提出

算法名称	时间复杂度
选择排序	$O(n^2)$
插入排序	$O(n^2)$
归并排序	$O(n \log n)$
快速排序	最差: $O(n^2)$
	期望: $O(n \log n)$



托尼 • 霍尔
Tony Hoare

1961年提出

思考：能否突破 $O(n \log n)$ ？

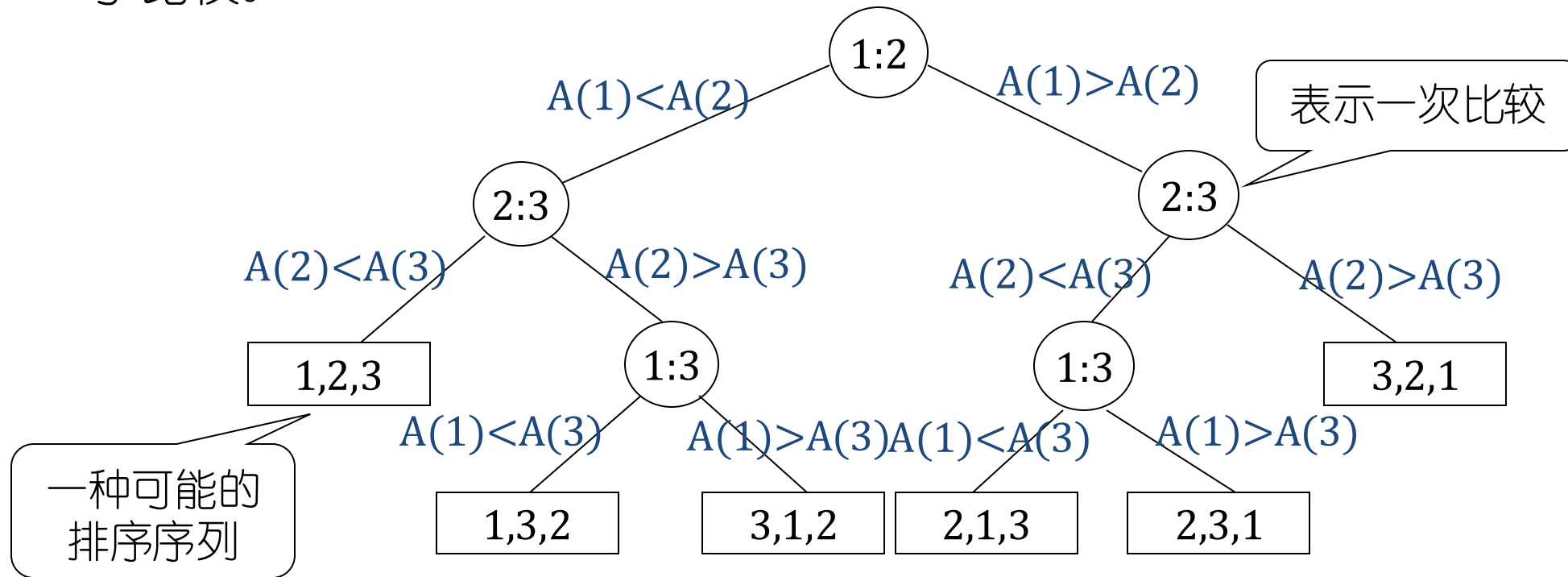


以比较为基础排序的时间下界

- 任何以关键字比较为基础的排序算法，最坏情况下的时间下界都是 $\Omega(n \log n)$ ，因此从数量级角度看, 归并算法是最坏情况下的最优算法。



- 假设参加排序的 n 个关键字 $A(1), \dots, A(n)$ 各不相同。采用二元比较树记录关键字比较。



对3个关键字排序的二元比较树



证明

- 已知 n 个关键字有 $n!$ 种排列。
- 观察二元比较树从根到外节点的每一条路径，分别与一种唯一的排列相对应。则比较树必定至少有 $n!$ 个外节点。且最坏情况下比较次数等于树高 $T(n)$ 。
- 则必有 $2^{T(n)} \geq n!$

$$2^{T(n)} \geq n! \geq n(n-1)(n-2) \dots (\lfloor n/2 \rfloor) \geq (n/2)^{n/2}$$

$$T(n) \geq (n/2) \log(n/2) \geq (n/4) \log n$$

$$T(n) = \Omega(n \log n)$$

$n \geq 4$ 时, 有
 $n/2 \geq n^{1/2}$

任何以关键字比较为基础的排序算法，最坏情况下的时间下界都是 $\Omega(n \log n)$ 。



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



矩阵相乘问题

- 假定矩阵A和B的级数n是2的幂，否则，添加适当的全零行和全零列。
- $A_{n \times n}$ 和 $B_{n \times n}$ 的乘积矩阵 $C_{n \times n}$ 中的元素 $C[i,j]$ 定义为：

$$C[i][j] = \sum_{k=1}^n A[i][k]B[k][j]$$



矩阵相乘问题

- 输入：矩阵A和B，级数n是2的幂
- 输出：A和B的矩阵乘积C
- 直接求解的时间复杂度
 - 令乘法运算为基本运算
 - 一个元素 $C[i][j]$ 需要做n次乘法和n-1次加法
 - 矩阵C一共有 n^2 个元素

$$O(n^3)$$



分治法求矩阵相乘问题

- 将A和B都分成4个 $(n/2) \times (n/2)$ 的方形矩阵。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

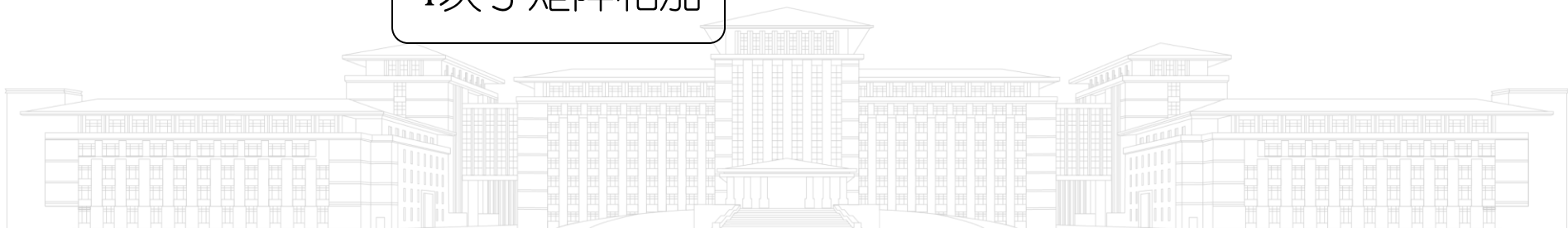
$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

- 上述分治法求解的时间复杂度

$$T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$$

4次子矩阵相加



分治法求矩阵乘法时间复杂度

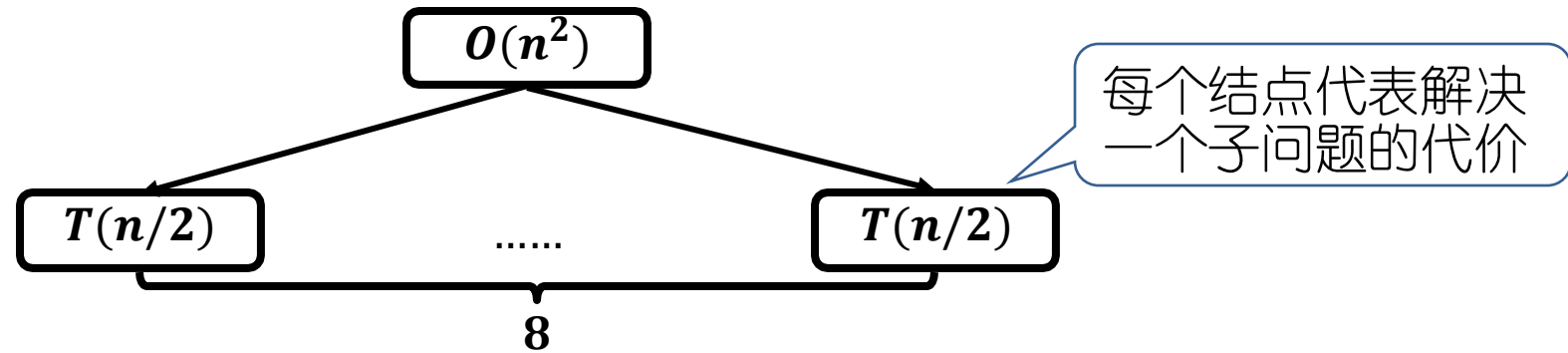
- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$

$T(n)$



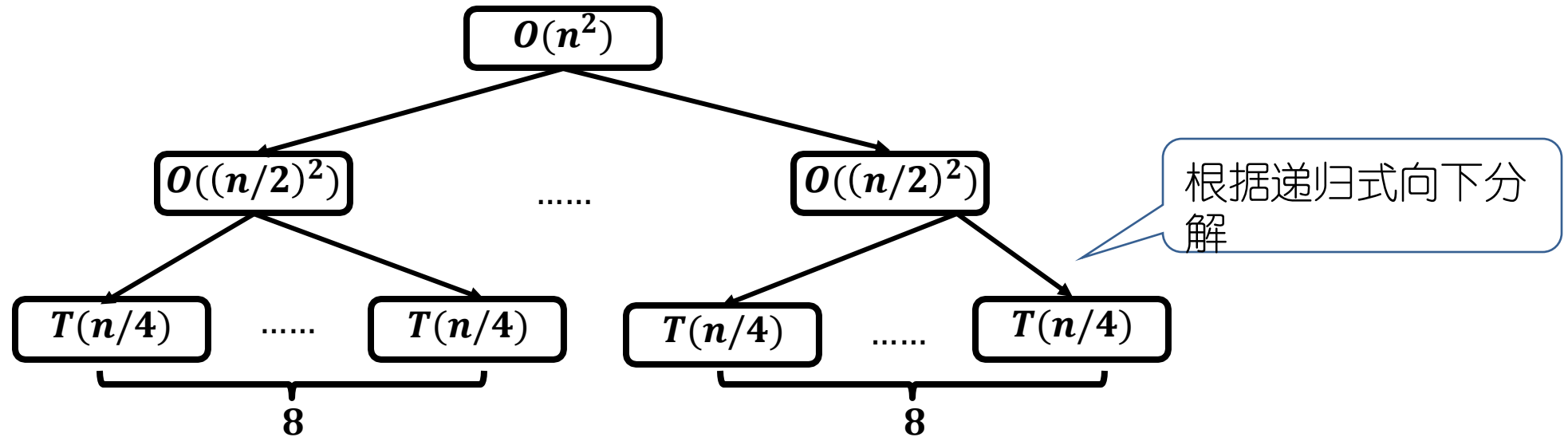
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归
$$T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$$



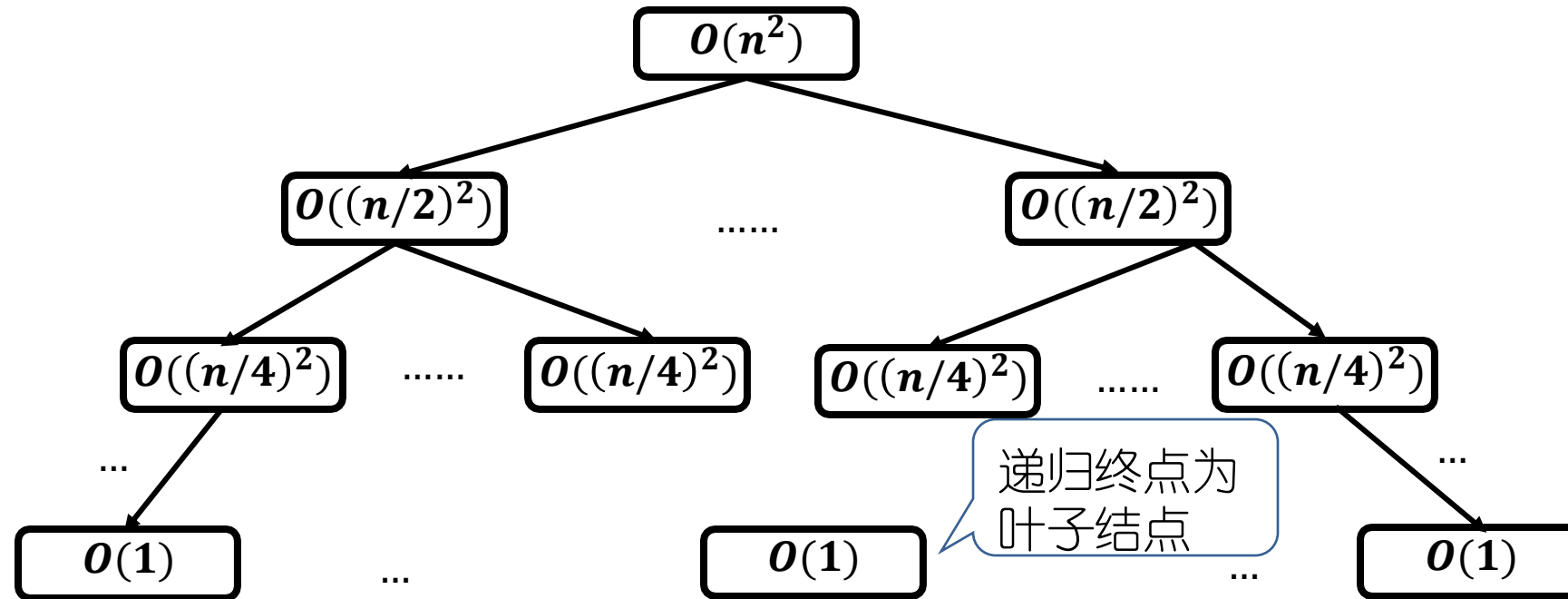
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归
$$T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$$



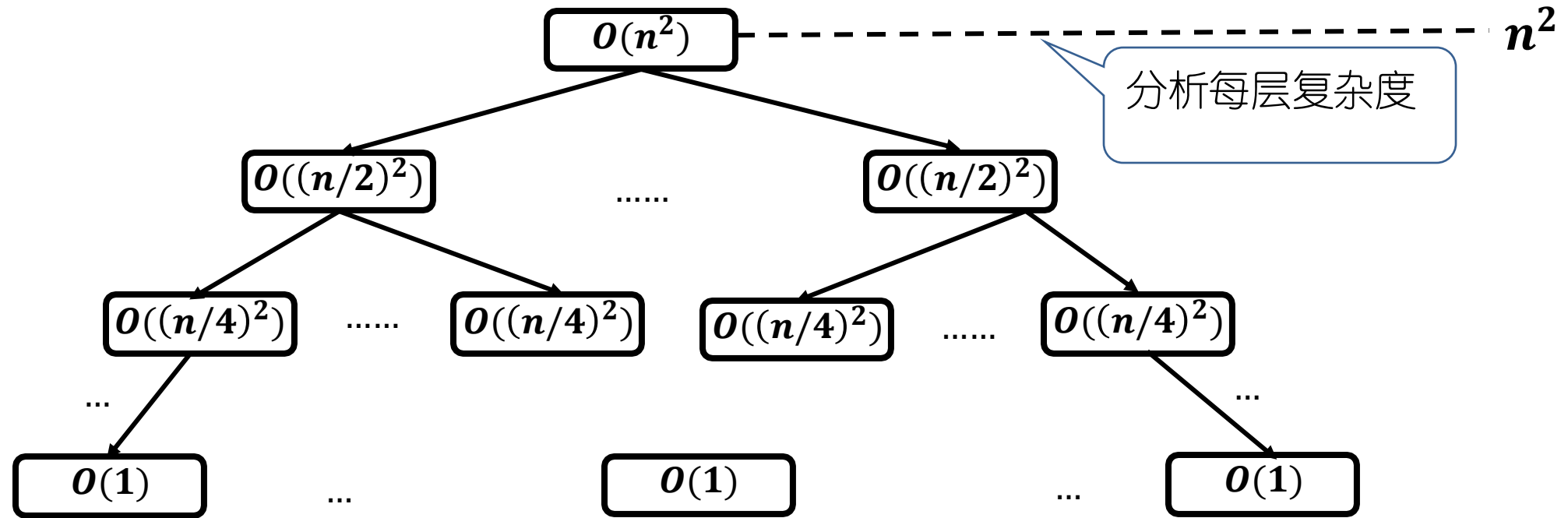
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$



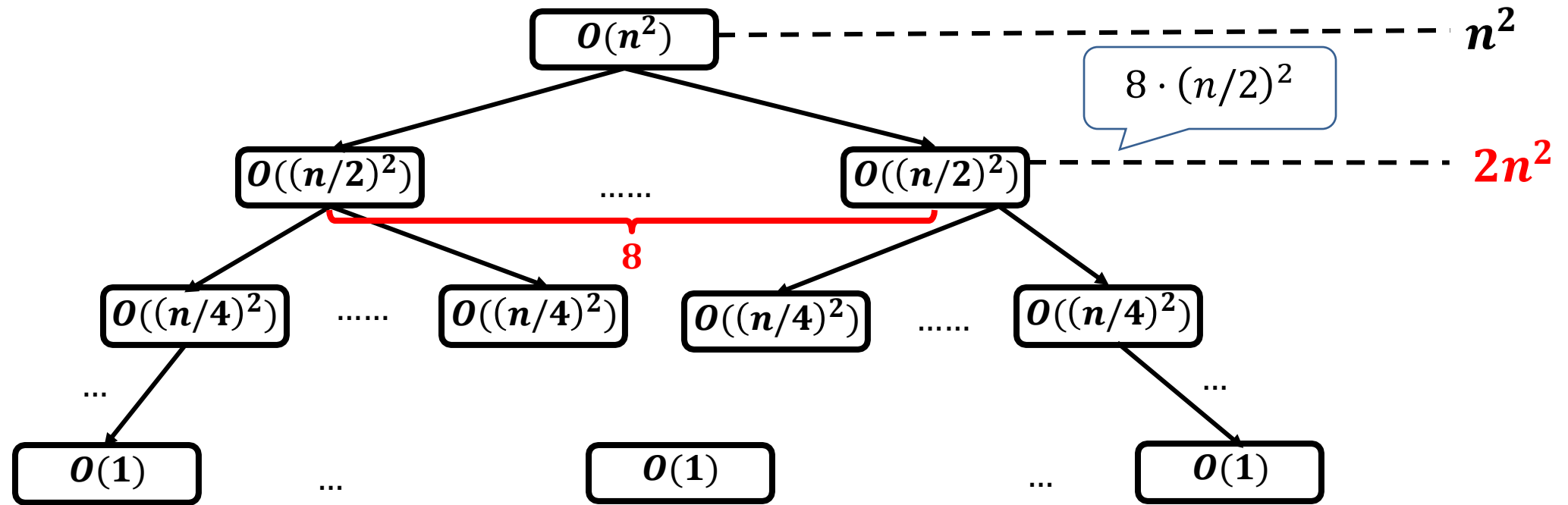
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$



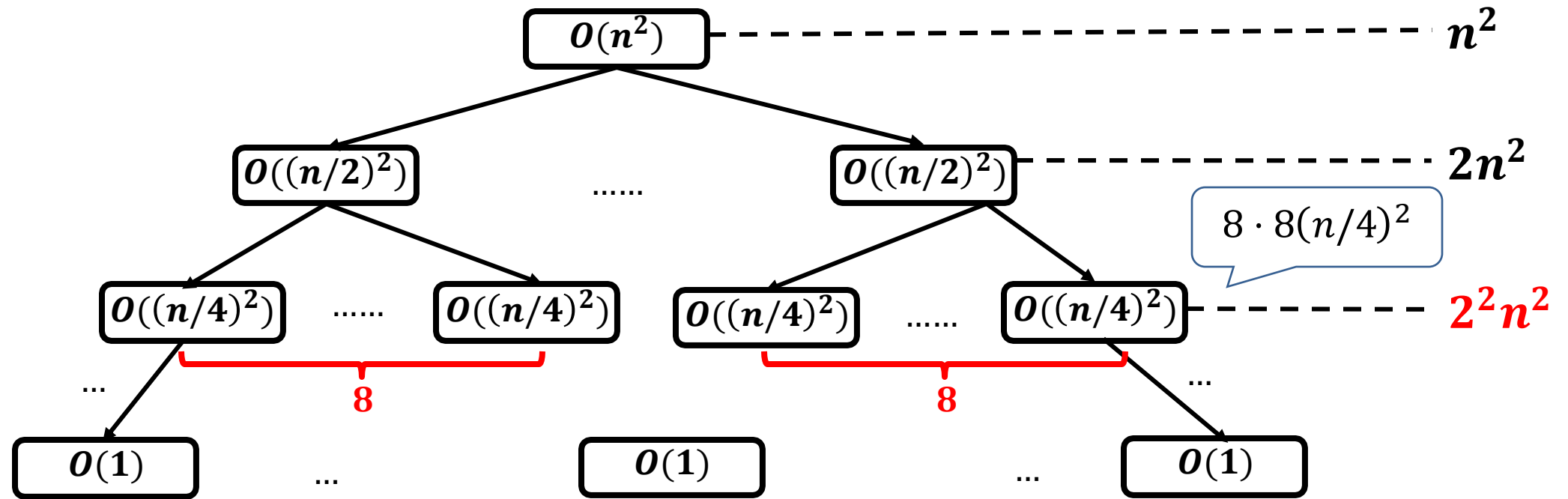
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$



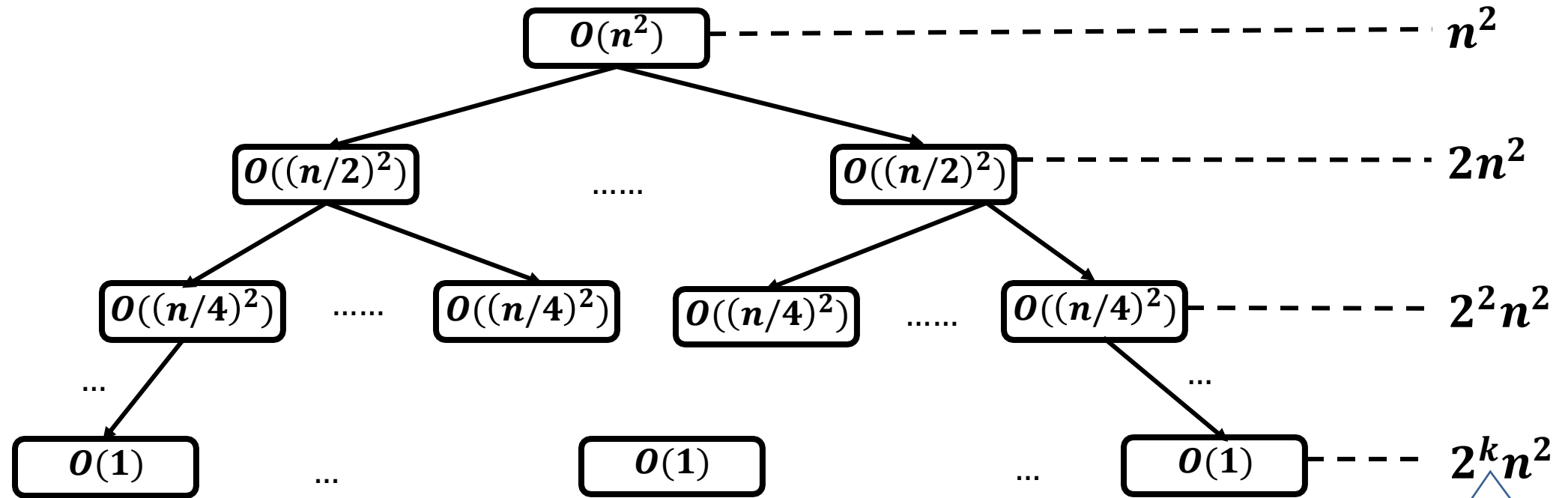
分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$



分治法求矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases}$



k 是树的深度



分治法求矩阵相乘问题

- 将A和B都分成4个 $(n/2) \times (n/2)$ 的方形矩阵。

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$C_{11} = A_{11}B_{11} + A_{12}B_{21}$$

$$C_{12} = A_{11}B_{12} + A_{12}B_{22}$$

$$C_{21} = A_{21}B_{11} + A_{22}B_{21}$$

$$C_{22} = A_{21}B_{12} + A_{22}B_{22}$$

- 上述分治法求解的时间复杂度

$$T(n) = \begin{cases} b & n = 2 \\ 8T(n/2) + dn^2 & n > 2 \end{cases} \quad \boxed{O(n^3)}$$

4次子矩阵相加

子问题个数多

思考：当前分治法没有提高求解效率的原因是什么？

斯特拉森矩阵乘法思想

- 斯特拉森在分治法的基础上，利用技巧减少了子问题的个数
 - 用7个乘法和10个加(减)法来算出7个 $(n/2)*(n/2)$ 的中间矩阵
 - 用8个加(减)法算出子问题的解 C_{ij}

$$M_1 = A_{11}(B_{12} - B_{22})$$

$$M_2 = (A_{11} + A_{12})B_{22}$$

$$M_3 = (A_{21} + A_{22})B_{11}$$

$$M_4 = A_{22}(B_{21} - B_{11})$$

$$M_5 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_6 = (A_{12} - A_{22})(B_{21} + B_{22})$$

$$M_7 = (A_{11} - A_{21})(B_{11} + B_{12})$$

$$C_{11} = M_5 + M_4 - M_2 + M_6$$

$$C_{12} = M_1 + M_2$$

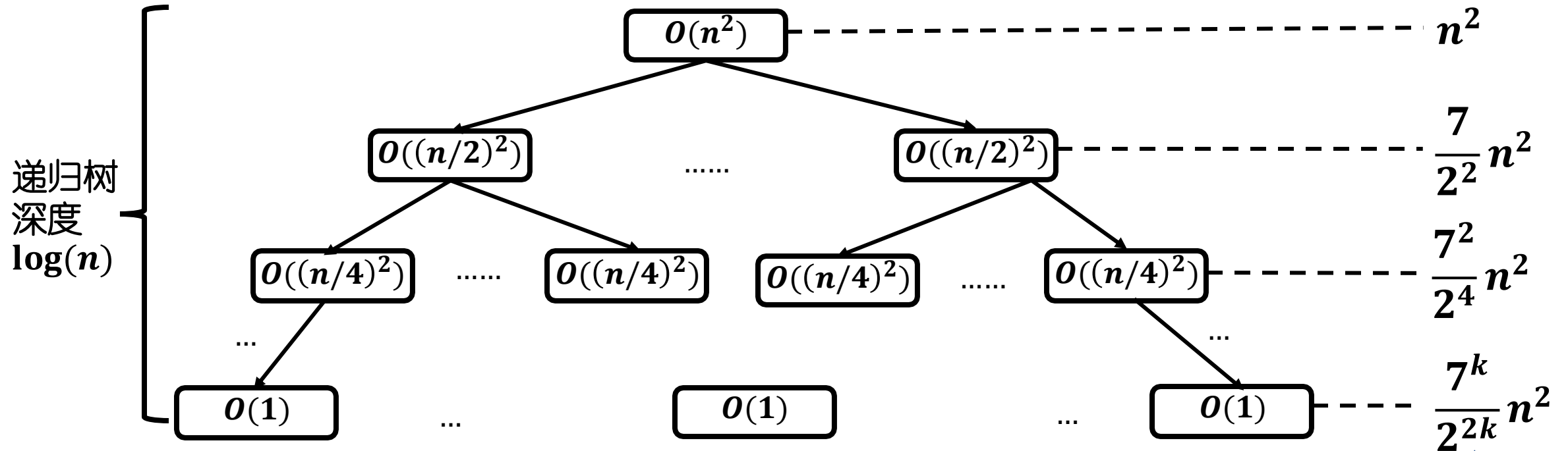
$$C_{21} = M_3 + M_4$$

$$C_{22} = M_5 + M_1 - M_3 - M_7$$



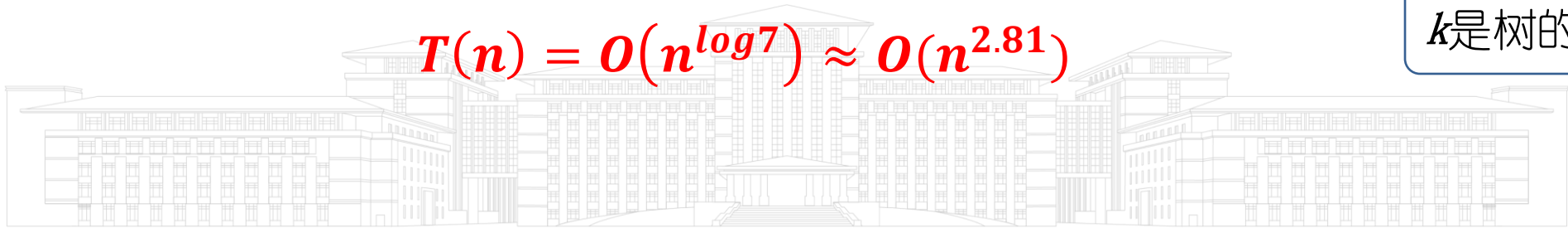
斯特拉森矩阵乘法时间复杂度

- 递归树法：用树表示递归 $T(n) = \begin{cases} O(1) & n = 2 \\ 7T(n/2) + O(n^2) & n > 2 \end{cases}$



k 是树的深度

$$T(n) = O(n^{\log 7}) \approx O(n^{2.81})$$



总结

- 分治法不一定总会提高算法效率
- 减少子问题个数是降低时间复杂度的一个有效途径



John Hopcroft
1986图灵奖得主

Hopcroft和Kerr已经证明(1971)，计算2个 2×2 矩阵的乘积，7次乘法是必要的。因此，要想进一步改进矩阵乘法的时间复杂性，就不能再基于计算 2×2 矩阵的7次乘法这样的方法了。或许应当研究 3×3 或 5×5 矩阵的更好算法。



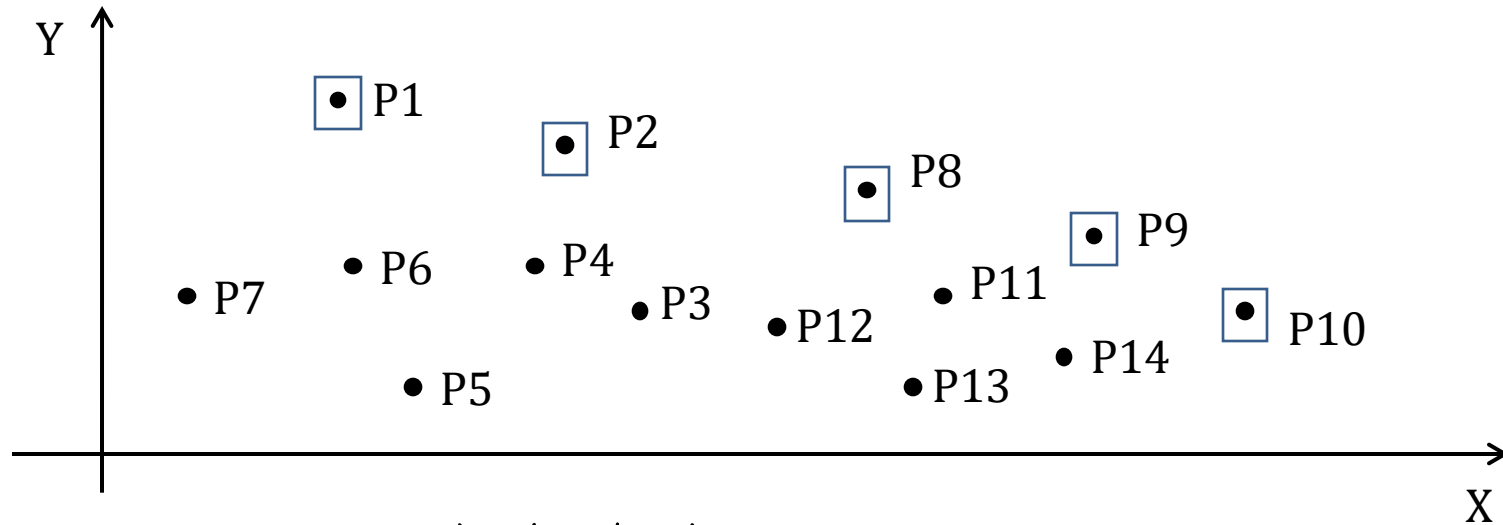
目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结

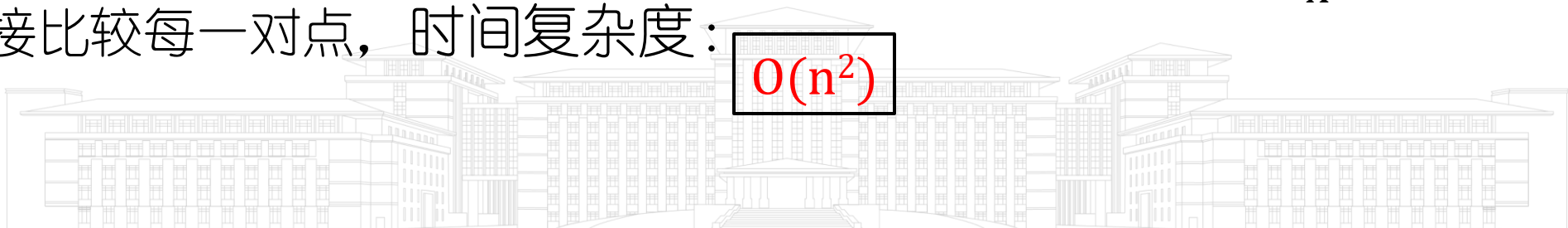


问题描述

- 支配规则：在二维空间中，如果 $x_1 > x_2$ ，且 $y_1 > y_2$ ，那么称点 (x_1, y_1) 支配点 (x_2, y_2)
- 极大点：如果一个点没有被其他点支配，则称其为极大点。
- 求极大点问题：已知有 n 个点的集合，找出其中的所有极大点。



- 直接比较每一对点，时间复杂度： $O(n^2)$



分治法设计思想

- 分：设计中位线 L ，将整个点集分为两个子集 S_L 和 S_R
 - 治：分别找出 S_L 和 S_R 的极大点
 - 合：基于支配规则合并 S_L 和 S_R 的极大点
-
- 思考1：怎样确定中位线？
 - 思考2：怎样设计递归出口？
 - 思考3：怎样实现支配规则？



问题分析

- 中位线：
 - 垂直于X轴的中位线L
 - 对集合S中的所有点按x值排序后，寻找第 $n/2$ 点位置
- 递归出口：
 - 如果集合S中只有一个点，那么该点为极大点
- 基于支配规则合并 S_L 和 S_R ：
 - 对于 S_L 中的极大点p，如果 y_p 小于 S_R 中极大点的y值，则p被支配，舍弃掉

可以是任何曲线，要选择最简单的方式，有利于提高效率。

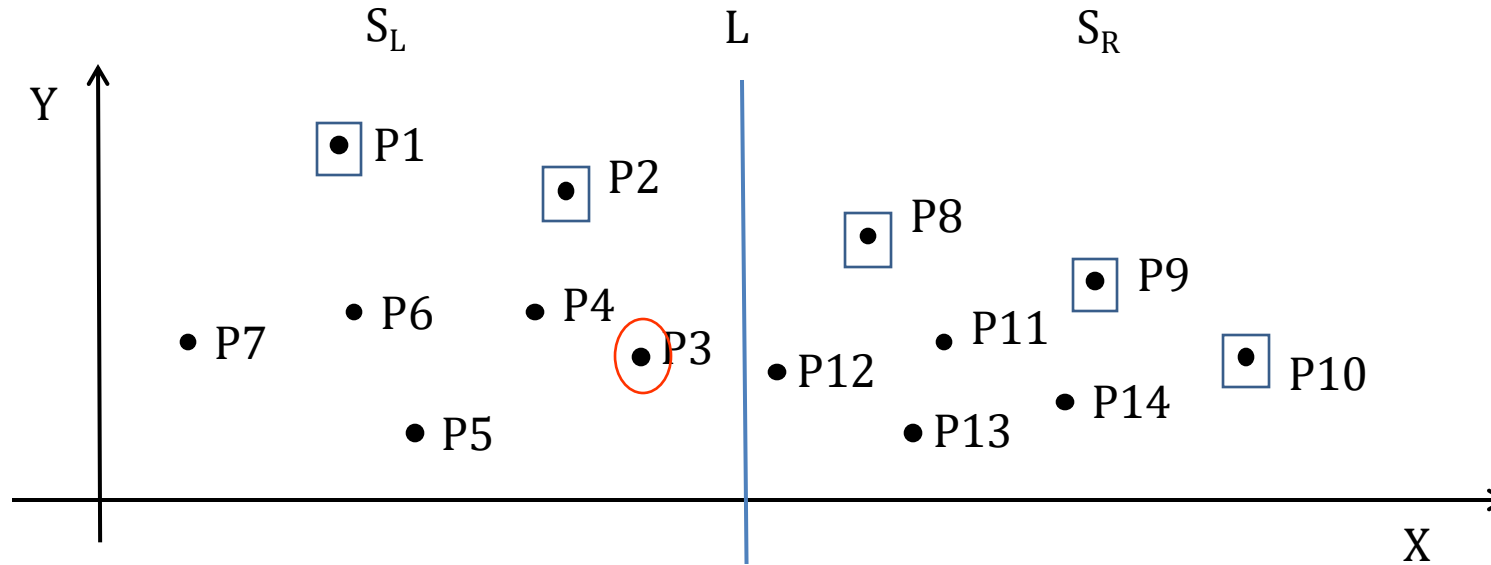
S_L 中的点的x值总是小于 S_R 中的点的x值



分治法描述：求二维极大点问题

- 输入： n 个平面点的集合 S
- 输出： S 的极大点
- step1：如果 S 只有一个点，输出该点为极大点；否则，寻找中位线 L ，将 S 划分为两个子集合 S_L 和 S_R ，使每个子集含有 $n/2$ 个点。
- step2：递归找出 S_L 和 S_R 的极大点。
- step3：判断 S_L 的所有极大点 $p \in S_L$ ，如果 y_p 值小于 S_R 中极大点的最大 y 值，那么舍弃点 p 。





$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + \text{中位线}L\text{的计算时间} + \text{基于支配规则合并的时间} & n>1 \end{cases}$$

治: $O(n)$ 分: $O(n)$ 合: $O(n)$

$\Omega(n \log n)$

预处理：提前排序

思考1：能够优化中位线的计算时间？是否有必要每次求解子问题都重新排序？

思考2： S_L 和 S_R ，先求解哪一个更合理？

时间复杂度

$$T(n) = \begin{cases} 1 & n=1 \\ 2T(n/2) + \underbrace{O(n)}_{\text{治}} + \underbrace{O(n)}_{\text{分}} + \underbrace{O(n)}_{\text{合}} & n>1 \end{cases}$$

$O(n \log n)$

工作量放到递归过程之外，降低重复处理

- 具有预排序的分治策略找极大点的时间复杂度是 $O(n \log n)$
 - 预排序 $O(n \log n)$
 - 分治法 $O(n \log n)$
- 优于直接方法的时间复杂度 $O(n^2)$



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结



效率低的原因

- 一些情况下，分治法比蛮力法(直接求解)明显改进了效率，但是也并不是处处有效
 - 原因1：子问题个数多，发生在**治**的环节
 - 原因2：递归过程内工作量过多，发生在**分/合**的环节

优化思路：变多为少



优化策略1

- 针对子问题个数多的问题
 - 寻找子问题之间的依赖关系，如果一个子问题的解可以通过其他子问题的解简单运算得到，那么不必重新递归计算该子问题的解，而是通过上述简单运算获得。
 - 斯特拉森矩阵相乘问题：矩阵相乘减少一个子问题个数
 - 归并排序问题：减少小规模下子问题个数



优化策略2

- 针对递归过程内工作量过多的问题
 - 把某些工作提取到递归过程之外预处理，从而减少递归内部的调用工作量
 - 二维极大点问题：预排序 x 值，求子问题时查询该序列，确定中位线



目录

- 4.1 一般方法
- 4.2 二分查找
- 4.3 归并排序
- 4.4 快速排序
- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
- 4.8 递归式分析方法
- 4.9 小结





递归树法

$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$



递归树法

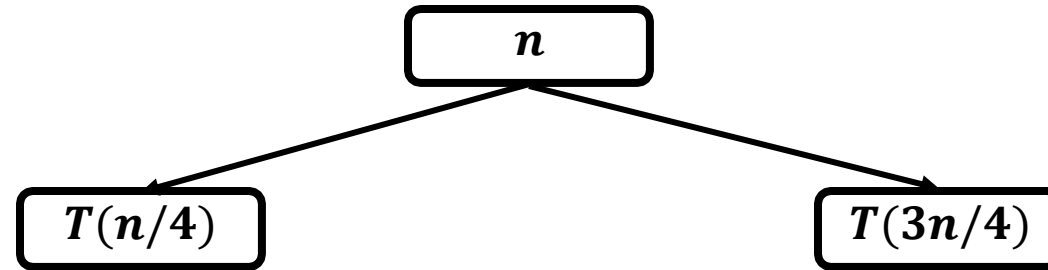
$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$

$T(n)$



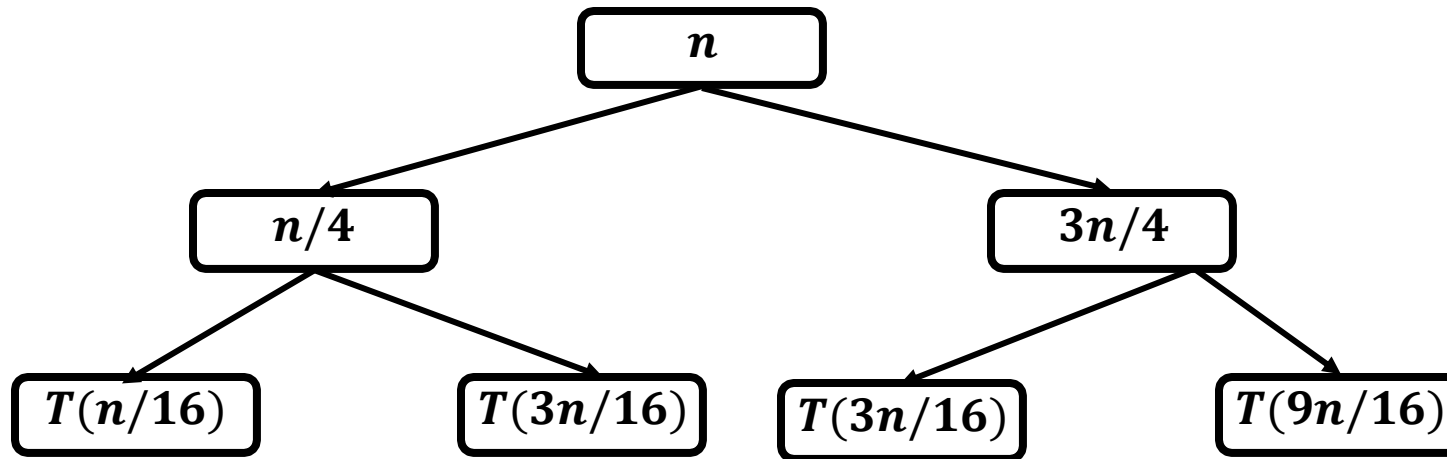
递归树法

$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$



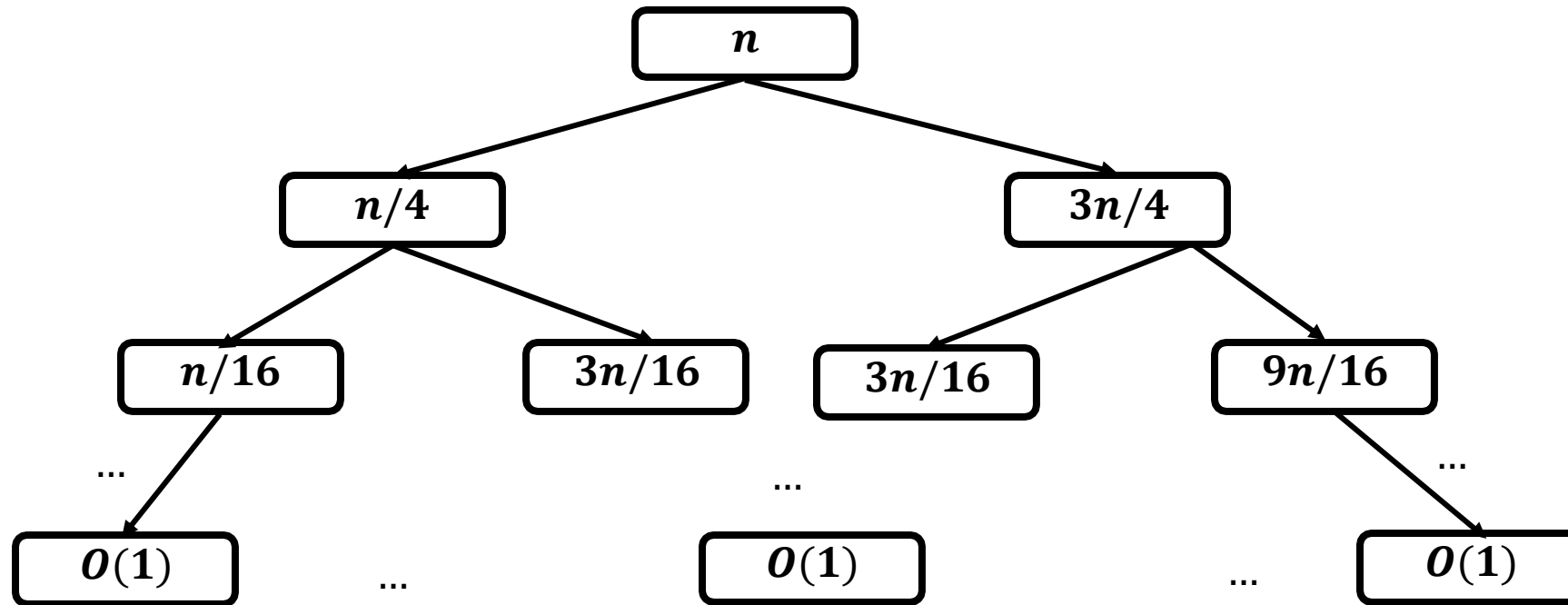
递归树法

$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$



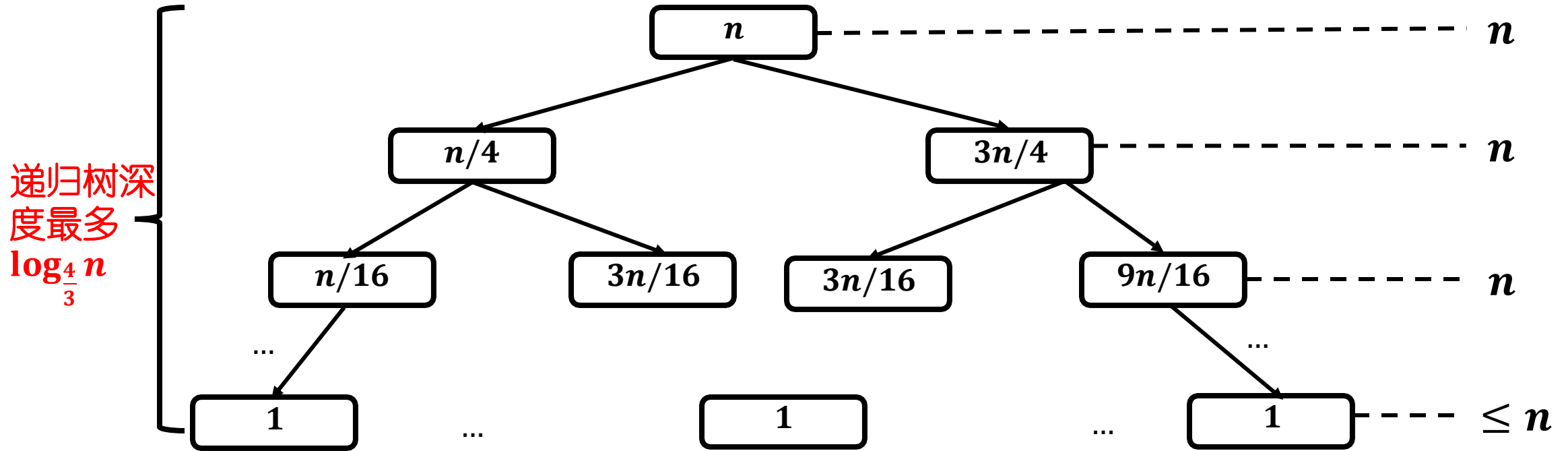
递归树法

$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$



递归树法

$$T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$$



$$T(n) = O\left(n \log_{\frac{4}{3}} n\right) = O\left(n \frac{\log_2 n}{\log_2 4/3}\right) = O(n \log n)$$

思考：是否是渐进紧确界？

代入法

- $T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$
- 猜想（瞪眼法）： $T(n) = \Theta(n \log n)$
 - 证明 $\exists c_1, c_2, n_0 > 0$, 使得 $\forall n > n_0, c_1 \cdot n \log n \leq T(n) \leq c_2 \cdot n \log n$

数学归纳法！



代入法

- $T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$
- 猜想（瞪眼法）： $T(n) = \Theta(n \log n)$
 - 证明 $\exists c_1, c_2, n_0 > 0$, 使得 $\forall n > n_0, c_1 \cdot n \log n \leq T(n) \leq c_2 \cdot n \log n$
- 数学归纳法
 - 基础情况： $n = 3$ 时, 使 $c_1 \cdot 3 \log 3 \leq 1 \leq c_2 \cdot 3 \log 3$, 取 $0 < c_1 < \frac{1}{3 \log 3}, c_2 \geq \frac{1}{3 \log 3}$
 - 小于 n 时： 假设命题成立
 - 等于 n 时： 代入假设
 - $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n$



代入法



$$\begin{aligned} T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\ &\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \end{aligned}$$



代入法

$$\begin{aligned}T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\&\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\&= c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4) + c_2 \frac{3n}{4} \cdot \left(\log n - \log \frac{4}{3}\right) + n\end{aligned}$$



代入法

$$\begin{aligned}T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\&\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\&= \boxed{c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4)} + \boxed{c_2 \cdot \frac{3n}{4} \cdot (\log n - \log \frac{4}{3})} + n \\&= c_2 n \log n\end{aligned}$$



代入法



$$\begin{aligned}T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\&\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\&= c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4) + c_2 \frac{3n}{4} \cdot \left(\log n - \log \frac{4}{3}\right) + n \\&= c_2 n \log n - c_2 n \left(\frac{\log 4}{4} + \frac{3 \log 4}{4} - \frac{3 \log 3}{4}\right) + n\end{aligned}$$



代入法



$$\begin{aligned}T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\&\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\&= c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4) + c_2 \frac{3n}{4} \cdot \left(\log n - \log \frac{4}{3}\right) + n \\&= c_2 n \log n - c_2 n \left(\frac{\log 4}{4} + \frac{3 \log 4}{4} - \frac{3 \log 3}{4} \right) + n \\&= c_2 n \log n - \left(c_2 \left(\log 4 - \frac{3}{4} \log 3 \right) - 1 \right) n\end{aligned}$$

希望此式 $\leq c_2 n \log n$



代入法

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\
 &\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\
 &= c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4) + c_2 \frac{3n}{4} \cdot \left(\log n - \log \frac{4}{3}\right) + n \\
 &= c_2 n \log n - c_2 n \left(\frac{\log 4}{4} + \frac{3 \log 4}{4} - \frac{3 \log 3}{4} \right) + n \\
 &= c_2 n \log n - \left(c_2 \left(\log 4 - \frac{3}{4} \log 3 \right) - 1 \right) n
 \end{aligned}$$

只需此部分 ≥ 0



代入法

$$\begin{aligned}
 T(n) &= T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \\
 &\leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n \\
 &= c_2 \cdot \frac{n}{4} \cdot (\log n - \log 4) + c_2 \frac{3n}{4} \cdot \left(\log n - \log \frac{4}{3}\right) + n \\
 &= c_2 n \log n - c_2 n \left(\frac{\log 4}{4} + \frac{3 \log 4}{4} - \frac{3 \log 3}{4} \right) + n \\
 &= c_2 n \log n - \left(c_2 \left(\log 4 - \frac{3}{4} \log 3 \right) - 1 \right) n
 \end{aligned}$$

只需此部分 ≥ 0

- 令 $c_2 \left(\log 4 - \frac{3}{4} \log 3 \right) - 1 \geq 0$, 解得 $c_2 \geq \frac{1}{\log 4 - \frac{3}{4} \log 3}$



代入法

- $T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$
- 猜想（瞪眼法）： $T(n) = \Theta(n \log n)$
 - 证明 $\exists c_1, c_2, n_0 > 0$, 使得 $\forall n > n_0, c_1 \cdot n \log n \leq T(n) \leq c_2 \cdot n \log n$

数学归纳法

- 基础情况： $n = 3$ 时，使 $c_1 \cdot 3 \log 3 \leq 1 \leq c_2 \cdot 3 \log 3$, 取 $0 < c_1 < \frac{1}{3 \log 3}$, $c_2 \geq \frac{1}{3 \log 3}$
- 小于 n 时：假设命题成立
- 等于 n 时：代入假设

- $T(n) = T\left(\frac{n}{4}\right) + T\left(\frac{3n}{4}\right) + n \leq c_2 \cdot \frac{n}{4} \cdot \log \frac{n}{4} + c_2 \cdot \frac{3n}{4} \cdot \log \frac{3n}{4} + n$

- 取 $c_2 \geq \frac{1}{\log 4 - \frac{3}{4} \log 3}$, 则 $T(n) \leq c_2 n \log n$

- $c_2 \geq \max \left\{ \frac{1}{\log 4 - \frac{3}{4} \log 3}, \frac{1}{3 \log 3} \right\}$



代入法

- $T(n) = \begin{cases} T(n/4) + T(3n/4) + n & \text{if } n > 3 \\ 1 & \text{if } n = 3 \end{cases}$
- 猜想（瞪眼法）： $T(n) = \Theta(n \log n)$
 - 证明 $\exists c_1, c_2, n_0 > 0$, 使得 $\forall n > n_0, c_1 \cdot n \log n \leq T(n) \leq c_2 \cdot n \log n$
- 数学归纳法
 - 基础情况： $n = 3$ 时，使 $c_1 \cdot 3 \log 3 \leq 1 \leq c_2 \cdot 3 \log 3$, 取 $0 < c_1 < \frac{1}{3 \log 3}, c_2 \geq \frac{1}{3 \log 3}$
 - 小于 n 时： 假设命题成立
 - 等于 n 时： 代入假设
 - 取 $c_2 \geq \max \left\{ \frac{1}{\log 4 - \frac{3}{4} \log 3}, \frac{1}{3 \log 3} \right\}$, 则 $T(n) \leq c_2 \cdot n \log n$
 - 取 $0 < c_1 \leq \min \left\{ \frac{1}{\log 4 - \frac{3}{4} \log 3}, \frac{1}{3 \log 3} \right\}$, 则 $T(n) \geq c_1 \cdot n \log n$
 - 得证 $T(n) = \Theta(n \log n)$

练习

主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$a \geq 1, b > 1$



主定理

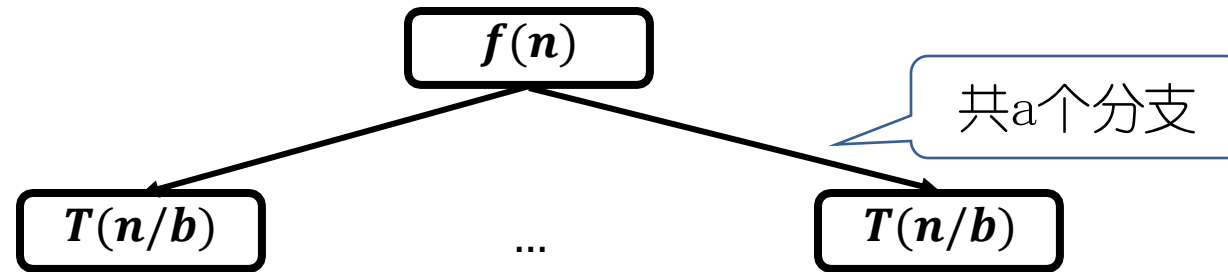
- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$f(n)$



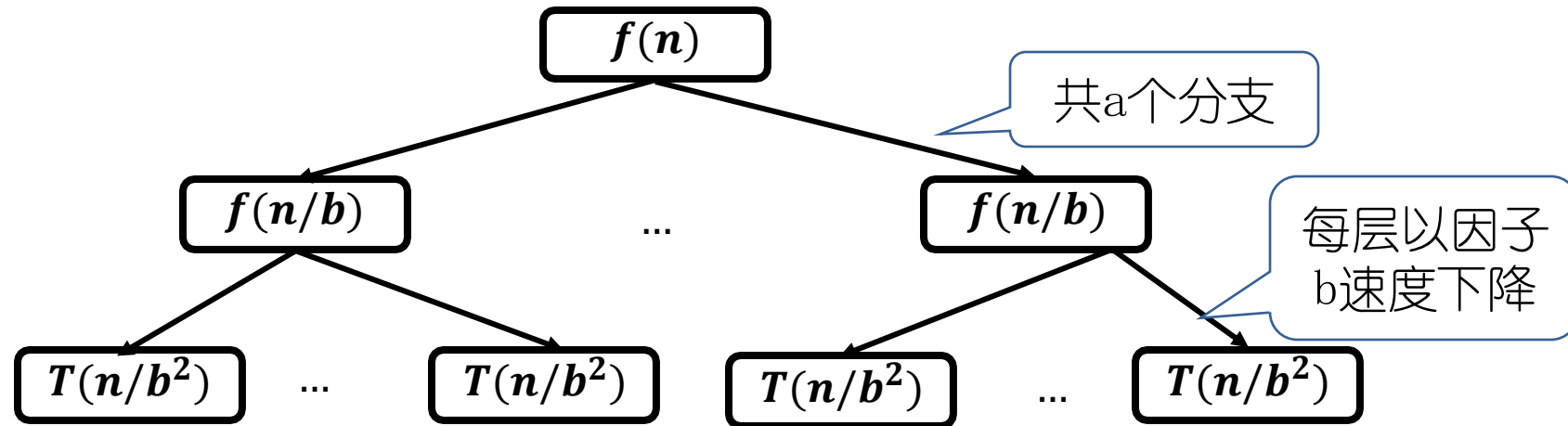
主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



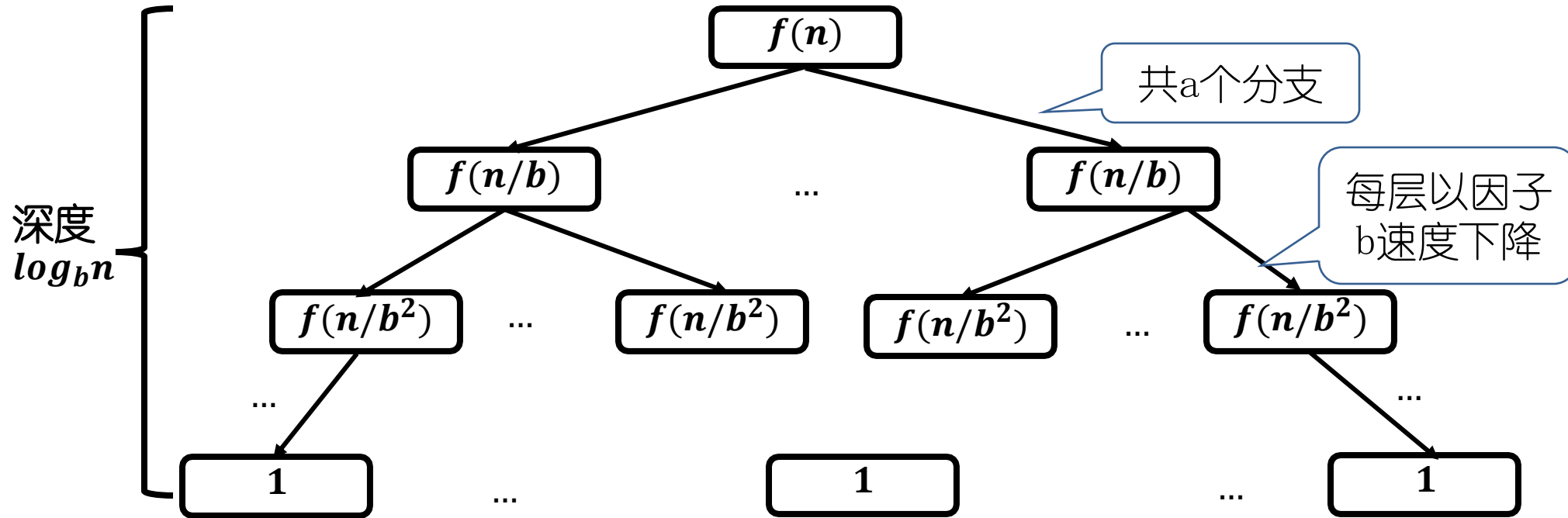
主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



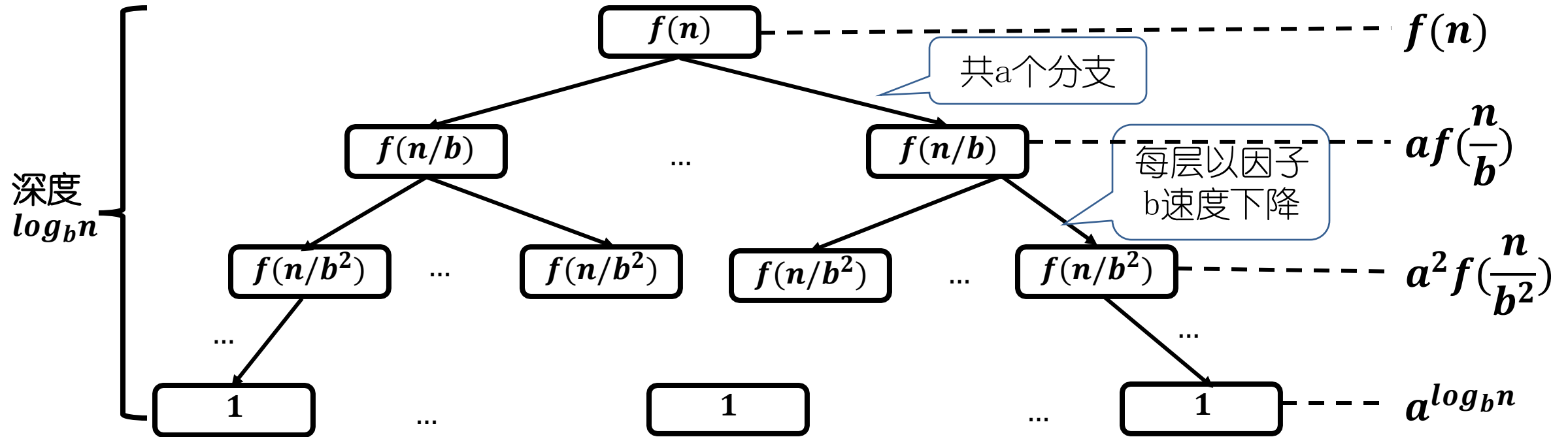
主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



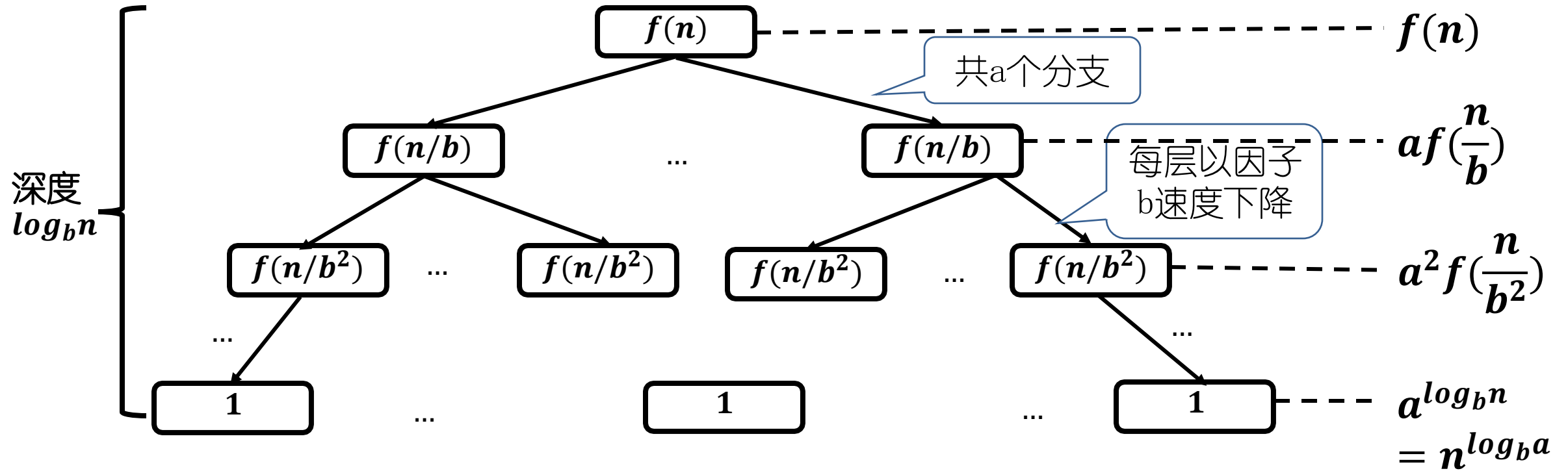
主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



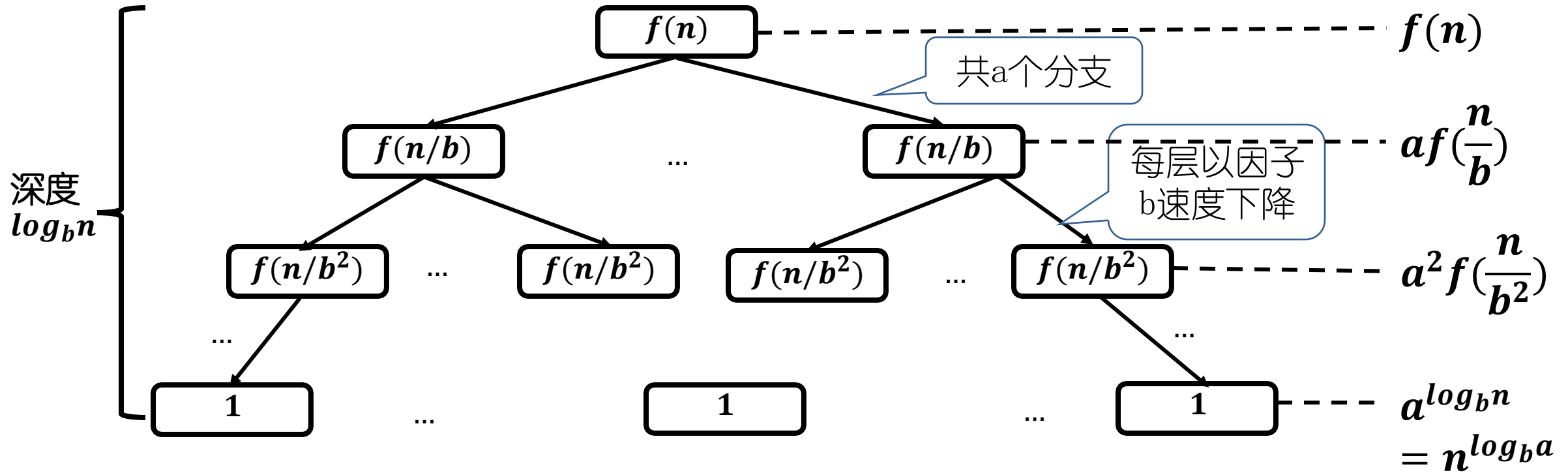
主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式



$$T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$$

主定理

$$a \geq 1, b > 1$$

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$
- 令 $g(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$



主定理

$$a \geq 1, b > 1$$

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$
- 令 $g(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$
 - 若存在常数 $\epsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $g(n) = O(n^{\log_b a})$

存在常数 c , 对足够大的 n , 有 $f(n) \geq cn^{\log_b a - \epsilon}$





主定理

$$a \geq 1, b > 1$$

• 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

• 令 $g(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

存在常数 c , 对足够大的 n , 有 $f(n) \geq cn^{\log_b a - \epsilon}$

• 若存在常数 $\epsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $g(n) = O(n^{\log_b a})$

$$g(n) \geq c \sum_{i=0}^{\log_b n - 1} a^i \left(\frac{n}{b^i}\right)^{\log_b a - \epsilon}$$

$$= c \sum_{i=0}^{\log_b n - 1} n^{\log_b a - \epsilon} \left(\frac{a}{b^{\log_b a - \epsilon}}\right)^i$$

$$= c \sum_{i=0}^{\log_b n - 1} n^{\log_b a - \epsilon} (b^\epsilon)^i$$

等比数列求和 $(n^\epsilon - 1)/(b^\epsilon - 1)$

$$= cn^{\log_b a - \epsilon} \sum_{i=0}^{\log_b n - 1} (b^\epsilon)^i$$

$$\leq c' n^{\log_b a}$$

主定理

$$a \geq 1, b > 1$$

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$
- 令 $g(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$
 - 若存在常数 $\epsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $g(n) = O(n^{\log_b a})$
 - 若 $f(n) = \Theta(n^{\log_b a})$, 则 $g(n) = \Theta(n^{\log_b a} \log n)$



主定理

$$a \geq 1, b > 1$$

• 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

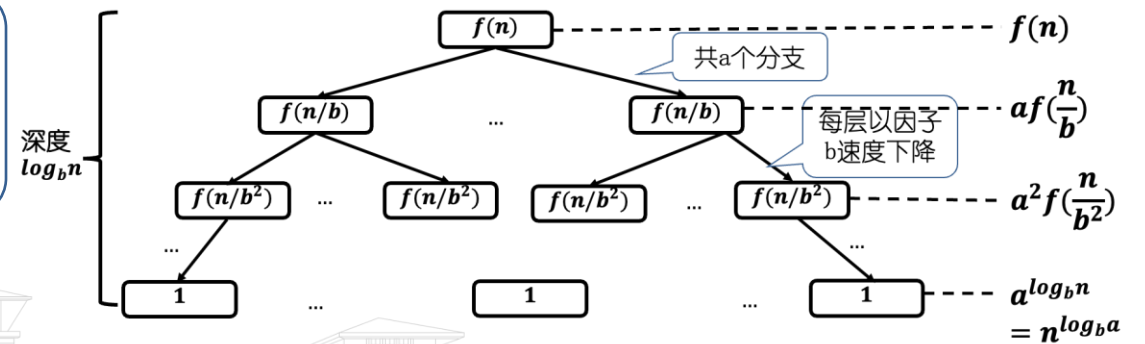
• 令 $g(n) = \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

• 若存在常数 $\epsilon > 0$, 使得 $f(n) = O(n^{\log_b a - \epsilon})$, 则 $g(n) = O(n^{\log_b a})$

• 若 $f(n) = \Theta(n^{\log_b a})$, 则 $g(n) = \Theta(n^{\log_b a} \log n)$

• 若 $f(n) = \Omega(n^{\log_b a + \epsilon})$, 且存在常数 $c < 1$ 和足够大的 n , 使得 $\frac{f(n/b)}{f(n)} \leq c/a$, 则 $g(n) = \Theta(f(n))$

保证根节点时间
大于下层节点时
间之和

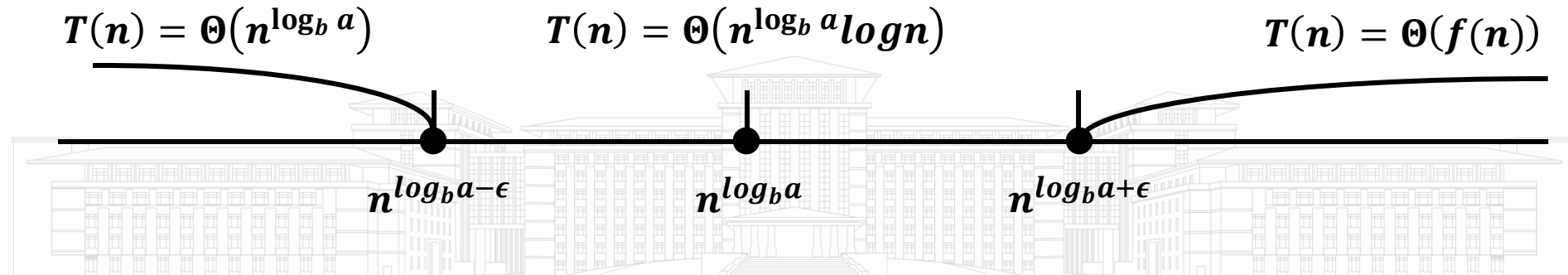


主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

额外假设



主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式: $T(n) = n^{\log_b a} + \sum_{i=0}^{\log_b n - 1} a^i f\left(\frac{n}{b^i}\right)$

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

额外假设

- 简化形式: 对形如 $T(n) = aT\left(\frac{n}{b}\right) + n^k$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } k < \log_b a \\ \Theta(n^k \log n) & \text{if } k = \log_b a \\ \Theta(n^k) & \text{if } k > \log_b a \end{cases}$$



主定理

- 简化形式：对形如 $T(n) = aT\left(\frac{n}{b}\right) + n^k$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } k < \log_b a \\ \Theta(n^k \log n) & \text{if } k = \log_b a \\ \Theta(n^k) & \text{if } k > \log_b a \end{cases}$$

- 例（归并排序）： $T(n) = 2T\left(\frac{n}{2}\right) + n$
 - $k=1, a=2, b=2$
 - $k=\log_b a$
 - $T(n) = \Theta(n \log n)$



主定理

- 简化形式：对形如 $T(n) = aT\left(\frac{n}{b}\right) + dn^k$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } k < \log_b a \\ \Theta(n^k \log n) & \text{if } k = \log_b a \\ \Theta(n^k) & \text{if } k > \log_b a \end{cases}$$

- 例（一般矩阵乘法）： $T(n) = 8T\left(\frac{n}{2}\right) + dn^2$
 - $k=2, a=8, b=2$
 - $k < \log_2 8$
 - $T(n) = \Theta(n^3)$



主定理

- 简化形式：对形如 $T(n) = aT\left(\frac{n}{b}\right) + dn^k$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } k < \log_b a \\ \Theta(n^k \log n) & \text{if } k = \log_b a \\ \Theta(n^k) & \text{if } k > \log_b a \end{cases}$$

- 例（斯特拉森矩阵乘法）： $T(n) = 7T\left(\frac{n}{2}\right) + dn^2$
 - $k=2, a=7, b=2$
 - $k < \log_2 7$
 - $T(n) = \Theta(n^{\log_2 7})$



主定理

- 简化形式：对形如 $T(n) = aT\left(\frac{n}{b}\right) + n^k$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } k < \log_b a \\ \Theta(n^k \log n) & \text{if } k = \log_b a \\ \Theta(n^k) & \text{if } k > \log_b a \end{cases}$$

- 例： $T(n) = 4T\left(\frac{n}{4}\right) + \sqrt{n}$
 - $k=1/2, a=4, b=4$
 - $k < \log_b a$
 - $T(n) = O(n)$



主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

- 例: $T(n) = 2T\left(\frac{n}{2}\right) + \log n$
 - $a=b=2$
 - $f(n) = \log n = O(n^{\log_2 2 - \epsilon})$
 - $T(n) = O(n)$



主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

- 例: $T(n) = 2T\left(\frac{n}{2}\right) + 2^n$

- $f(n) = 2^n = \Omega(n^{\log_2 2 + \epsilon})$

- 存在常数 $c < 1$ 和足够大的 n , 使得 $\frac{f(\frac{n}{b})}{f(n)} \leq c/a$

额外假设

- $T(n) = \Theta(2^n)$



主定理

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{if } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

- 例: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$

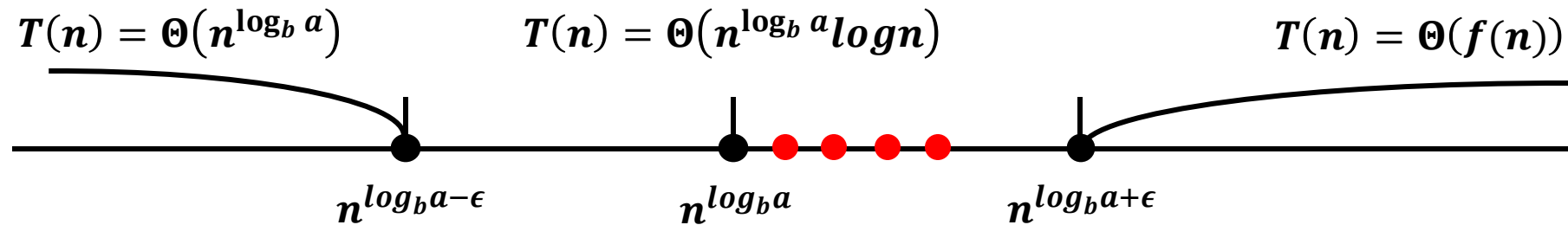
- $a=b=2$, $\log_b a = 1$
- $f(n) = n \log n = \Omega(n)$
- $\nexists c > 0$, 使得 $f(n) = \Omega(n^{1+\epsilon})$
- 不符合任何条件



主定理 (扩展形式)

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log^{k+1} n) & \text{if } f(n) = \Theta(n^{\log_b a} \log^k n), k \geq 0 \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$



主定理 (扩展形式)

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log^{k+1} n) & \text{if } f(n) = \Theta(n^{\log_b a} \log^k n), k \geq 0 \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

- 例: $T(n) = 2T\left(\frac{n}{2}\right) + n \log n$
 - $a=b=2, \log_b a = 1$
 - $f(n) = \Theta(n \log n)$
 - $T(n) = \Theta(n \log^2 n)$

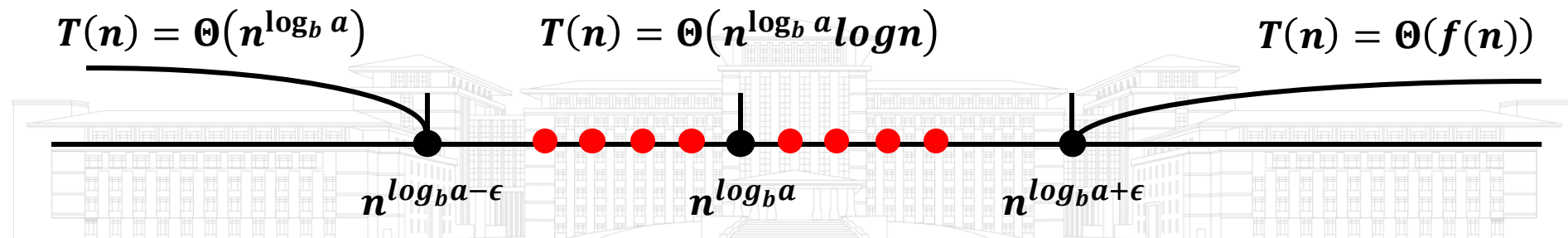


主定理 (扩展形式)

- 对形如 $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ 的递归式

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{if } f(n) = O(n^{\log_b a - \epsilon}) \\ \Theta(n^{\log_b a} \log^{k+1} n) & \text{if } f(n) = \Theta(n^{\log_b a} \log^k n), k \geq 0 \\ \Theta(f(n)) & \text{if } f(n) = \Omega(n^{\log_b a + \epsilon}) \end{cases}$$

$$T(n) = \begin{cases} \Theta(n^{\log_b a} \log^{k+1} n) & \text{if } k > -1 \\ \Theta(n^{\log_b a} \log \log n) & \text{if } k = -1 \\ \Theta(n^{\log_b a}) & \text{if } k < -1 \end{cases}$$



4.8 小结

- 分治法特点
 - 分-治-合
 - 子问题独立：两个子问题不会共享一个问题域更小的公共子问题
 - 自顶向下将原问题分解成子问题



4.8 小结

- 4.1 一般方法
 - 掌握分治法适用的问题特点、求解思想和计算时间等基本内容。能掌握分治法求解问题的一般过程。
- 4.2 二分查找
 - 二分查找成功、不成功情况下最好、最坏、平均时间复杂度
- 4.3 归并排序
 - 掌握分治算法的一般设计思想，算法正确性证明、时间复杂度分析和求解的一般方法。
- 4.4 快速排序
 - 随机化快速排序



4.8 小结

- 4.5 斯特拉森矩阵乘法
- 4.6 二维极大点问题
- 4.7 分治法的优化
 - 深入理解分治法适用的问题特征，掌握复杂问题求解办法。分析分治法时间复杂度的影响因素，掌握分治算法的优化技巧。掌握分治算法优化的一般原理和策略。
- 4.8 递归式分析方法
 - 掌握递归式求解的一般方法

能够识别出适合分治法的可计算性问题、独立设计算法和分析算法复杂度。